

INF01151 – Sistemas Operacionais II

Aula 01 - Introdução à disciplina

Prof. Alberto Egon Schaeffer Filho

Objetivos da disciplina

- Estudo de problemas clássicos de programação concorrente
 - Contém duas ou mais tarefas executando "*ao mesmo tempo*"
 - Aspectos importantes: **sincronização** e **comunicação**
 - Transportados para várias situações reais
- Apresentação de problemas comuns a várias áreas:
 - Sistemas operacionais
 - Sistemas distribuídos
 - Sistemas de tempo real

Histórico: programação concorrente

1962

- Controladores de dispositivos de hardware
- Possibilitam que CPU execute nova aplicação
 - Enquanto operações de I/O são executadas para programas suspensos
- Comunicação entre controladores e processador através de interrupções

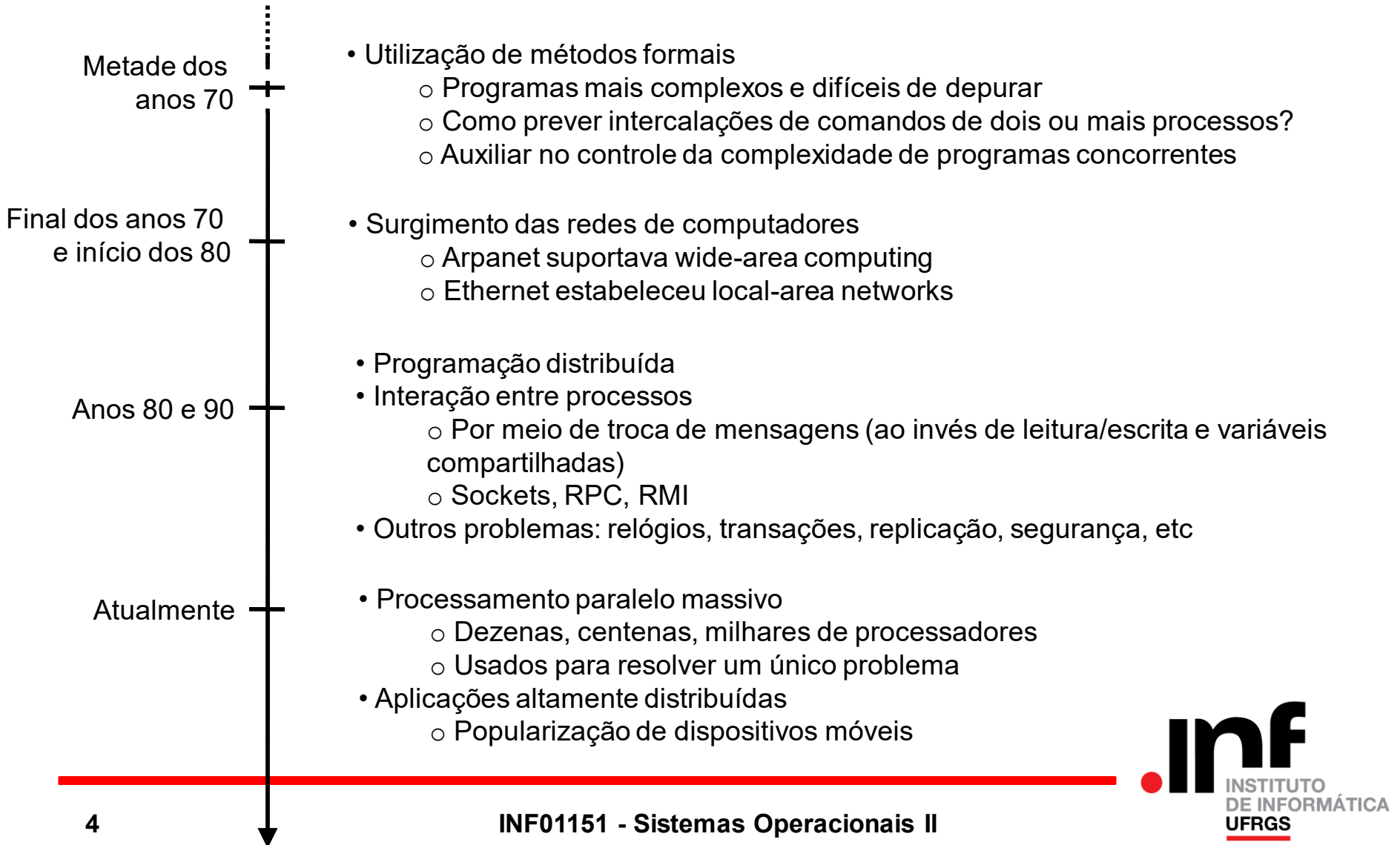
Concorrente = acontecendo ao mesmo tempo
Preocupação de projetistas de SO

- Primeiro grande desafio
- Problema da seção crítica
 - Se uma parte de um programa estiver atualizando uma variável, uma interrupção pode acontecer e outro processo pode tentar atualizar a mesma variável
- Desenvolvimento de primitivas de sincronização
 - Mutex, semáforos, monitores

Final dos
anos 60

- Surgimento de computadores com múltiplos processadores
 - Oportunidade para programadores de aplicações
 - Novo desafio para projetistas de SOs
- Diferentes programas executando ao mesmo tempo em diferentes processadores, ou
- Única aplicação executando mais rápido se re-escrita pra usar múltiplos processadores
 - Sincronização torna-se ainda mais difícil de gerenciar

Histórico: programação concorrente



Objetivos da disciplina (versão 2)

- Visão pragmática
 - Compreensão dos **conceitos fundamentais, mecanismos e construções** empregados para programação concorrente (do ponto de vista de sistemas operacionais)
 - Desenvolver **habilidade para solucionar problemas** de sincronização em programação concorrente
 - Compreender os **principais problemas** envolvidos no desenvolvimento de sistemas distribuídos
 - Conhecimento prático e uso das primitivas e **técnicas de programação** das atuais **linguagens de programação** concorrente

Súmula

- Conteúdo programático
 - **1ª Parte: Programação concorrente**
 - Introdução, especificação de concorrência, atomicidade e sincronização
 - Processos e threads, exclusão mútua, semáforos e monitores
 - Mecanismos de IPC (Inter Process Communication):
 - Tipos de comunicação (assíncrona/síncrona, RPC, RMI, web services)
 - **2ª Parte: Sistemas distribuídos**
 - Objetivos, conceitos de software e questões de projeto
 - Sincronização temporal, relógios físicos e relógios lógicos
 - Algoritmos distribuídos: eleição, exclusão mútua, deadlock
 - Comunicação em grupo e multicast
 - Outros tópicos: transações, replicação, segurança

Bibliografia

- Bibliografia base/essencial:
 - **Andrews, G.** – *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - **Silberschatz, A. and Galvin, P.** – *“Operating Systems Concepts”*. Addison-Wesley, 1994.
 - **Coulouris, G.; Dollimore, J.; Kindberg, T.** – *“Distributed Systems: Concepts and Design”* (5th edition), Addison-Wesley, 2012.
- Bibliografia complementar:
 - **Toscani, S. S.; Oliveira, R. S.; Carissimi, A.** – *“Sistemas Operacionais e Programação Concorrente”*. Sagra Luzzatto.

Introdução (light)

- Aspectos da computação concorrente
 - Multi-thread
 - Paralela
 - Distribuída
- Conhecimentos básicos
 - Conceitos de programação concorrente (INF01142)
 - Threads/Processos
 - Estados, execução, escalonamento
 - Problemas de sincronização
- Linguagens de programação
 - C & Java

Concorrência versus paralelismo

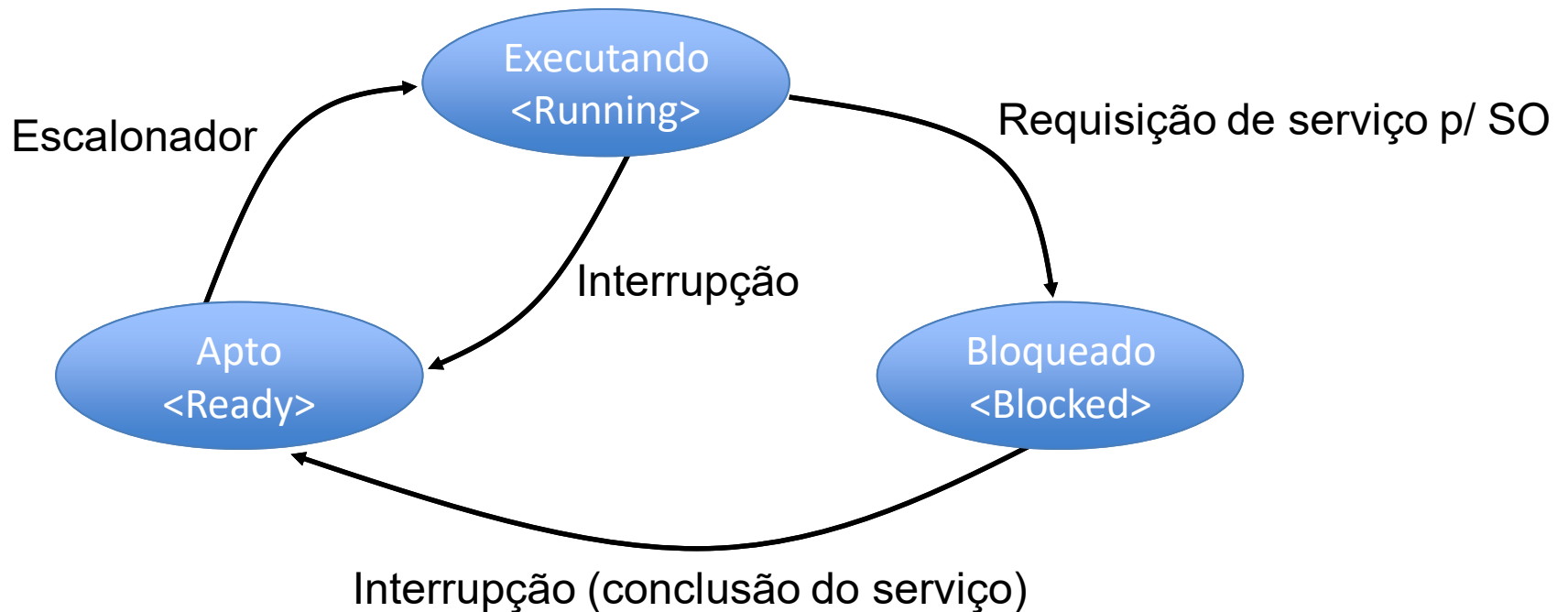
Concorrência $\neq$ Paralelismo

- Paralelo
 - Execução realizada fisicamente ao mesmo tempo
 - Com o objetivo de reduzir o tempo de resposta
 - Ex. simultaneamente em processadores paralelos
- Concorrente
 - Execução realizada logicamente ao mesmo tempo
 - Pode ser implementada sem nenhum paralelismo real

Multi-thread x paralela x distribuída

- Aplicação multi-thread
 - **O que é?** Possui mais de 1 thread (normalmente compartilhando CPU)
 - **Por que?** Melhor aproveitamento da CPU, boa forma de organizar sistemas de software modernos
- Aplicação paralela
 - **O que é?** Processos executando fisicamente ao mesmo tempo em diferentes processadores
 - **Por que?** Resolver um problema mais rapidamente -- ou resolver um problema maior
- Aplicação distribuída
 - **O que é?** Processos em diferentes *hosts* se comunicam através da rede
 - **Por que?** Distribuir trabalho, acessar dados inerentemente distribuídos, computação paralela em larga-escala sobre a rede

Estados de um processo



Escalonamento de processos

- Em arquiteturas Von Neumann os processos “parecem” executar ao mesmo tempo
 - Mas, na verdade, são intercalados (vale para threads também)
 - Fila de processos *ready* não é necessariamente FIFO...
- Escalonamento
 - Emprega conceitos de processo, chaveamento de contexto e política de escalonamento
 - *Dispatcher* passa o controle da CPU ao processo selecionado
 - Existem várias políticas
 - First-come, first-served
 - Shortest job first
 - Round robin
 - Priority

Escalonamento de processos

- Dois casos extremos
 - Não-preemptivo (cooperativo)
 - Um processo/thread executa até liberar o processador voluntariamente, ou terminar
 - Preemptivo
 - Um processo/thread é interrompido para permitir que outros processos/threads executem
- A grande maioria dos sistemas é preemptiva
 - Windows em versões recentes (Windows 3.x era não-preemptivo)
 - Mac OS X

Tipos de processos

- ~~Independentes~~
 - Não interessantes
- Competitivos
 - Típicos em sistemas operacionais modernos
 - Competem por recursos compartilhados da máquina
- Colaborativos
 - Processos executam para resolver um problema comum

Problemas comuns

- A competição leva:
 - Deadlock (impasses)
 - Starvation (inanição)
- Atomicidade
 - Uma ação dita atômica é aquela que tem a garantia de ser executada sem interrupção (do ponto de vista de outros processos)
 - A execução de processos/threads são intercaladas, como definir então ações atômicas?
 - Instruções individuais??
 - Blocos de instruções/código??

Exemplo de aplicação

- Produtor/consumidor
 - Frequentemente encontrado em programação concorrente
 - Principais componentes
 - Um processo produtor que produz itens
 - Um processo consumidor que consome itens
 - Um recurso compartilhado: objeto, buffer, banco de dados, etc
 - Principais problemas?
 - Itens produzidos podem ficar sem serem consumidos
 - Consumidor pode ler dados apenas parcialmente produzidos
 -
- Código exemplo em Java...

Exemplo de aplicação

Not looking so good...

```
Albertos-MacBook-Pro:ProducerConsumer alberto$ java ProducerConsumer
Consumed: 0 0 0
Produced: 0 0 0
Produced: 1 1 1
Consumed: 0 1 1
Produced: 2 2 2
Consumed: 2 2 2
Consumed: 3 3 2
Produced: 3 3 3
Consumed: 3 3 3
Produced: 4 4 4
Produced: 5 5 5
```

```
Albertos-MacBook-Pro:ProducerConsumerFixed alberto$ java ProducerConsumer
Produced: 0 0 0
Consumed: 0 0 0
Produced: 1 1 1
Consumed: 1 1 1
Produced: 2 2 2
Consumed: 2 2 2
Produced: 3 3 3
Consumed: 3 3 3
Produced: 4 4 4
Consumed: 4 4 4
```

Much better!

INF01151 – Sistemas Operacionais II

Aula 02 - Introdução à programação
concorrente

Prof. Alberto Egon Schaeffer Filho

Programação multi-thread x distribuída x paralela

- Programação multi-thread
 - Duas ou mais **tarefas** trabalhando na resolução de um problema
 - Pressupõe o paradigma de **memória compartilhada** (**shared variables**)
- Programação distribuída
 - Programa concorrente executado sobre um sistema computacional que possui **memória distribuída**
 - Comunicação através de **troca de mensagens** (**message-passing**)
 - Processo em um computador pode acessar (remotamente) dados em outros computadores
- Programação paralela
 - Programa concorrente, executando sobre *memória compartilhada ou memória distribuída*, com o objetivo de **reduzir tempo** de processamento
 - Executando “fisicamente” ao mesmo tempo

Introdução

- Tarefa:
 - Parte de um **conjunto de ações** para realizar um trabalho ou solucionar um problema
 - **Abstração** independente de modelo e linguagem de programação e de plataforma (hardware + software)
- As tarefas são implementadas através de:
 - Processos
 - Threads
 - Problemas de sincronização acontecem tanto em processos quanto em threads

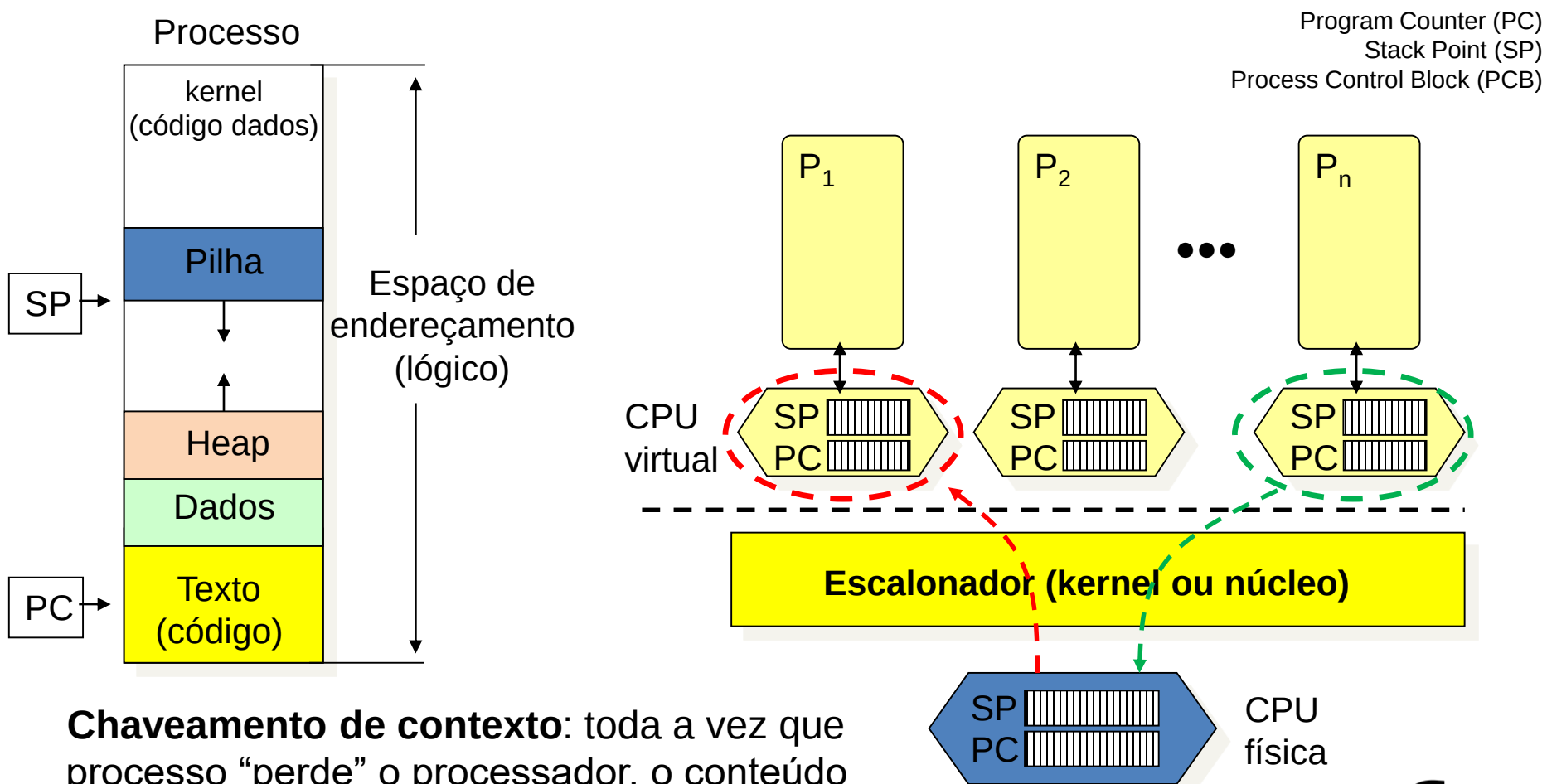
Criação de tarefas

- Questões de projeto: **Como? Por quem? Quando? Quantos?**
- Linguagens **imperativas** versus **declarativas**
 - **Imperativas**: criação é feita explicitamente pelo programador (como e quem)
 - Quando o programador quiser e quantos ele definir
 - **Declarativa**: a concorrência é implícita e partes do programa podem executar em paralelo, conforme determinado pelo compilador/runtime (como e quem)
 - Quando o compilador/runtime achar apropriado e quantos ele achar necessário
- Outro aspecto: **estático** versus **dinâmico**
 - **Estático**: programa declara um conjunto fixo de processos
 - **Dinâmico**: criados e terminados durante a execução por demanda

Representação de processo

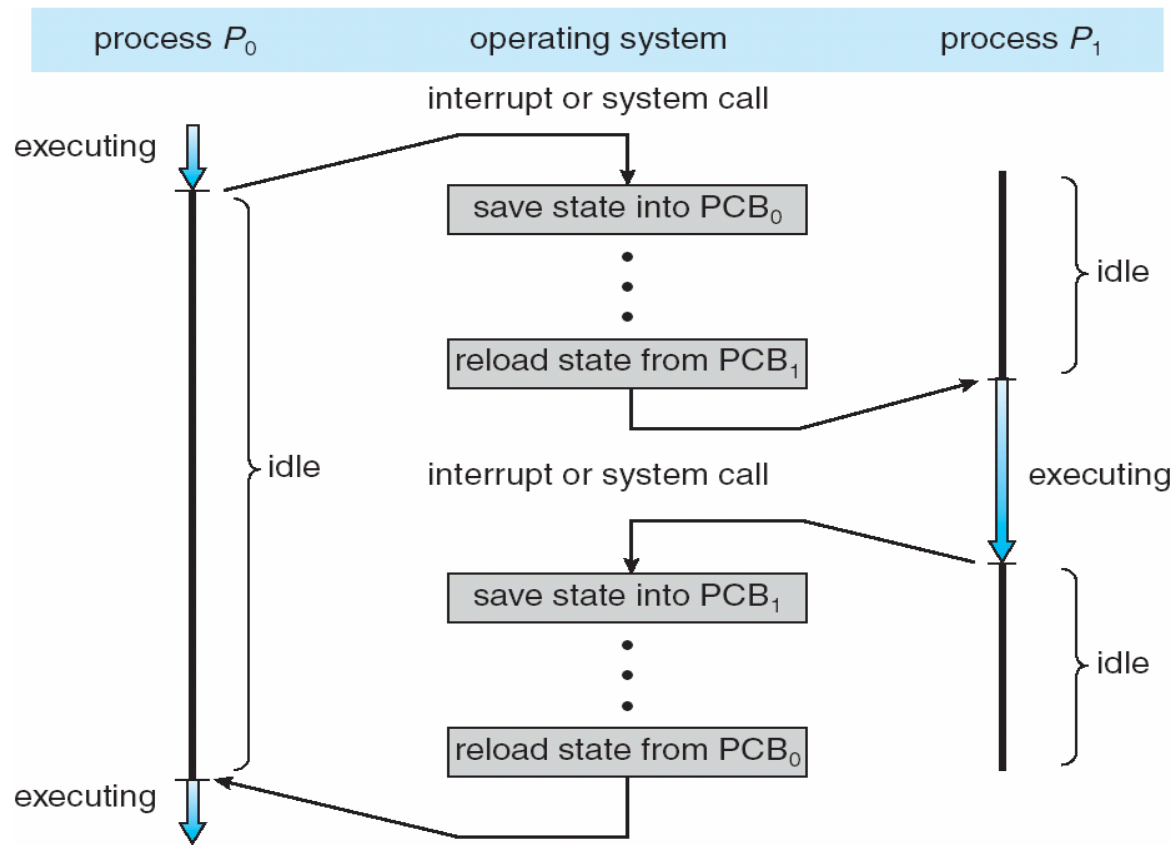
- Processo é um **programa em execução**
 - Programa é uma entidade *passiva*; processo é uma entidade *ativa*
 - Contém áreas na memória para código, dados e pilha
- Possui uma série de **estados (ready, running, blocked)** para representar sua evolução no tempo, implica:
 - Determinar eventos que realizam a transição entre os estados
 - Determinar quando um processo tem direito a “utilizar” o processador
 - Organizar processos nos diferentes estados em estruturas do kernel
 - E.g., **device** queue linked list, ready **queue** linked list, etc
- **Descritor específico** Process Control Block (PCB)
 - Usado para **manter informações** a respeito do processo
 - E.g., process state, program counter, CPU registers, CPU scheduling information, accounting information, memory-management information

A arte de enganar... processador como máquina virtual



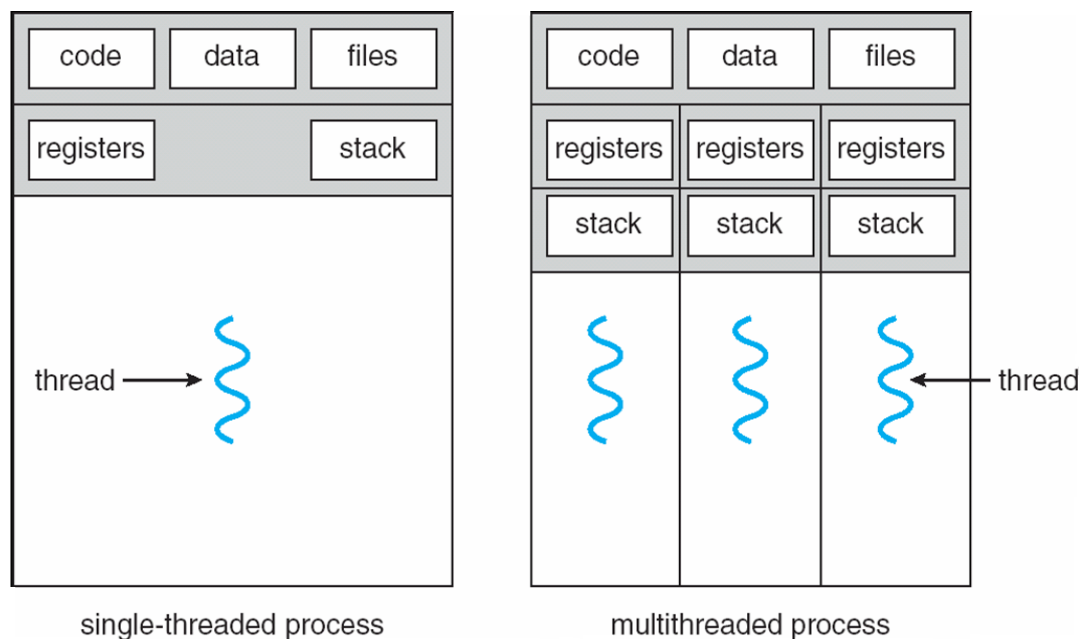
Chaveamento de contexto: toda a vez que processo “perde” o processador, o conteúdo dos registradores é salvo no seu descritor

CPU switch from process to process



Do modelo de processo para *multithreading*

- Thread:
 - Fluxo de execução independente dentro de um processo
 - Executam como parte de uma aplicação
 - Múltiplas partes da aplicação podem ser implementadas como threads separadas
 - Threads compartilham entre si os recursos de um processo



Cada thread contém:

- Thread ID
- Registradores (inc. PC)
- Stack

Compartilha com outras threads (mesmo processo)

- Code section
- Data section
- OS resources (files)

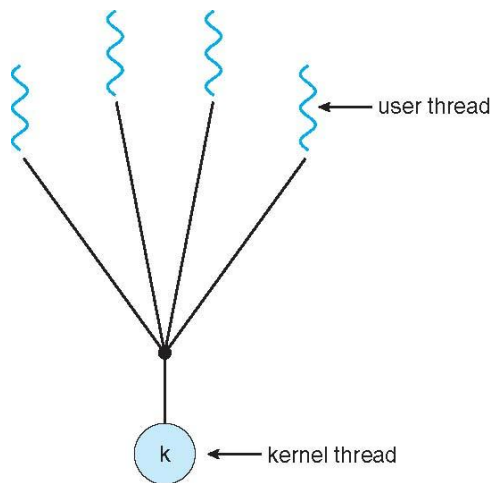
Recursos do sistema e as threads

- Gerência de threads é mais “leve”
 - Chaveamento de contexto entre duas threads de um mesmo processo é mais rápido, devido ao compartilhamento de recursos
- As threads dividem com os processos:
 - Espaço de endereçamento (código e dados)
 - Variáveis globais
 - Arquivos abertos, diretório corrente, sinais, processos filhos
 - Informações de contabilização, estatísticas do SO, registro descritor
- Porém, tem seus próprios recursos:
 - Pilha
 - Registradores, incluindo stack pointer (SP) e program counter (PC)

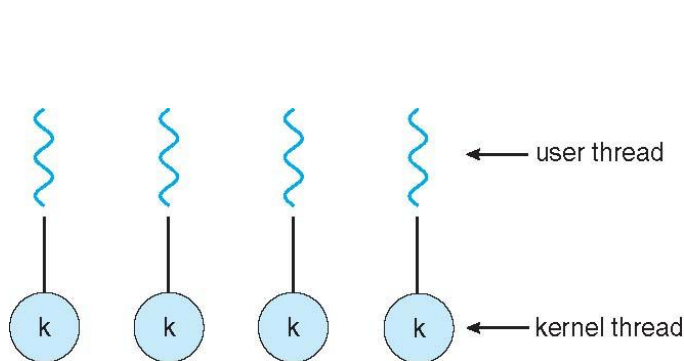
Modelos de suporte a multithreading

- Suporte para threads:
 - **User threads:** gerenciadas por *user-level libraries*, sem suporte do kernel
 - **Kernel threads:** gerenciadas diretamente pelo sistema operacional
- Deve existir uma relação entre user threads e kernel threads ...

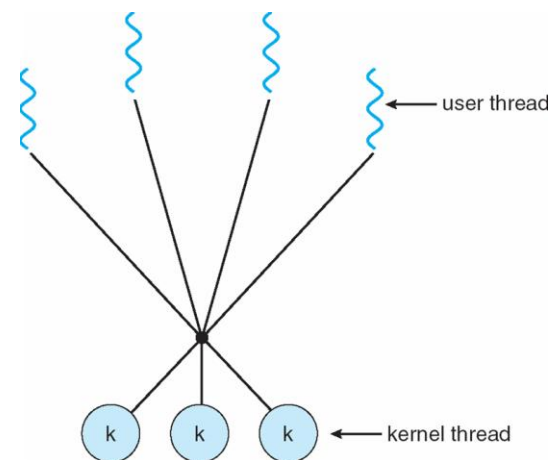
Many-to-One



One-to-One



Many-to-Many



Modelos de suporte a multithreading

- Suporte para threads:
 - **User threads:** gerenciadas por *user-level libraries*, sem suporte do kernel
 - **Kernel threads:** gerenciadas diretamente pelo sistema operacional
- Deve existir uma relação entre user threads e kernel threads ...

Many-to-One

✓ Gerenciamento de threads feito no espaço de usuário (mais eficiente)

✗ Processo inteiro irá bloquear se uma thread bloquear

✗ Múltiplas threads não irão executar em paralelo em multicore

One-to-One

✓ Maior concorrência, permite que outra thread execute se alguma fizer chamada bloqueante

✓ Permite execução paralela em múltiplos processadores

✗ Overhead: criar user-thread implica em criar kernel-thread (restringir # de kernel-threads suportado pelo sistema)

Many-to-Many

✓ Pode-se criar quantidade de user-threads que for necessária

✓ Kernel threads poderão executar em paralelo em multi-processadores

✓ Quando uma thread ficar bloqueada, kernel poderá escalonar outra thread

Programação concorrente

- Programação concorrente:
 - Duas ou mais tarefas realizadas em conjunto para efetuar um trabalho
 - Tarefa implementada como **processo** ou **thread**
- Dois aspectos da definição:
 - Cada tarefa é um “programa” sequencial
 - Programa sequencial: um único fluxo de controle (thread), portanto um programa concorrente possui múltiplos fluxos de controle
 - Trabalhar em conjunto implica:
 - Coordenação entre tarefas → sincronização
 - Troca de informações entre tarefas → comunicação

Comunicação entre tarefas

- Necessário para a produção de resultados finais consistentes
 - Programada utilizando variáveis compartilhadas ou troca de mensagens
 - Depende do paradigma de programação ...
- **Memória compartilhada**
 - Tarefas tem acesso a uma **mesma área de endereçamento**
 - Variável em memória: escrita e leitura em posições de memória
 - Comunicação **implícita**
- **Memória distribuída**
 - Tarefas possuem **áreas de endereçamento diferentes**
 - Necessário informar valores através do envio e recebimento de mensagens
 - Troca de mensagens: primitivas send e recv
 - Comunicação **explícita**

Tarefa = processo ou *thread*

Sincronização entre tarefas

- Necessário coordenar a execução de ações entre processos
 - Tanto em memória compartilhada como em memória distribuída
- Duas situações:
 - **Exclusão mútua (EM)**: duas ou mais tarefas não podem estar simultaneamente em uma **seção crítica**
 - **Condição (C)**: atrasar a execução de uma tarefa até que uma determinada condição seja verdadeira
- Exemplo: produtor-consumidor visto na última aula
 - Só pode obter dados de um buffer, se houver dados disponíveis **(C)**
 - Só escreve dados em um buffer, se houver espaço disponível **(C)**
 - Atualização e leitura consistente do objeto compartilhado **(EM)**

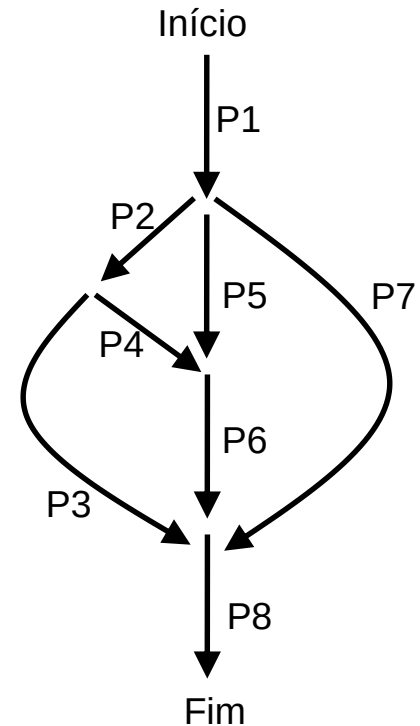
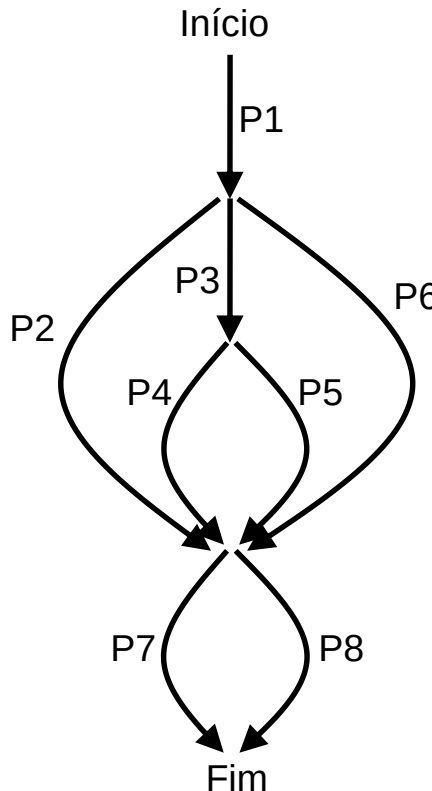
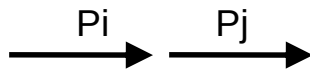
Tarefa = processo ou *thread*

Expressão da concorrência

- Forma de especificar a criação e controle de programas concorrentes
 - Grafo de precedência
 - Funções S e P
 - Operadores Cobegin/Coend
 - Fork/Join
 - Notação programação concorrente [Andrews, G.]

Grafos de precedência

- Grafos de precedência
 - Fornecer a ordem de execução de um conjunto de processos
 - Especificados através de um DAG (Directed Acyclic Graph)
 - Cada arco representa a execução de um processo
 - Um processo P_j só pode ser executado após P_i



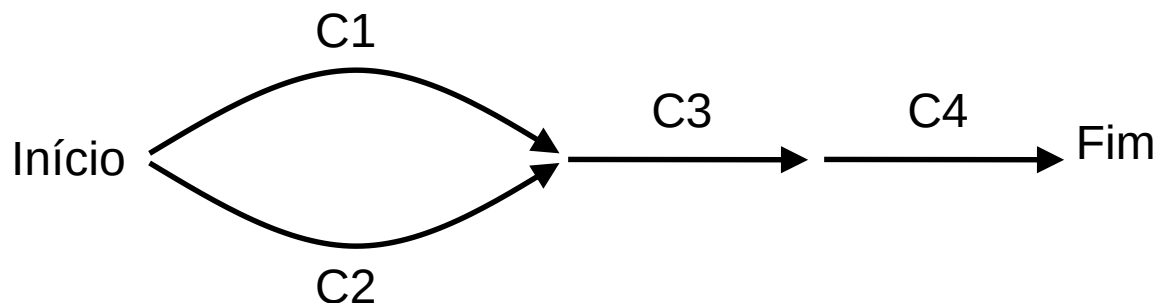
Normalmente utilizado em projeto, especificação, análise, etc

Exemplo de grafo de precedência

- Programa:

```
C1: a = x + y;  
C2: b = z + 1;  
C3: c = a - b;  
C4: w = c + 1;
```

- O comando C3 não pode ser executado antes de C1 e C2, pois C3 necessita dos novos valores das variáveis a e b. O comando C4 não pode ser executado antes que a variável c seja atualizada por C3.

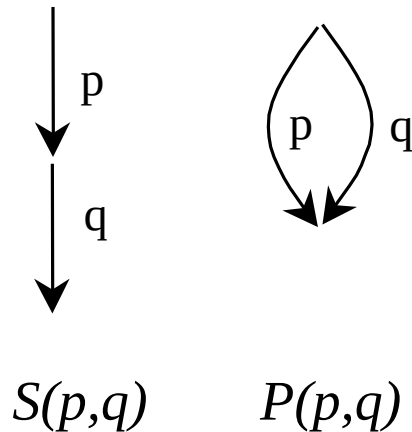


Limitações de grafo de precedência

- Não consegue representar adequadamente
 - Comunicação contínua entre duas ou mais tarefas
 - Condições usadas para criação/término de tarefas
 - Dificuldade de retratar de forma simples interações mais complexas
- Na literatura se encontram extensões ou outras propostas para contornar essas e outras limitações
 - Um dos aspectos considerados nessas extensões/propostas é fornecer capacidades para realizar prova formal de correção

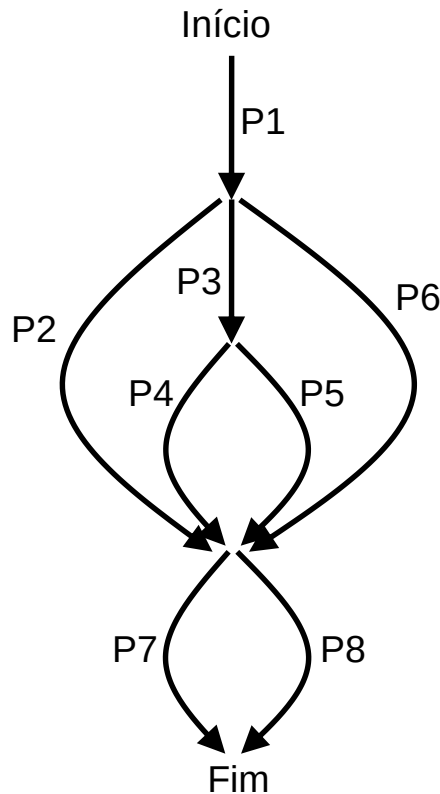
Funções S e P

- Expressam a relação de precedência entre os processos
 - $S(p, q)$: os processos p e q são executados em série
 - $P(p, q)$: os processos p e q são executados em paralelo



Grafo propriamente aninhado

- Aquele que pode ser expresso através da composição de S e P



Considera-se a figura de dentro para fora:

Passo 1: $P(p4, p5)$

Passo 2: $S(p3, P(p4, p5))$

Passo 3: $P(p2, S(p3, P(p4, p5)))$

Passo 4: $P(P(p2, S(p3, P(p4, p5))), p6)$

Passo 5: $S(p1, P(P(p2, S(p3, P(p4, p5))), p6))$

Passo 6: $P(p7, p8)$

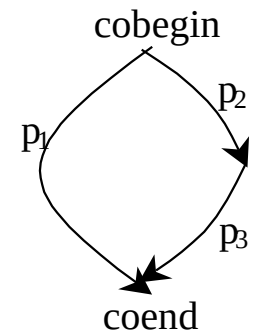
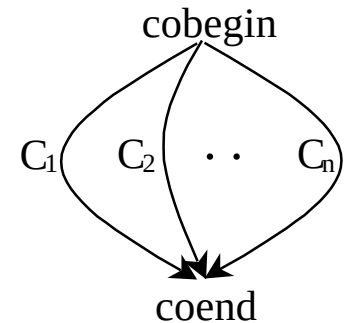
Passo 7: $S(S(p1, P(P(p2, S(p3, P(p4, p5))), p6)), P(p7, p8))$

Operadores cobegin/coend

- Propostos originalmente por Dijkstra (1965)
 - Chamados então de parbegin/parend
- Sintaxe:

cobegin C1 | C2 | ... | Cn coend

- Quando execução chega em cobegin
 - Criação de n tarefas independentes
- C_i é um trecho de código autônomo
- Quando todas as n tarefas terminam sua execução
 - Comando seguinte ao coend é executado
- Operadores “|” e “;”: equivalem a P e S, respectivamente
 - Precedência de “;” sobre |

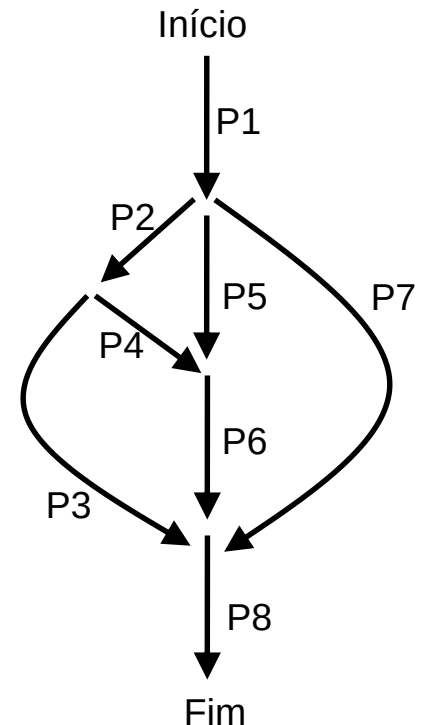


Exercício

- Represente o grafo abaixo utilizando a notação P e S (ou cobegin/coend)

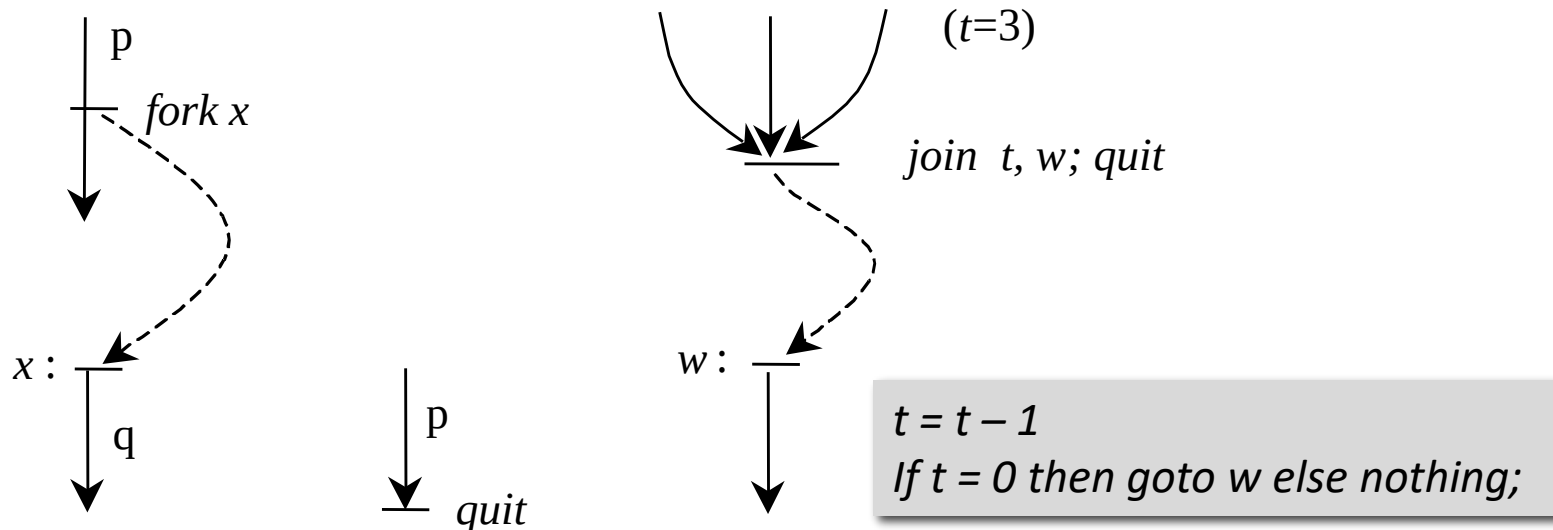
- Grafo **não é propriamente aninhado!!!**
- Não há como representar as dependências através de P e S (ou cobegin/coend)

Solução: Utilizar primitivas fork, quit e join...



Primitivas fork, join e quit

- Definidas por Conway (1963)
- Permitem descrever qualquer grafo de precedência
 - Implementadas em diversos sistemas operacionais e linguagens



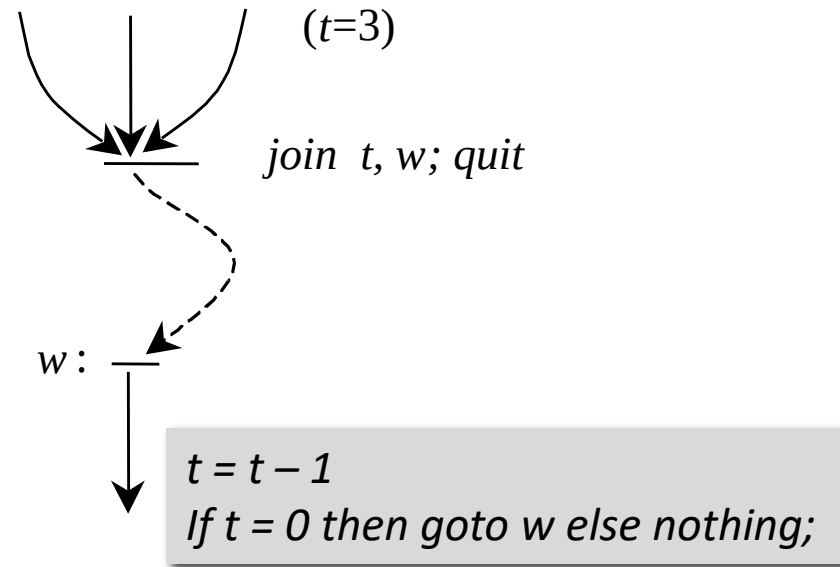
Operações *fork*, *quit* e *join*

Semântica fork()

- Produz duas execuções concorrentes em um programa
 - Uma execução inicia na instrução rotulada com x, enquanto a outra segue a instrução seguinte ao fork
- Permite a criação dinâmica de tarefas
- Área de dados (variáveis): duas alternativas possíveis
 - Pai e filho compartilham variáveis
 - Pai e filho possuem cópias privadas das variáveis
- Inconveniente:
 - Estrutura do tipo goto implícita na semântica da primitiva
 - Pouca legibilidade

Semântica join()

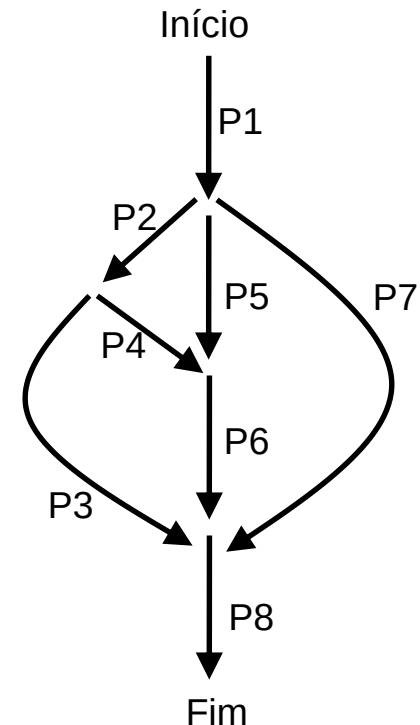
- Agrupa computações concorrentes em uma
 - Primitiva *join* é parametrizada com um contador t e um rótulo w
 - Cada processo que executa o join decrementa o contador
 - Quando contador chegar em zero, processo será direcionado para instrução rotulada em w
- Variações:
 - Agrupar n processos
 - Esperar pelo término de um processo P_k específico



Exemplo fork, join e quit

- Descrever o grafo de precedência utilizando primitivas fork, join e quit

```
t1:=2; t2:=3;  
p1; fork a2; fork a5; fork a7; quit;  
a2: p2; fork a3; fork a4; quit;  
a3: p3; join t2, a8; quit;  
a4: p4; join t1, a6; quit;  
a5: p5; join t1, a6; quit;  
a6: p6; join t2, a8; quit;  
a7: p7; join t2, a8; quit;  
a8: p8; quit;
```



Estudo de caso: fork, wait e exit no UNIX

- Semânticas diferentes: Atenção!!
- `fork( )` cria um **processo filho idêntico ao processo pai**, incluindo **cópias privativas das variáveis** do processo pai
 - Após o `fork( )` tem-se dois processos:
 - Processo pai: `fork` retorna o pid do filho
 - Processo filho: o `fork` retorna zero
 - Espaços de endereçamento distintos,
 - **Não há compartilhamento** de variáveis
 - Combinado com a chamada de sistema `exec( )`
- `wait(pid_t pid)` faz o processo que a executa esperar pelo término do processo pid
 - Equivalente ao `join`, mas com semântica diferente
- `exit(int status)` termina o processo

```
id = fork()
```

```
id = fork()
if (id == 0)
    then
        { proc. filho }
    else
        { proc. pai }
```


Notação programação concorrente (Andrews)

- Comando **co** (*concurrent*) especifica que dois ou mais comandos podem ser executados em paralelo (SR lang)
- Primeira forma:
 - Comandos são delimitados por co-oc e separados por //
 - Executa cada um dos comandos em paralelo e espera por eles para terminar
- Segunda forma:
 - Mesma semântica, apenas simplifica escrita usando quantificador
 - Executa em paralelo, n vezes, um conjunto de comandos

```
co statement 1;  
// ...  
// statement n;  
oc
```

```
co [i = 0 to n- 1] {  
    a[i] = 0; b[i] = 0;  
}
```

Multiplicação de matrizes: Notação Andrews

```
for [ i = 0 to n-1] {  
  for [ j = 0 to n-1] {  
    #compute inner product of a[i,*] and b[*],j  
    c[i,j] = 0.0 ;  
    for [k = 0 to n-1]  
      c[i,j] = c[i,j] + a[i,k]*b[k,j]  
    }  
  }  
}
```

Versão sequencial

```
co [ i = 0 to n-1] { #compute rows in parallel  
  for [ j = 0 to n-1] {  
    c[i,j] = 0.0 ;  
    for [k = 0 to n-1]  
      c[i,j] = c[i,j] + a[i,k]*b[k,j]  
    }  
  }  
}
```

Versão paralela (por linha)

Considerações finais

- Programa concorrente é mais difícil de ser implementado devido a **comunicação** e a **sincronização** entre tarefas
 - Programação distribuída e programação paralela são formas de programação concorrente
- Tarefas em programa concorrente podem ser:
 - Processos
 - Threads
- Expressão da concorrência
 - Grafo de precedência
 - Funções S e P
 - Operadores Cobegin/Coend
 - Fork/Join
 - Notação Andrews

Leituras adicionais

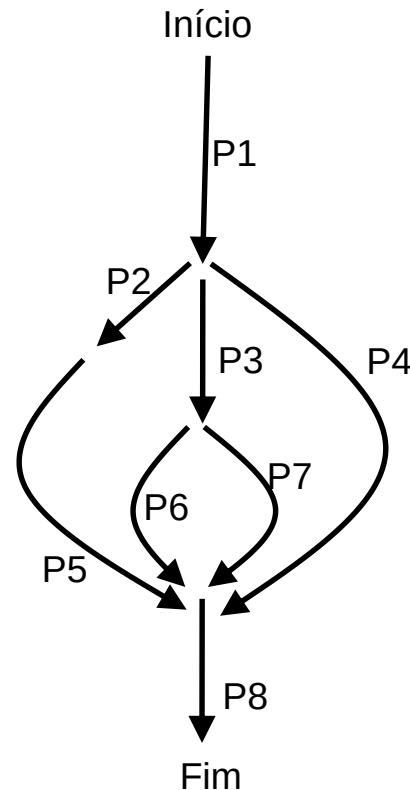
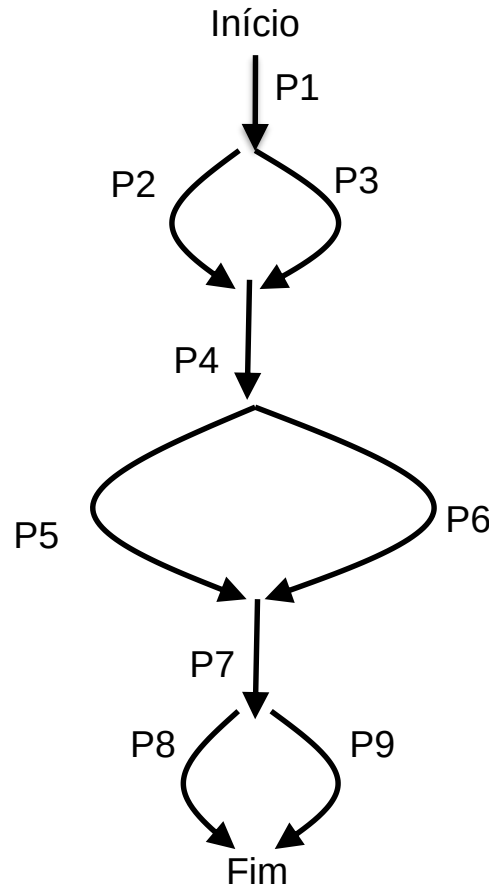
- Silberschatz, A.; Galvin, P.; Gagne, G. – *"Operating Systems Concepts"* (8th edition), John Wiley & Sons, 2009.
 - Capítulo 3: até seção 3.3 (inclusive)
 - Capítulo 4: até seção 4.2 (inclusive)
- Andrews, G. *"Foundations of Multithreaded, Parallel and Distributed Programming"*, Addison-Wesley, 2000.
 - Capítulo 1
- Toscani, S.T.; Oliveira, R.S.; Carissimi, A. *"Sistemas Operacionais e Programação Concorrente"*, Sagra-Luzzato, 2003.
 - Capítulos 2 & 3

Exercícios

1. Especifique um grafo de precedência que satisfaça as seguintes relações:
 - P2 e P3 só podem ser executados depois da execução de P1;
 - P4 só pode ser executado depois da execução de P2;
 - P5 e P6 só podem ser executados depois da execução de P4;
 - P7 só pode ser executado depois da execução de P5, P6 e P3;

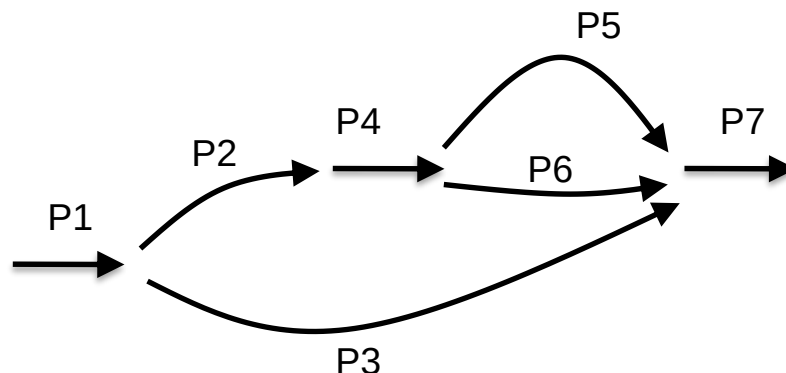
Exercícios

2. Utilizando as primitivas fork, quit e join (modelo clássico) especifique a execução dos processos de modo a satisfazer as relações de precedência



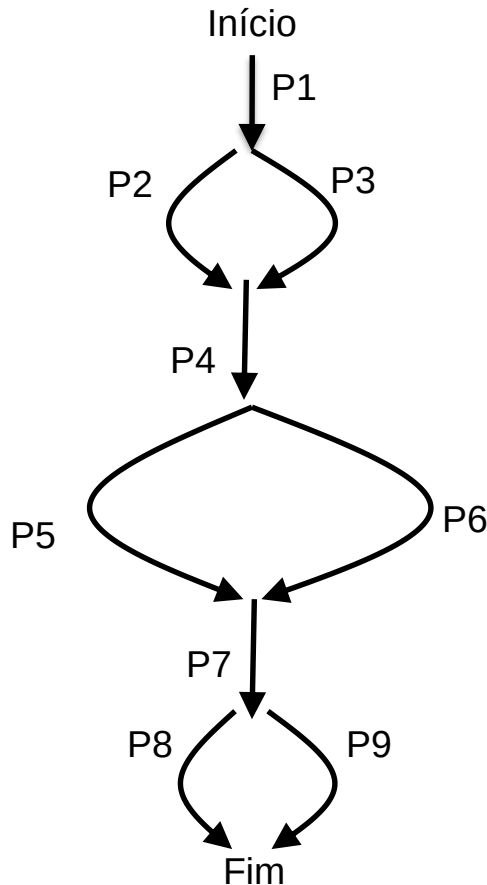
Exercícios (Respostas)

1. Especifique um grafo de precedência que satisfaça as seguintes relações:
- P2 e P3 podem ser executados depois da execução de P1;
 - P4 pode ser executado depois da execução de P2;
 - P5 e P6 podem ser executados depois da execução de P4;
 - P7 só pode ser executado depois da execução de P5, P6 e P3;



Exercícios (Respostas)

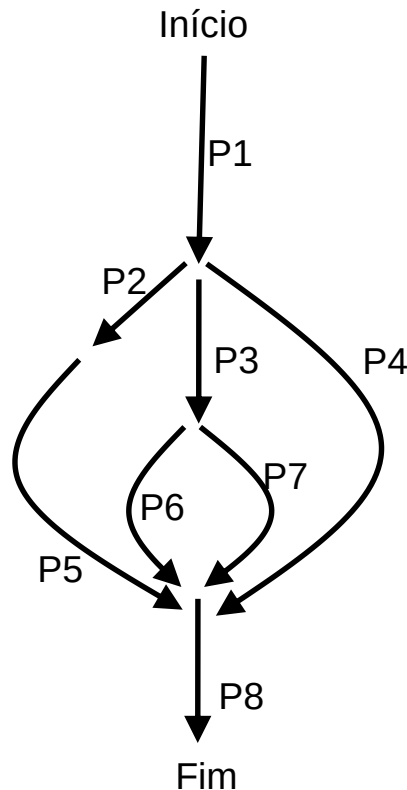
2. Utilizando as primitivas fork, quit e join (modelo clássico) especifique a execução dos processos de modo a satisfazer as relações de precedência



```
t1 := 2; t2 := 2;  
P1; fork a2; fork a3; quit;  
a2: P2; join t1, a4; quit;  
a3: P3; join t1, a4; quit;  
a4: P4; fork a5; fork a6; quit;  
a5: P5; join t2, a7; quit;  
a6: P6; join t2, a7; quit;  
a7: P7; fork a8; fork a9; quit;  
a8: P8; quit;  
a9: P9; quit;
```


Exercícios (Respostas)

2. Utilizando as primitivas fork, quit e join (modelo clássico) especifique a execução dos processos de modo a satisfazer as relações de precedência

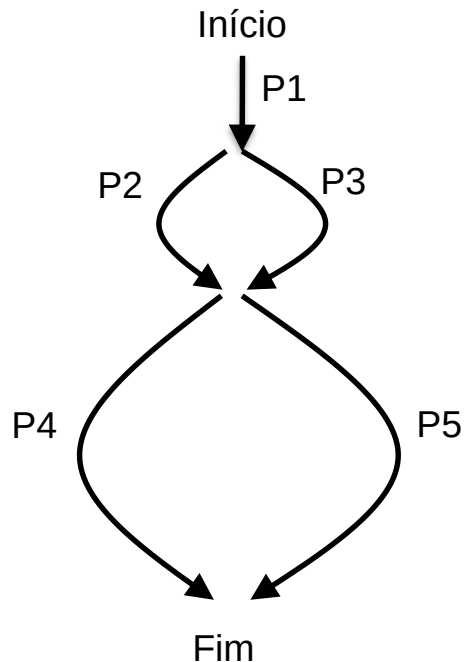


```
t1 := 4;  
P1; fork a2; fork a3; fork a4; quit;  
a2: P2; fork a5; quit;  
a3: P3; fork a6; fork a7; quit;  
a4: P4; join t1, a8; quit;  
a5: P5; join t1, a8; quit;  
a6: P6; join t1, a8; quit;  
a7: P7; join t1, a8; quit;  
a8: P8; quit;
```

Exercícios

Atenção – Fork logo após join

- O último processo a passar pelo join (quando t1 se tornará 0) será direcionado para ax, e executará os comandos fork para criação de P4 e P5



```
t1 := 2;  
P1; fork a2; fork a3; quit;  
a2: P2; join t1, ax; quit;  
a3: P3; join t1, ax; quit;  
ax: fork a4; fork a5; quit;  
a4: P4; quit;  
a5: P5; quit;
```

INF01151 – Sistemas Operacionais II

Aula 03 - Exclusão mútua

Prof. Alberto Egon Schaeffer Filho

Dois problemas para aquecimento...

1. Implementar em pseudo-código um programa que permita a operadores de viagem reservar e emitir bilhetes aéreos:
 - Faça apenas a função para reserva de assentos e emissão do bilhete, para um determinado assento
2. Usando a notação vista na aula passada, diga qual é o comportamento esperado para o seguinte trecho de programa:
 - Descreva todas as suas suposições

```
continue = true;  
co while(continue);  
// continue = false;  
oc
```

Tempo para a atividade: 10 minutos

Seção crítica

- Problema da reserva (compra) de uma passagem aérea
 - Agentes de viagem executam um código do tipo:

```
void reserveSeat(Position p) {  
    if (seat.free(p))  
        seat.reserve(p)  
}
```

- ...emitindo um bilhete aéreo para uma poltrona p

Execução possível

Operador A

```
void reserveSeat(Position p) {  
    → if (seat.free(p))  
        seat.reserve(p)  
}
```

Operador B

```
void reserveSeat(Position p) {  
    → if (seat.free(p))  
        seat.reserve(p)  
}
```

Qual é o valor final de counter?

- Considerando o trecho de código:

```
counter = 5;  
co counter++;  
// counter--;  
oc
```

counter++ <==>

```
MOV R1, counter;  
INC R1;  
MOV counter, R1;
```

counter-- <==>

```
MOV R2, counter;  
DEC R2;  
MOV counter, R2;
```

Execução 1:

```
T0: MOV R1, counter;  
T1: INC R1;  
T2: MOV counter, R1;  
T3: MOV R2, counter;  
T4: DEC R2;  
T5: MOV counter, R2;
```

Execução 2:

```
T0: MOV R2, counter;  
T1: DEC R2;  
T2: MOV counter, R2;  
T3: MOV R1, counter;  
T4: INC R1;  
T5: MOV counter, R1;
```

Execução 3:

```
T0: MOV R1, counter;  
T1: INC R1;  
T2: MOV R2, counter;  
T3: DEC R2;  
T4: MOV counter, R1;  
T5: MOV counter, R2;
```

Qual é o valor final de counter?

- Considerando o trecho de código:

```
counter = 5;  
do counter++;  
// counter--;  
oc
```

counter++ <==>

```
MOV R1, counter;  
INC R1;  
MOV counter, R1;
```

counter-- <==>

```
MOV R2, counter;  
DEC R2;  
MOV counter, R2;
```

Execução 1:

```
T0: MOV R1, counter;  
T1: INC R1;
```

Execução 2:

```
T0: MOV R2, counter;  
T1: DEC R2;
```

Execução 3:

```
T0: MOV R1, counter;  
T1: INC R1;
```

Condição de corrida: “Situação onde diferentes processos acessam e manipulam o mesmo dado concorrentemente e o resultado da execução depende da ordem em que os acessos acontecem.”

Como evitar? Sincronização de processos!

Introdução

- Unidades de execução: processo ou *thread*
 - Existe concorrência sempre que houver mais de uma unidade de execução no sistema
 - Concorrem (disputam) o recurso CPU (pelo menos)
- Execução independente ou cooperativa/competitiva
 - Se cooperativa/competitiva, duas ou mais unidades de execução necessitam interagir
 - Comunicar e sincronizar com seus pares

IMPORTANTE: Daqui para frente, o discurso é válido tanto para processos como para threads

Estados de um programa concorrente

- Estado de programa concorrente
 - Conjunto de valores das variáveis do programa em um instante de tempo
 - **Implícitas**: contador de programa (PC), apontador de pilha (SP), etc
 - **Explícitas**: variáveis declaradas pelo próprio programador
 - Programa inicia execução em algum estado inicial S_0
 - Cada processo do programa concorrente executa independentemente
 - Examina e altera o estado do programa
- Cada processo
 - Executa sequência de comandos
 - Comando = sequência de uma ou mais **ações atômicas** (indivisíveis), que examinam ou modificam estado do programa sem interrupções
 - Instruções de máquina que carregam ou armazenam palavras de memória são ações atômicas

Programa concorrente

- Execução de um programa concorrente
 - Resulta na intercalação de sequências de ações atômicas executadas por cada processo

P1: $C_{11} \rightarrow C_{12} \rightarrow C_{13} \rightarrow \dots \rightarrow C_{1n}$

P2: $C_{21} \rightarrow C_{22} \rightarrow C_{23} \rightarrow \dots \rightarrow C_{2n}$

- Uma execução em particular de um programa concorrente pode ser vista como um histórico

histórico: $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_n$

- Partindo de um estado inicial, S_0
- Cada estado subsequente é atingido por uma ação atômica que altera o estado

Programa concorrente

- Execução de um programa concorrente
 - Resulta na **intercalação de sequências de ações atômicas** executadas por cada processo
 - Cada intercalação irá produzir um histórico diferente
- Número de combinações possíveis de intercalações é enorme!!
 - Algumas intercalações podem ser **indesejáveis!!!**
- Objetivo
 - Limitar o número de históricos possíveis, eliminando os indesejáveis

Como fazer?

Sincronização de processos!

Sincronização de processos

- **Exclusão mútua**

- Composição de ações atômicas, implementadas diretamente por hardware, em um bloco de ações chamado seção crítica
- Seção crítica:
 - Conjunto de ações de um processo não podem ser intercaladas com ações de outros processos que também referenciem as mesmas variáveis
 - Ações “parecem” ser atômicas
 - Garante acesso exclusivo (sequencial) ao recurso

- **Sincronização condicional**

- Tarefa só pode prosseguir depois que uma condição seja satisfeita
 - Permite “atrasar” a execução de uma tarefa

- **Outros tipos... Barreira [mecanismo não tão popular]**

- Todas as tarefas sincronizam em um determinado ponto
 - As 1^{as} tarefas esperam pelas últimas na barreira

Propriedades de programa concorrente

- Propriedade
 - Atributo que é **verdadeiro em todos os históricos** possíveis do programa
 - Portanto, em todas as suas execuções
- Dois tipos de propriedades de programas
 - **Safety property**: programa nunca entra em um estado ruim
 - Estado onde algumas variáveis possuem valores indesejáveis
 - **Liveness property**: programa chegará a um estado bom
 - Estado onde variáveis tem valores desejáveis

Safety property

Exclusão mútua

Estado ruim é aquele em que dois processos estão executando ações nas seções críticas simultaneamente

Liveness property

Entrada na seção crítica

Estado bom para cada processo é aquele em que processo terá chance de estar executando dentro da seção crítica

Ações atômicas

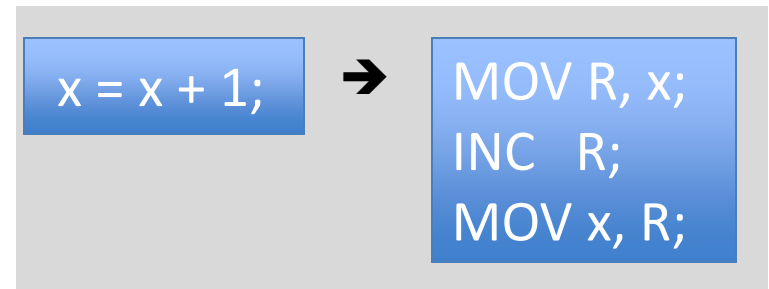
- Ação indivisível de uma tarefa: estados intermediários da ação não percebidos por outras tarefas
 - **Atomicidade fina**
 - Algumas ações são atômicas por hardware, por exemplo
 - Leitura e escrita de uma palavra de memória
 - **Atomicidade grossa**
 - Ações de sw normalmente não são atômicas
 - Se atomicidade for necessária
 - Uso de mecanismos de sincronização de exclusão mútua
 - Sequência de ações com atomicidade fina (hw) tornada atômica por mecanismos de sincronização
- Questão: verificar quando atomicidade grossa é necessária
 - Serializa tarefas, reduzindo vantagens da prog. concorrente

Ações atômicas

- Atribuição de valores parece ser atômica, mas geralmente não é
 - Implementado através de sequência de instruções de máquina

Valores possíveis de x?

```
int y = 0, z = 0;  
co x = y+z; // y = 1; z = 2; oc;
```



- Modelo de processador
 - Valores de tipos básicos de dados (p. ex. int) são armazenados em memória e lidos ou escritos de forma atômica
 - Para manipulação de valores, estes são carregados em registradores, manipulados, e armazenados na memória
 - Cada processo tem seu conjunto de registradores (lógico), salvo e restaurado através de chaveamento de contexto
 - Valores intermediários de expressões complexas são armazenados em registradores ou em posições de memória (pilha) privadas

Avaliação de expressões

- Com o modelo de máquina apresentado...
- Se expressão *e* em um processo não fizer referencia à variável alterada por outro processo, avaliação irá “*parecer*” atômica
 - Mesmo que necessite executar várias operações de máquina
 - Isto porque
 - Nenhum dos valores em que *e* depende poderia ter sido alterado
 - Nenhum outro processo pode ver valores temporários que possam ser criados durante avaliação da expressão
- Restrição forte
 - Porém, existe um requisito mais relaxado...

Propriedade at-most-once

- Referência crítica:
 - Variável em uma expressão e que pode ser modificada por outro processo
 - Suponha que variáveis são armazenadas em memória, lidas/escritas atomicamente
- Propriedade **at-most-once** em $x = e$
 1. A expressão e possui no máximo uma referência crítica e x não é lido por outro processo

ou

 1. A expressão e não possui qualquer referência crítica, e x pode ser lido por outro processo



Valores lidos dependerão da ordem de execução, mas serão sempre valores válidos!

Propriedade at-most-once: exemplos

Se propriedade for satisfeita, execução de atribuição parecerá ser atômica...

```
int x = 0; y = 0;  
co x = x + 1;  
// y = y + 1;  
oc OK
```

Sem referências críticas em qualquer processo

```
int x = 0; y = 0;  
co x = y + 1;  
// y = y + 1;  
oc OK
```

Um processo faz referência a *y*, mas *x* não é lido por outro processo. O valor final de *x* pode ser 1 ou 2, e o valor final de *y* é sempre 1.

```
int x = 0; y = 0;  
co x = y + 1;  
// y = x + 1;  
oc NOK
```

Cada expressão possui uma referência crítica e cada processo atribui a uma variável lida pelo outro. Os valores de *x* e *y* podem ser 1 e 2, 2 e 1 ou 1 e 1 (inválido).

Necessidade de sincronização

- Se expressão ou atribuição **não satisfizer propriedade at-most-once**, é necessário fazer com que ela execute atomicamente
- Genericamente, podemos precisar executar qualquer sequência de comandos como uma ação atômica
 - Precisamos de um mecanismo de sincronização para construir atomicidade grossa

Condição de corrida e seção crítica

- Condição de corrida (*race condition*)
 - Situação onde diferentes processos acessam e manipulam o mesmo dado concorrentemente
 - Resultado da execução depende da **ordem em que os acessos acontecem**
 - Alguns ordenamentos geram resultados inconsistentes
 - Ocorre quando o resultado de uma computação varia de acordo com as velocidades (ou “*timing*”) dos processos → **imprevisível!!!**

Operações		Conflito	Motivo
read	read	não	Operações de read independem da ordem em que são executadas
read	write	sim	O valor do read depende se é feito antes ou depois de uma escrita
write	write	sim	O valor final depende da ordem em que os writes são feitos

Solução para problema seção crítica: exclusão mútua

- Garantir acesso exclusivo de um processo à seção crítica
 - Se um processo está executando instruções da seção crítica, outro processo é impedido de entrar até que o primeiro saia
 - É o que se denomina de exclusão mútua
 - Resulta em uma **serialização** no acesso
- Primitivas para exclusão mútua: **Enter_EM** e **Exit_EM**



Primitiva de sincronização: await (Andrews)

- Notação:

`< await (B) S;>`

- Executa o comando S de forma indivisível (atômica) somente quando a expressão B (*boolean*) for verdadeira
- Condição para atrasar execução do processo especificada por condição B

- Propriedades:

- B garantidamente será true quando S começar a executar
- Nenhum estado interno em S é visível por outros processos

- Exemplo:

`< await (s > 0) s = s - 1; >`

- Espera até que s seja positivo, e então decrementa s
- Valor de s necessariamente será positivo antes de ser decrementado

Primitiva de sincronização: await (Andrews)

- Forma genérica que expressa exclusão mútua e sinc. condicional
 - Ambos, simultaneamente ou independentemente
- Para expressar apenas:
 - Exclusão mútua: omite o *await* (B)
 - Operador de atomicidade é “<” e “>”

```
<x = x + 1; y = y + 1;>
```

- Estado intermediário onde x foi incrementado mas y não foi, por definição, não é visível por outros processos
- Sincronização condicional: omite o S;

```
<await (count > 0);>
```

- Atrasa execução do processo até que count > 0

Se a expressão B atender propriedade **at-most-once** é possível implementar **<await (B); >** simplesmente como **while (not B);**

Exemplo: produtor/consumidor

- Situação comum em várias situações: buffer de comunicação, filas de impressão, etc...
- Princípio:
 - Produtor “produz” itens enquanto tiver capacidade de armazenamento.
 - Não pode exceder a sua capacidade
 - Consumidor “consome” itens enquanto houver disponibilidade
 - Não pode consumir o que não existe
- Variações:
 - n produtores, m consumidores, buffer unitário, buffer ilimitado, etc..
- Problema: **copiar todos os elementos** de um array, do produtor para o consumidor, utilizando um **buffer unitário** (int)
 - Buffer como variável compartilhada usada para comunicação entre P/C
 - Como sincronizar acesso ao buffer?

Exemplo: produtor/consumidor

```
int buf, p = 0, c = 0;

process producer{
    int a[n];
    while (p < n) {
        <await (p == c);>
        buf = a[p];
        p = p + 1;
    }
}

process consumer{
    int b[n];
    while (c < n) {
        <await (p > c);>
        b[c] = buf;
        c = c + 1;
    }
}
```

→ p e c armazenam o número de itens produzidos e consumidos, respectivamente

→ while (not (p == c));

Já que B referencia uma variável alterada por outro processo (*at most once*)



Spin lock ou busy waiting!!

Como implementar operações atômicas?

- Sem suporte de mecanismos de hardware, i.é., em software puro
 - Algoritmo de Dekker
 - Algoritmo de Peterson
 - Algoritmo de Lamport
- Com suporte de mecanismos de hardware
 - Baseado em habilitação/desabilitação de interrupções
 - Instruções atômicas (teste-and-set ou swap)

Leituras adicionais

- Andrews, G. *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - Capítulo 2 (seções 2.1 a 2.5)

Exercício

1. Considere os 3 comandos abaixo:

```
C1: x = x + y;  
C2: y = x - y;  
C3: x = x - y;
```

Assuma que x é inicialmente 2 e que y é inicialmente 5. Para cada um dos casos abaixo, quais são os valores finais possíveis para x e y? Explique.

- a) C₁; C₂; C₃;
- b) co <C₁> // <C₂> // <C₃> oc
- c) co <await (x > y) C₁; C₂> // <C₃> oc

INF01151 – Sistemas Operacionais II

Aula 04 - Exclusão mútua (implementação)

Prof. Alberto Egon Schaeffer Filho

O problema da seção crítica ...

- Seção crítica **Processo deve executar a seção crítica o mais rapidamente possível!**
 - Porção de código que não pode ser executado por dois ou mais processos simultaneamente sob pena de inconsistência de dados
- Necessário garantir acesso exclusivo de um processo à seção crítica
 - Se um processo está executando instruções da seção crítica, outro processo é impedido de entrar até que o primeira saia
 - É o que se denomina de **exclusão mútua**
 - Garantir acesso **exclusivo!**
 - Resulta em uma **serialização** no acesso
- Primitivas para exclusão mútua:
 - **Enter_EM**
 - **Exit_EM**



Como implementar?

- Primitiva genérica: `<await (B); S>`
 - Exclusão mútua e sincronização condicional
- Como implementar o operador exclusão mútua (“<” e “>”)?
 - Sem suporte de hardware, ou seja, em software puro
 - Algoritmo de Dekker
 - Algoritmo de Peterson
 - Algoritmo de Lamport
 - Com suporte de hardware
 - Baseado em habilitação/desabilitação de interrupções
 - Instruções atômicas (test-and-set e swap)

Propriedades para primitivas de exclusão mútua

(1) Exclusão mútua

- Garantir que no máximo um processo esteja executando dentro da seção crítica em um determinado instante

(2) Progressão

- Um processo fora da seção crítica não pode impedir outro processo de entrar

(3) Espera limitada

- Um processo não deve ser indefinidamente impedido de entrar na seção crítica

(4) Não deve fazer suposições sobre a velocidade relativa da execução dos processos, nem velocidade de processadores

Processo ou *thread*

Considerações gerais

- Para simplificar a análise:
 - Duas unidades de execução (threads ou processos)
 - Algoritmos podem ser adaptados para n threads ou processos
- Máquinas com um só processador (e um só core)
- Programas tem três partes:
 - Instruções críticas (modificam dados compartilhados)
 - Instruções não críticas (não atualizam dados compartilhados)
 - Primitivas para exclusão mútua (**Enter_EM** e **Exit_EM**)

Enter_EM

Seção
crítica

Exit_EM

Seção
não-crítica

Abordagem algorítmica

- Abordagem algorítmica é caracterizada por:
 - Não empregar serviços providos pelo núcleo (kernel) para bloquear, sinalizar ou retardar a execução de um processo
 - Não empregar instruções especiais do processador
- Vantagem:
 - É uma solução independente de sistema operacional e de plataforma de execução
- Desvantagem:
 - Emprego de espera ativa (busy waiting)
 - Complexidade de programação

Algoritmo de Dekker (versão I)

```
Var      turn: 1..2
Begin
    turn := 1;

Parbegin
    repeat
        while (turn = 2) do {nothing};
        { seção crítica};
        turn := 2;
        {seção não crítica}
    until (forever);
Parenend ;
End.
```

Problema?

Viola propriedade da progressão

```
repeat
    while (turn = 1) do {nothing};
    { seção crítica};
    turn := 1;
    {seção não crítica}
until (forever);
```

Algoritmo de Dekker (versão II)

```
Var      p1Inside, p2Inside: 0..1;
Begin
    p1Inside := 0;
    p2Inside := 0;
Parbegin
    repeat
        while (p2Inside = 1) do {nothing};
        p1Inside := 1;

        {seção crítica}

        p1Inside := 0;

        {seção não crítica}
    until (forever);
Parend;
End.
```

p1Inside e p2Inside são flags para indicar se o processo P_i está na região crítica. Antes de entrar na SC um processo verifica se a mesma livre (se o outro processo não está dentro dela).

```
repeat
    while (p1Inside = 1) do {nothing};
    p2Inside := 1;

    {seção crítica}

    p2Inside := 0;

    {seção não crítica}
until (forever);
```

Problema?

Viola regra da exclusão mútua

Algoritmo de Dekker (versão III)

```
Var      p1WantsToEnter,  
         p2WantsToEnter: 0..1;  
Begin  
    p1WantsToEnter := 0;  
    p2WantsToEnter := 0;
```

```
Parbegin
```

```
  repeat
```

```
    p1WantsToEnter := 1;  
    while (p2WantsToEnter = 1)  
      do {nothing};
```

{seção crítica}

```
    p1WantsToEnter := 0;
```

{seção não crítica}

```
  until (forever);
```

```
  Parend;
```

```
End.
```

Problema?

Viola regra da espera limitada
(*livelock*)

```
repeat
```

```
  p2WantsToEnter := 1;  
  while (p1WantsToEnter = 1)  
    do {nothing};
```

{seção crítica}

```
  p2WantsToEnter := 0;
```

{seção não crítica}

```
until (forever);
```

Algoritmo de Dekker (versão IV)

```
Var      p1WantsToEnter,
         p2WantsToEnter: 0..1;

Begin
    p1WantsToEnter := 0;
    p2WantsToEnter := 0;

Parbegin
    repeat
        p1WantsToEnter := 1;
        while (p2WantsToEnter = 1) do {
            p1WantsToEnter := 0;
            timewait(rand);
            p1WantsToEnter := 1;
        }
        {seção crítica}

        p1WantsToEnter := 0;
        {seção não crítica}
    until (forever);
Parend;
end.
```

Problema?

Um processo pode sempre dar preferência para o outro e nenhum deles ingressar na SC (problema do adiamento indefinido)

```
repeat
    p2WantsToEnter := 1;
    while (p1WantsToEnter = 1) do {
        p2WantsToEnter := 0;
        timewait(rand);
        p2WantsToEnter := 1;
    }
    {seção crítica}

    p2WantsToEnter := 0;
    {seção não crítica}
until (forever);
```

Algoritmo de Dekker (versão V)

```
Var turn: 1..2;  
    p1WantsToEnter, p2WantsToEnter: 0..1;  
Begin  
    p1WantsToEnter := 0;  
    p2WantsToEnter := 0;  
    turn := 1;  
Parbegin  
    repeat  
        p1WantsToEnter := 1;  
        while (p2WantsToEnter = 1) do  
            if (turn = 2) then {  
                p1WantsToEnter := 0;  
                while (turn = 2) do {nothing};  
                p1WantsToEnter := 1;  
            }  
        {seção crítica}  
        turn := 2; p1WantsToEnter := 0;  
    {seção não crítica}  
    until (forever);  
Paren.  
End.
```

Problema?

Dessa vez não... funcionamento está correto! Mas e para n processos?

```
repeat  
    p2WantsToEnter := 1;  
    while (p1WantsToEnter = 1) do  
        if (turn = 1) then {  
            p2WantsToEnter := 0;  
            while (turn = 1) do {nothing};  
            p2WantsToEnter := 1;  
        }  
    {seção crítica}  
    turn := 1; p2WantsToEnter := 0;  
    {seção não crítica}  
until (forever);
```


Algoritmo de Peterson (1981)

```
Var turn: 1..2;  
    p1WantsToEnter, p2WantsToEnter: 0..1;  
Begin  
    p1WantsToEnter := 0;  
    p2WantsToEnter := 0;  
Parbegin  
    repeat  
        p1WantsToEnter := 1;  
        turn := 2;  
        while (p2WantsToEnter = 1 and turn = 2)  
            do {nothing};  
  
        {seção crítica}  
  
        p1WantsToEnter := 0;  
  
        {seção não crítica}  
    until (forever);  
Parend.  
End.
```

Problema?

Também está correto... mas e para n processos?

```
repeat  
    p2WantsToEnter := 1;  
    turn := 1;  
    while (p1WantsToEnter = 1 and turn = 1)  
        do {nothing};  
  
    {seção crítica}  
  
    p2WantsToEnter := 0;  
  
    {seção não crítica}  
until (forever);
```

Algoritmos para n processos

- Algoritmo de Dijkstra (1965)
 - Possibilidade de adiamento indefinido
- Algoritmo de Knuth (1965)
 - Resolvía adiamento indefinido, mas permite longos atrasos
- Algoritmo de Eisenberg e McGuire (1972)
 - Entrada na seção crítica no máximo após $n-1$ tentativas
- Algoritmo de Lamport (1974)
 - Utiliza sistema do tipo “pegue uma ficha”, por isso chamado de *“algoritmo da padaria”*

Algoritmo da Padaria (Lamport, 1974)

```
const n;  
Var choosing = array[0..n-1] of boolean;  
    ticket = array[0..n-1] of integer;  
Begin  
  for j=0 to n-1 do {  
    choosing[j] := false;  
    ticket[j] := 0;  
  }
```

Parbegin Process **Pi**:

repeat

```
  choosing[i] = true;  
  ticket[i] := max(ticket[0]...ticket[n-1]) + 1;  
  choosing[i] := false;  
  for j = 0 to n-1 do {  
    while (choosing[j]) do {nothing};  
    while (ticket[j] ≠ 0 and (ticket[j], j) < (ticket[i], i)) do {nothing};  
  }
```

{Seção crítica}

```
  ticket[i] := 0;
```

{Seção não crítica}

until (forever);

Parend.

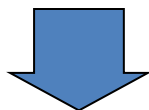
End.

Tie break (relação precedência)

$(\text{ticket}[j], j) < (\text{ticket}[i], i) \rightarrow$
 $(\text{ticket}[j] < \text{ticket}[i])$ or
 $(\text{ticket}[j] = \text{ticket}[i] \text{ and } j < i)$

Problemas com soluções em software

- Software puro: algoritmos de Dekker, Peterson e Lamport
 - Principais problemas?
 - **Complexidade**
 - **Problema de desempenho**
 - **Espera ativa** (busy wait)
 - **Falta de clareza** entre o que é **lógica de controle** de acesso a seção crítica e o que é da **lógica do programa**



Solução

Contar com o auxílio de mecanismos de hardware (projeto do processador)

Mecanismos de hardware

- Habilitação e desabilitação de interrupções
- Instruções específicas a serem usadas como base para se implementar primitivas de exclusão mútua:
 - Test and set
 - Swap



Unidades de execução: *thread* ou processos
(discurso válido para ambos!)

Desabilitando interrupções

- Preempção permite que múltiplos processos acessem dados compartilhados assincronamente
 - Preempção geralmente devida a ocorrência de **interrupções de tempo** (sinalizam expiração do quantum)
- Desabilitação de interrupções! Problemas?
 - Poder demais para um usuário! Se processo entrar em loop, não irá sair
 - Não funciona em máquinas multiprocessadoras, pois dois processos ainda podem executar ao mesmo tempo em processadores diferentes



Instruções especiais de hardware

- Desabilitar interrupções raramente é uma solução prática...
- Projeto de processadores levam em conta que eles serão usados em ambientes de programação concorrente
- Instruções assembly para leitura e escrita de posições de memória de forma atômica (indivisível)
 - **Test and Set: test-and-set (a, b)**
 - Lê o valor de uma posição de memória e coloca nela um valor não zero
 - **SWAP: swap (a, b)**
 - Permutação de valores

Instrução test-and-set

- Problema comum de sincronização: preempção de thread após ter lido valor de memória, mas antes de ter escrito novo valor
 - Verificar exemplos de exclusão mútua em software: thread precisa ler um valor para determinar que nenhum outro está na seção crítica, e atualizar uma variável de controle
 - Porém thread poderia sofrer preempção entre as duas operações!!
- Instrução test-and-set: habilita leitura de um valor e escrita de um novo valor atomicamente
 - Instrução em si não impõe exclusão mútua, mas pode ser usada por programadores para facilitar soluções

test-and-set(*a*,*b*)

- Lê valor de ***b*** (true|false). Copia valor para ***a*** e coloca o valor de ***b*** em true

test-and-set(*p1MustWait*, *occupied*)

Instrução test-and-set

- Problema comum de sincronização: preempção de thread após ter lido valor de memória, mas antes de ter escrito novo valor
 - Verificar exemplos de exclusão mútua em software: thread precisa ler um valor para determinar que nenhum outro thread pode escrever em uma variável de controle
 - Porém thread poderia sofrer preempção
- Instrução test-and-set: habilita uma thread a escrever um novo valor atomicamente
 - Instrução em si não impõe exclusão mútua, mas permite que programadores facilitem a implementação de soluções de exclusão mútua

```
while (p1MustWait)
{
    test-and-set(p1MustWait, occupied);
}
```

// código da seção crítica

```
p1MustWait = true;
occupied = false;
```

test-and-set(a, b)

- Lê valor de **b** (true|false). Copia valor p

test-and-set(p1MustWait, occupied)

Instrução swap

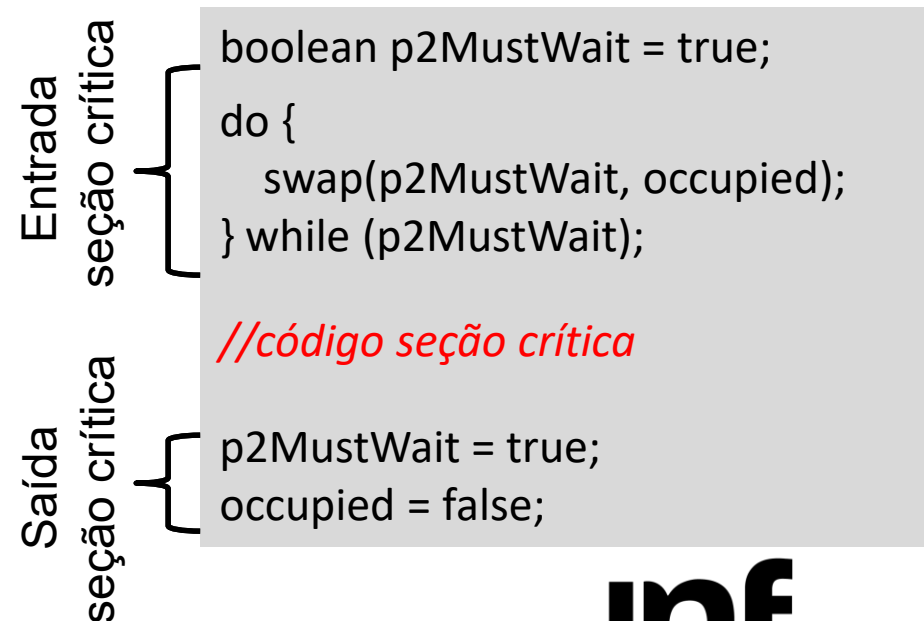
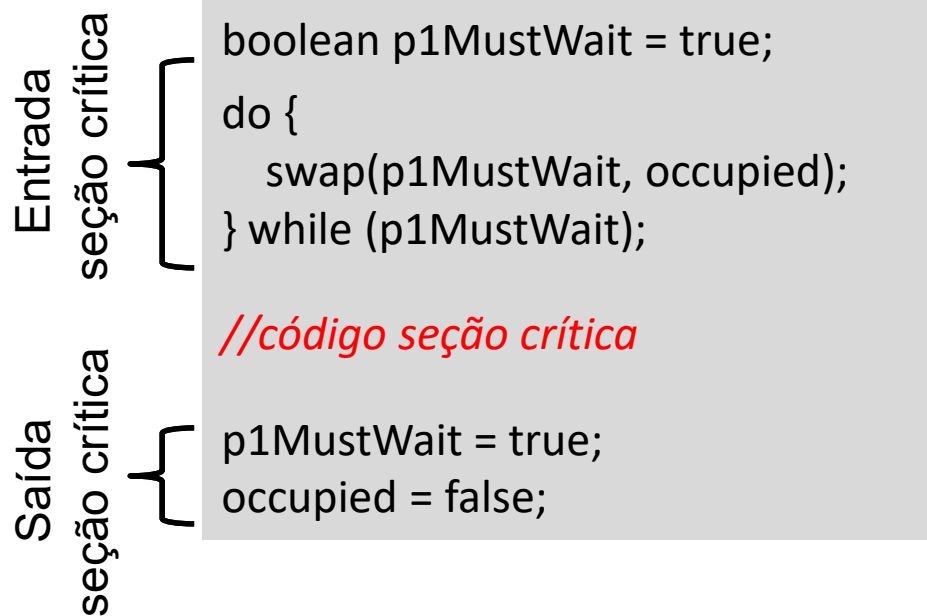
- Instrução Swap:

- Pode fornecer funcionalidade similar à instrução test-and-set

swap (a,b)

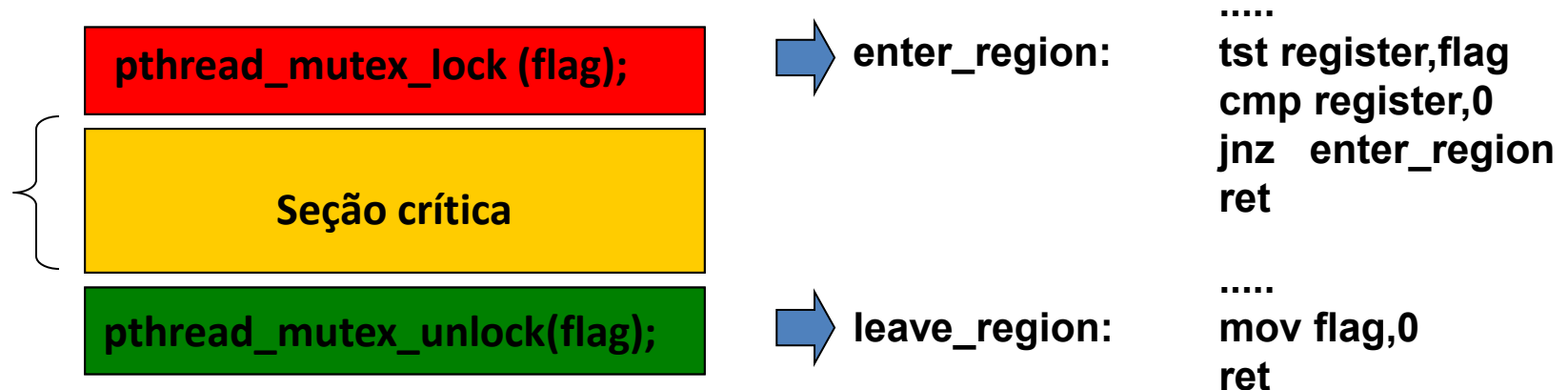
- Instrução carrega valor de **b** (true|false) em registrador **temporário**.
Copia valor de **a** em **b**. Copia valor do registrador **temporário** para **a**

```
tem = b;  
b = a;  
a = tem;
```



Primitivas *lock* e *unlock*

- Instruções especiais podem ser usadas para implementar primitivas de mais alto nível
 - Solução freqüentemente referenciada como ***mutex*** (***pthread_mutex_t***)



Problemas com a solução *lock/unlock*

- Uso correto das primitivas *lock* e *unlock*
 - Programador deve estar ciente (para não dizer “ser competente!”) de que é necessário liberar o acesso a seção crítica (*unlock*)
 - Esquecimento gera, normalmente, *deadlock*
- Espera ativa (*busy wait*)
- Inversão de prioridade

Problema e solução para a espera ativa

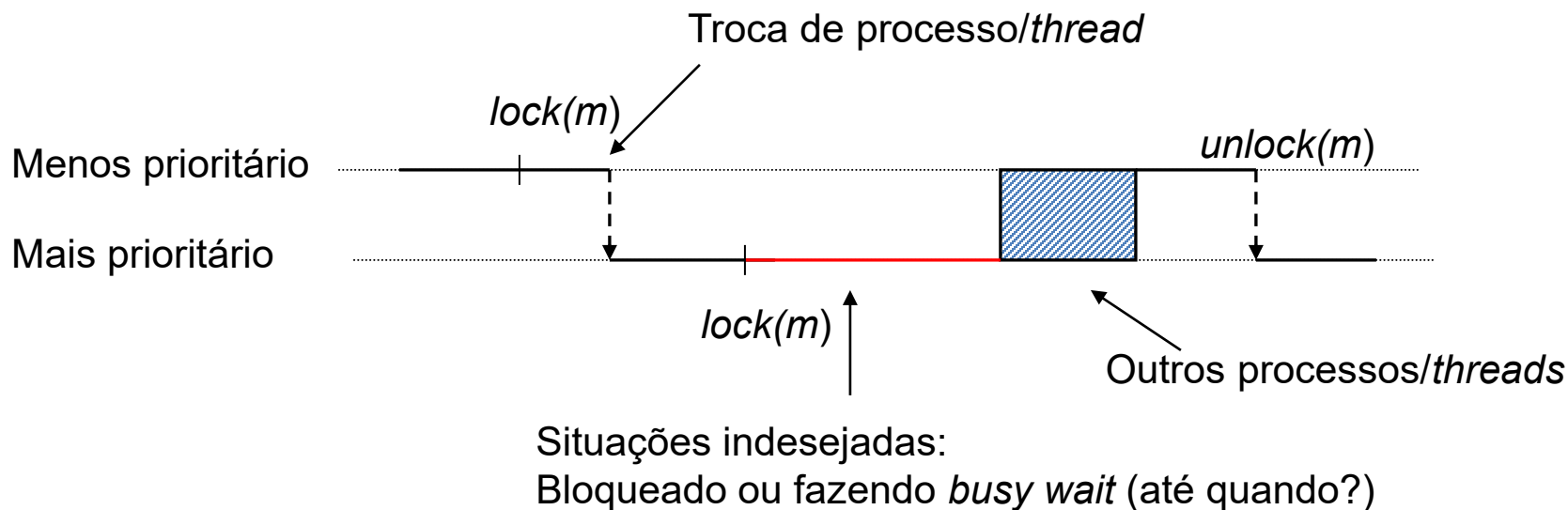
- Espera ativa ou *busy wait* (*spin lock*)
 - Sem sentido, pois para alguém liberar o recurso deve executar, mas para executar tem que usar a CPU
 - Argumento válido para monoprocessadores (*monocore*)
- Solução alternativa: bloquear o processo (*thread*)
 - Baseado em duas novas primitivas mais elaboradas
 - *sleep*: Bloqueia um processo a espera de uma sinalização
 - `pthread_cond_wait`
 - *wakeup*: Sinaliza um processo
 - `pthread_cond_signal`

Mas, espera ativa nem sempre é um problema...

- Em multiprocessadores (multicore) o *busy wait* pode ser interessante
 - Há outro “processador” para liberar o recurso
 - Custo de chaveamento **pode** não compensar (relação com o tamanho da seção crítica)
 - ✓ Porém viola a regra sobre “fazer suposições sobre processadores”
- Possibilidade: primitiva *trylock*
 - Faz *busy wait* por um período de tempo, se durante esse período o *mutex* não tiver sido liberado bloqueia (*sleep*)
 - Programador escolhe entre *trylock* e *lock*
 - Problema é dimensionar o período de tempo
 - Compromisso entre o tamanho da seção crítica, tempo de chaveamento de contexto e tempo estimado de espera

Problema: Inversão de prioridades

- Uma *thread* de mais baixa prioridade impedir a progressão de uma *thread* de mais alta prioridade



Soluções para inversão de prioridade

- Teto para prioridade (priority ceiling)
 - Aumento da prioridade de uma thread para um valor pré-determinado sempre que ela adquirir um mutex (lock)
 - Teto deve ser maior que a prioridade da maior thread
 - Na liberação (unlock) a thread retorna a sua prioridade original
- Herança (priority inheritance)
 - A prioridade da thread que detém o mutex é elevada ao nível da thread mais prioritária que solicita o recurso
 - Elevação da prioridade só ocorre se houver risco de conflito
 - Na liberação (unlock) a thread retorna a sua prioridade original

Leituras adicionais

- Deitel, Deitel e Choffnes, Sistemas Operacionais, 3a edição, Pearson Prentice Hall
 - Capítulo 5 (seções 5.1 a 5.5)

INF01151 – Sistemas Operacionais II

Aula 05 - Semáforos

Prof. Alberto Egon Schaeffer Filho

Introdução

- Sistema computacional pode oferecer diversos mecanismos para sincronização, além das variáveis tipo *lock* (mutex)
 - Semáforos
 - Variáveis de condição
 - Monitores
- Porque outros mecanismos?
 - Facilitar a programação
 - Oferecer primitivas mais flexíveis e robustas
 - Exclusão mútua não é a única necessidade de programas concorrentes

Semáforos

- Semáforos definidos por E.W. Dijkstra (1968)
- Diversas possibilidades de usos:
 1. Proteger seção crítica de código através de exclusão mútua
 2. Alocação de recursos
 3. Sincronização de condição
 - Uma operação só pode executar após outra ter sido executada
- Conceito e nome motivados pela forma como tráfego em ferrovia é sincronizado para evitar colisões
 - Indica se o **trilho** a frente está **livre ou ocupado** por outro trem
 - Semáforos podem estar **acessos ou apagados**

Semântica e primitivas de semáforos

- É uma estrutura de dados que contém:
 - **Variável protegida** (contador com valor não-negativo) acessada/modificada por *primitivas atômicas próprias*
 - Lista de **fluxos de execução** a espera do semáforo
 - Importante: não é especificada a política da lista (fifo, lifo, prioridade, etc)
- Duas primitivas atômicas próprias
 - **P (Proberen, testar)** → Também chamado de **wait**
 - Se valor do semáforo for positivo, deve decrementá-lo e processo segue adiante; caso contrário, bloqueia processo até a ocorrência de um evento
 - **V (Verhogen, incrementar)** → Também chamado de **signal**
 - Sinaliza a ocorrência de um evento, incrementando valor do semáforo

Semáforos devem ser inicializados antes de serem utilizados:

Configura o valor da variável protegida (**conforme o uso**) e cria uma estrutura que armazena referências a processos em espera

Implementação de semáforos (original – Dijkstra)

```
Var semaphore s;  
  
....  
  
procedure P(s)  
begin  
    if (s > 0)  
        then s = s - 1;  
        else {bloqueia processo P em s};  
end;  
  
procedure V(s)  
begin  
    if (existe processos bloqueados em s)  
        then {desbloqueia um processo}  
        else s = s + 1;  
end;
```

- Considerações:
 - Variável semáforo possui valor não-negativo
 - As operações *P* e *V* são atômicas
 - Implementadas pelo sistema operacional (sem condição de corrida)
 - Disponibilizadas via uma API
 - Diferentes implementações
 - Espera ativa (*busy wait*)
 - Processos são bloqueados

P(s): < await (s > 0) s = s – 1; >

1º uso: exclusão mútua

```
Var semaphore s = 1;  
  
Parbegin  
  repeat  
    P(s);  
    {seção crítica}  
    V(s);  
    {seção não crítica}  
  until (forever);  
Parend.  
End.
```

```
repeat  
  P(s);  
  {seção crítica}  
  V(s);  
  {seção não crítica}  
until (forever);
```

init_semaphore(s, 1);

P(s);

Seção crítica

V(s);

- Tipo especial de semáforo: ***semáforo binário*** (ou *mutex*)
 - Assume apenas dois valores (1 e 0)
 - Indica se a seção crítica está livre (1) ou ocupada (0)
 - Se o semáforo estiver ocupado, processo é bloqueado

Inconvenientes de semáforos para executar exclusão mútua

- Não há garantias quanto à propriedade de *espera limitada*
 - Na semântica de semáforos não há nenhuma definição quanto a ordem que processos são desbloqueados pela primitiva V
 - Embora **algumas implementações** possam garantir uma determinada política, por exemplo, **fifo**
- A correção do programa depende do bom uso de P e V
 - Pode levar o programa a *deadlock*

2º uso: alocação de recursos

- Tipo especial de semáforo: ***semáforo contador***
 - Variável protegida é inicializada com um **valor ≥ 1**
 - Usado para representar um conjunto de recursos idênticos
- Princípio básico
 - ***Alocar recursos = $P(s)$***
 - Enquanto houver recursos, é possível usá-los sem ser bloqueado
 - Na falta de recursos, processo é bloqueado até que um seja liberado
 - ***Liberar recursos = $V(s)$***
 - Ao terminar de usar uma unidade do recurso, incrementa o contador
- Caso particular: valor = 1 (*recurso unitário $\rightarrow$ exclusão mútua*)
 - Semáforo binário (semáforo assume apenas 0 ou 1)

Alocação de recursos com semáforos contadores

```
Var semaphore s = 3; //exemplo
```

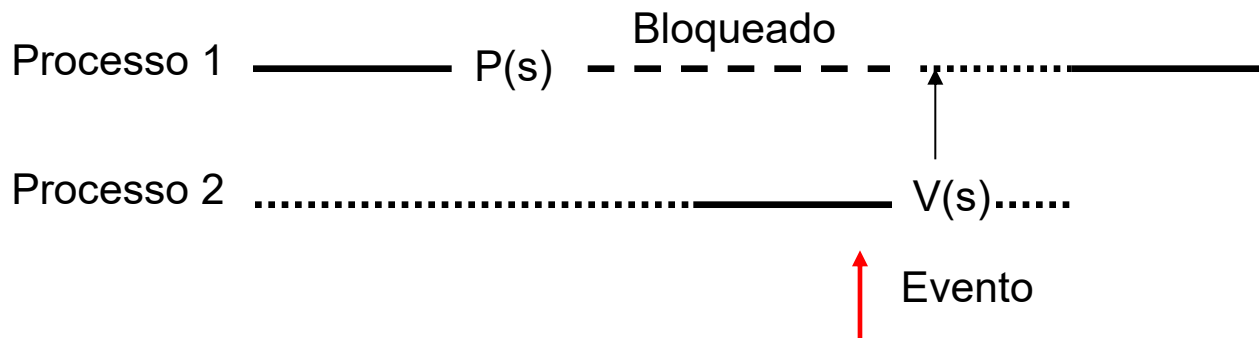
```
Parbegin
  repeat
    P(s);
    {acesso ao recurso}
    V(s);
    {restante do código}
  until (forever);
Parend.
End.
```

```
repeat
  P(s);
  {acesso ao recurso}
  V(s);
  {restante do código}
until (forever);
```

- Situação de uso:
 - Vários processos podem acessar recurso sem problema de consistência
- Emprego:
 - Inicializar uma variável semáforo com um valor c
 - n processos podem **usar recurso simultaneamente** ($1 \leq n \leq c$)

3º uso: sincronização de condição

- Notificar ou esperar a ocorrência de um evento particular
 - Primitiva **V** é também denominada de **signal**
 - Primitiva **P** é também denominada de **wait**
- Princípio de funcionamento:
 - Semáforo é inicializado com um **valor zero**
 - Para esperar o evento se faz um **P(s)**
 - Para sinalizar a ocorrência se faz um **V(s)**



Utilizando semáforos para sincronização de condição

- Situação de uso:
 - Quando um processo deve realizar uma **operação a_2** somente **após** um outro processo ter executado uma **operação a_1**
- Emprego:
 - Semáforo inicializado com **zero**
 - Um processo **espera** por um evento executando **$P(s)$**
 - Outro processo **sinaliza** um evento executando **$V(s)$**

```
Var semaphore s = 0
Parbegin
    ....
    P(s);
    executa operação a2
    ....
Parend.
End.
```

```
....
executa operação a1
V(s);
....
```

Resumo: uso de semáforos em programação concorrente

Uso	Descrição
Exclusão mútua	Obtido através de um semáforo inicializado em 1. Um processo realiza uma operação $P(s)$ antes de entrar na seção crítica e uma operação $V(s)$ no momento de sair. Tipo especial de semáforo (binário) também denominado de <i>mutex</i> .
Alocação de recursos	Permite o uso concorrente de um recurso por até n processos, ou seja, $1 \leq n \leq c$, onde c é uma constante. Um semáforo inicializado com c é usado para implementar a limitação de concorrência. São os semáforos denominados de <i>contadores</i> .
Sincronização de condição	Quando um processo necessita realizar uma operação a_2 somente após um outro processo ter realizado uma operação a_1 . Semáforo é inicializado em 0. Forma de sincronização também conhecida por <i>senalização</i> .

Exemplos de uso de semáforos: clássicos da programação concorrente

1. Jantar dos filósofos
2. Produtor-consumidor
 - a) Buffer unitário
 - b) Buffer limitado (com capacidade n)
3. Leitores e escritores

Jantar dos filósofos

- Considere um grupo de N filósofos que passam seu tempo **pensando** ou **comendo spaghetti**, mas obedecem algumas regras
 - Sentados em uma mesa circular
 - Existem apenas N garfos
 - São necessários dois garfos para comer
 - Filósofo só pode utilizar os garfos imediatamente de sua direita ou esquerda

```
Process Philosopher [int i=0 to N-1] {  
    while (true) {  
        Think  
        Acquire forks;  
        Eat;  
        Release forks;  
    }  
}
```



Objetivos

- Jantar dos filósofos ilustra problemas de concorrência
 - **Alocação de recursos:** garfos para filósofos
 - **Exclusão mútua:** acesso aos garfos
 - **Sincronização condicional:** só pode comer quando tiver os dois garfos
- Necessidade de evitar
 - Livelock
 - Deadlock
 - Starvation
- Uma solução: cada **garfo** é representado por um **semáforo**
 - Um filósofo tenta **adquirir um garfo** executando **P**
 - Um filósofo **libera um garfo** executando **V**

Primeira tentativa

```
N = 5
Var semaphore forks[N] = {1,1,1,1,1}

Process Philosopher [int i=0 to N-1] {
    int left = i;
    right = (i+1)%N;

    while (true) {
        Think;
        P(forks[left]);
        P(forks[right]);
        Eat;
        V(forks[left]);
        V(forks[right]);
    }
}
```

Problema?

Suponha que todos os filósofos fiquem com fome ao mesmo tempo, e que cada um consiga pegar seu garfo à esquerda. Quando um filósofo tentar pegar seu garfo à direita, ficará bloqueado para sempre (*deadlock*)

Deadlock

- Se todos os filósofos pegam o garfo da esquerda, ocorre um impasse entre eles (*deadlock*)
 - Deadlock pode ocorrer quando há uma espera circular de recursos (**mais tarde**: necessário 4 condições)
- Diversas soluções possíveis...
 - ① **Quebrar o ciclo**: um filósofo pega primeiro o garfo da direita, enquanto que os outros pegam primeiro o garfo da esquerda
 - ② **Controlar alocação de cadeiras**: permitir que no máximo $N-1$ filósofos sentem simultaneamente na mesa
 - ③ **Garantir aquisição de ambos os garfos**: permitir que um filósofo pegue os seus garfos somente se ambos estiverem disponíveis

Solução 1

- Um filósofo pega o garfo da direita e depois da esquerda, e os demais pegam primeiro da esquerda e depois o da direita

```
Process Philosopher [int i=0 to N-1] {  
    int left, right;  
  
    if (i==0) {  
        left = 1;  
        right = 0  
    }  
    else {  
        left = i;  
        right = (i+1)%N;  
    }  
    ...  
}
```

Solução 2

- Se no máximo $N-1$ filósofos tentarem comer haverá sempre um que terá a chance de pegar dois garfos
- Deve utilizar um semáforo adicional para representar o número máximo de filósofos autorizados a comer simultaneamente ($N-1$)

```
Var semaphore chairs =  $N-1$ ;  
...  
P(chairs) ;  
P(forks[left]);  
P(forks[right]);  
//Eat  
V(forks[left]);  
V(forks[right]);  
V(chairs) ;  
...
```

Produtor-consumidor

(de novo... mas agora com semáforos!!)

- É uma relação bastante comum entre processos
- Problema fundamental:
 - Como permitir diferentes taxas de produção e consumo sem haver problemas?
 - Não deve consumir o que não existe
 - Não deve produzir sobre o que já existe
- Variantes:
 - Buffer unitário, buffer limitado com capacidade n
 - 1 produtor e 1 consumidor / n produtores e m consumidores

Buffer com slot unitário

- Situação:
 - Múltiplos consumidores e produtores
 - Apenas um slot
- Duas razões para bloquear:
 - Produtores **esperam** que o *slot* esteja **vazio** para “produzir”
 - Consumidores **esperam** que o *slot* esteja **cheio** para “consumir”
- Solução emprega dois semáforos binários:
 - *empty* inicializado em 1
 - *full* inicializado em 0

```
Var semaphore empty = 1;  
    semaphore full   = 0;
```

```
Data buf; // buffer unitário
```

Buffer com slot unitário

```
Var semaphore empty = 1;  
    semaphore full = 0;
```

```
Data buf = null;
```

```
Process Producer[i=1 to M] {  
    while (true) {  
        //produce data  
        P(empty) ;  
        buf = data;  
        V(full) ;  
    }  
}
```

```
Process Consumer[j=1 to N] {  
    Data myData;  
    while (true) {  
        P(full) ;  
        myData = buf;  
        V(empty) ;  
        //Consume myData  
    }  
}
```

split binary semaphore: no máximo um entre *empty* ou *full* é 1, a qualquer momento (invariante)

Buffer limitado com capacidade n

- Uso de semáforo como contador de recursos
- Inicialização:
 - Empty – número de slots vazios = n
 - Full – número de slots cheios = 0
- Invariante:
 - $\text{empty} + \text{full} = n$

```
Var semaphore empty = n;  
    semaphore full  = 0;  
  
Data buf[n];  
int front = 0, rear = 0;
```


Buffer limitado com capacidade n: um único produtor e um único consumidor

```
Var semaphore empty = n;  
    semaphore full = 0;  
  
Data buf[n];  
int front = 0, rear = 0;
```

```
Process Producer {  
    while (true) {  
        //produce data  
        P(empty) ;  
        buf[rear] = data;  
        rear = (rear + 1) % n;  
        V(full) ;  
    }  
}
```

Exemplo: uso de semáforos para alocação de recursos

- Empty representa número de slots vazio
- Full representa número de slots cheios

```
Process Consumer {  
    while (true) {  
        //fetch data  
        P(full) ;  
        result = buf[front];  
        front = (front + 1) % n;  
        V(empty) ;  
    }  
}
```

Buffer limitado com capacidade n: múltiplos produtores

```
Var semaphore empty = n,  
    semaphore full = 0;  
    semaphore mutexP = 1, mutexC = 1;  
    int front = 0, rear = 0;
```

```
Process Producer [int i=1 to M] {  
    while (true) {  
        //produce data  
        P(empty);  
        P(mutexP);  
        buf[rear] = data;  
        rear = (rear+1)%n;  
        V(mutexP);  
        V(full);  
    }  
}
```

Com múltiplos produtores há disputa para usar e atualizar a variável *rear* (seção crítica)

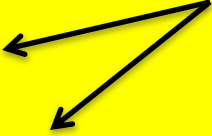
- Produtores e consumidores podem executar concorrentemente
- Mas dois produtores (ou dois consumidores) não

Buffer limitado com capacidade n: múltiplos consumidores

```
Var semaphore empty = n,  
    semaphore full = 0;  
    semaphore mutexP = 1, mutexC = 1;  
    int front = 0, rear = 0;
```

```
Process Consumer [int i=1 to N] {  
    Data myData;  
    while (true) {  
        //produce data  
        P(full);  
        P(mutexC);  
        myData = buf[front];  
        front = (front+1)%n;  
        V(mutexC);  
        V(empty);  
        //Consume myData;  
    }  
}
```

Com múltiplos consumidores
há disputa para usar e atualizar
a variável *front* (seção crítica)



Leitores e escritores

- Sistema com dados compartilhados onde um conjunto de processos apenas lê os dados (leitores) e outro conjunto modifica esses dados (escritores)
- Condições a serem satisfeitas
 - Os leitores podem ler concorrentemente
 - Os escritores precisam ter acesso exclusivo
 - Não deve ocorrer leitura se uma escrita estiver em curso, e vice-versa
 - Apenas um processo escritor pode alterar os dados em um dado instante de tempo
- Questão adicional: quem espera por quem?
 - Leitor tem uma prioridade não preemptiva sobre um escritor (*readers preferred readers-writes system*)
 - Escritor tem uma prioridade não preemptiva sobre um leitor (*writers preferred readers-writes system*)

Leitores e escritores:

readers preferred readers-writes system

```
Var semaphore mutex = 1;  
    semaphore rw = 1;  
    int readcount = 0;
```

```
Process Writer {  
    while (true) {  
        P(rw) ;  
        // writing is performed  
        V(rw) ;  
    }  
}
```

```
Process Reader {  
    while (true) {  
        P(mutex) ;  
        readcount++;  
        if (readcount == 1)  
            P(rw) ;  
        V(mutex) ;  
        // reading is performed  
        P(mutex) ;  
        readcount--;  
        if (readcount == 0)  
            V(rw) ;  
        V(mutex) ;  
    }  
}
```

Leituras adicionais

- Andrews, G. *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - Capítulo 4 (até seção 4.4.1 inclusive)
- Silberschatz, A. and Galvin, P. *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 6 (seção 6.5 e 6.6)

Exercício

- Barreira é um mecanismo de sincronização onde um grupo de tarefas se sincronizam em um determinado ponto do código. As 1^{as} tarefas a chegarem na barreira esperam pelas outras antes de prosseguirem. Quando a última tarefa do grupo chegar à barreira, todas poderão prosseguir.
 - Implemente em pseudo-código uma barreira para dois processos utilizando semáforos

INF01151 – Sistemas Operacionais II

Aula 06 - Monitores

Prof. Alberto Egon Schaeffer Filho

Introdução

- Exclusão mútua pode ser obtida com variáveis *lock* e semáforos
 - São mecanismos “primitivos” e estão sujeitos a erros no seu uso
 - ***lock/unlock* estão sujeitos a erros de:**
 - » Desatenção na sequência de primitivas *lock/unlock*
 - » Esquecimento de primitiva *lock* ou *unlock*
 - **Semáforos estão sujeitos a erros de:**
 - » Esquecimento de P ou de V
 - » Inicialização
 - » Ordem e quantidade de primitivas
 - Erros levam a *deadlocks* e violação da exclusão mútua

Problemas: utilização incorreta de semáforos

- Erros de programação na utilização de semáforos:

Processo troca a ordem de execução das primitivas:

```
V(s);  
...  
{critical section}  
...  
P(s);
```

Violação da exclusão mútua

Processo substitui chamada de V por P:

```
P(s);  
...  
{critical section}  
...  
P(s);
```

Deadlock

Processo omite chamada a P ou V:

```
P(s);  
...  
{critical section}  
...  
V(s);
```

Violando exclusão mútua ou causando deadlock

Além disso ...

- Variáveis tipo lock
 - Apenas exclusão mútua

- Semáforos

- Exclusão mútua, sincronização condicional, etc
 - Objetivos diferentes, deveriam ser obtidos c/ programação diferente
 - Dificuldade adicional para escrever e compreender um programa

- Solução: construção de linguagem de mais alto nível



Monitores

Módulos de um programa que oferecem mais estrutura que semáforos

De um modo geral:

- É melhor que **exclusão mútua** seja **implícita**, pois automaticamente evita interferência
- É melhor que **sincronização de condição** seja **explícita**, pois diferentes programas precisam diferentes condições

Monitor

- Monitor pode ser visto como um objeto (ou um módulo, ou um abstract data type) que encapsula:
 - **Variáveis permanentes** representando dados para serem compartilhados
 - Dados internos que armazenam o estado do objeto
 - Privados e acessíveis somente dentro do monitor
 - **Procedimentos** para manipular esses dados (única forma de manipulação)
 - Procedimentos são executados de forma mutuamente exclusiva
 - Exclusão mútua é garantida pelo ambiente (**implicitamente!**)
 - Compilador insere todas as chamadas de sistema e código necessários para criar filas de espera por recursos

Sintaxe e semântica de um monitor

```
monitor name {
```

```
    Declaração de variáveis permanentes
```

```
    Inicialização
```

```
    Procedimentos
```

```
}
```

- Compartilhadas por todos os procedimentos
- Mantém os valores enquanto o monitor existir (estado)

- Procedimentos implementam as operações
- Variáveis locais

- Propriedades de um monitor:
 - Apenas os nomes de procedimentos são visíveis do exterior
 - Procedimentos de um monitor não acessam variáveis externas
 - Variáveis permanentes são inicializadas antes de qualquer procedimento
 - Quando o monitor é criado

Semântica do monitor: exclusão mútua

- No máximo uma instância de um procedimento de um monitor pode estar sendo executada (ativa) em um instante de tempo t
 - Responsabilidade do compilador garantir a exclusão mútua
 - Usa lock e/ou semáforos (uso correto → garantido pelo compilador!)
 - Oferecida **implicitamente** pelo monitor
 - Exclusão mútua
 - Entre chamadas para procedimentos diferentes
 - Entre chamadas para um mesmo procedimentos

Semântica do monitor: variáveis de condição

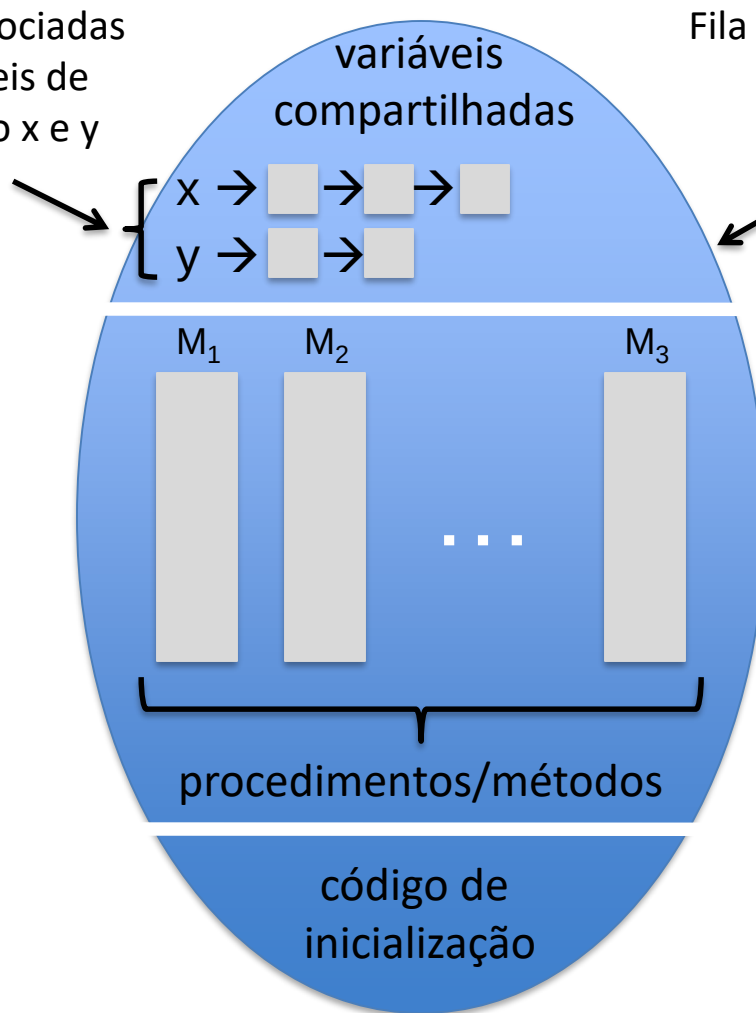
- Variável de condição
 - Sincronização de condição (coordenação de ações)
 - Utilizada para **atrasar um processo** que não pode continuar executando até que o estado do monitor satisfaça alguma condição booleana
 - Também utilizada para **acordar um processo** esperando quando condição se tornar verdadeira
- Variáveis especiais ligadas ao conceito de monitor
 - Declaradas e usadas apenas ***dentro*** do monitor
 - Possuem uma fila associada
 - O “valor” da variável de condição é a fila, mas ela não é vista, nem acessível para o programador
- Só podem ser acessadas via primitivas específicas
 - **wait(cv)** e **signal(cv)**

Primitivas *wait* e *signal*

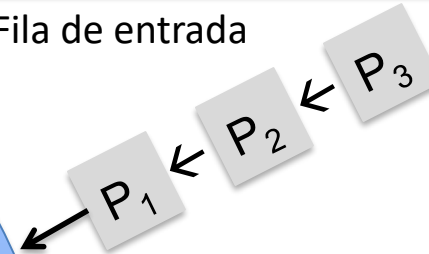
- **`wait (condition_variable)`**
 - Processo que executa **`wait`** bloqueia na variável de condição
 - Retira do monitor, incondicionalmente, o processo que a executa
 - Coloca processo no final da fila associada à variável de condição
- **`signal (condition_variable)`**
 - Acorda um único processo da fila associada à condição sinalizada
 - Se fila estiver vazia no momento do **`signal`**, ele é perdido
 - Não “guarda memória” (diferentemente de semáforos!)
 - As primitivas **`wait`** e **`signal`** implementam uma política FIFO
 - **`wait`** insere o processo no final da fila e **`signal`** acorda o processo da frente da fila
→ comportamento *default*
 - Possível haver outras políticas (e.g., prioridade)

Representação de um monitor

Filas associadas
a variáveis de
condição x e y



Fila de entrada



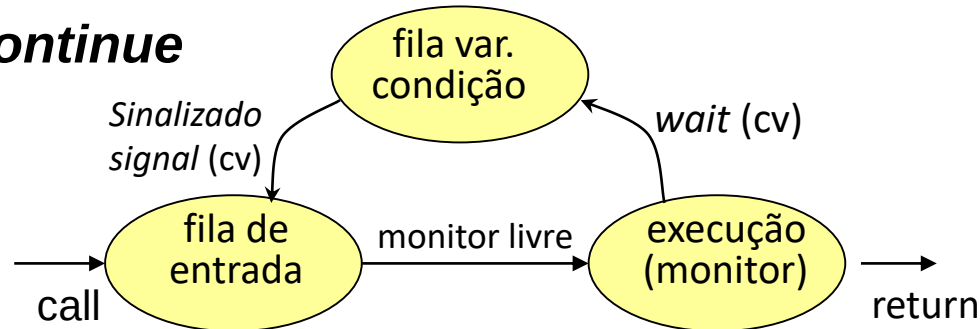
- Uma única fila de espera para entrar no monitor
- Uma fila para cada variável de condição
- Questão é o que fazer com os processos sinalizadores?
 - Semânticas para o **signal**

Estratégias de sinalização

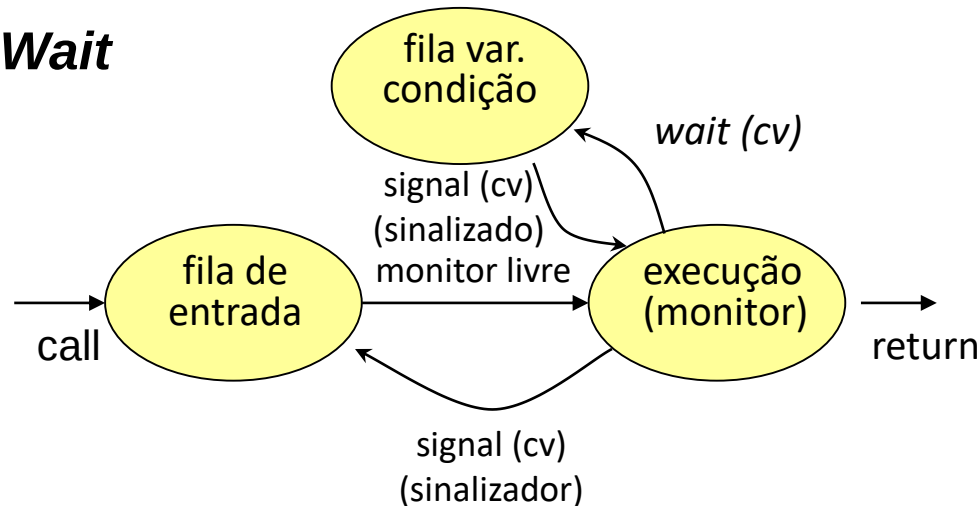
- Quando processo executa **signal**, ele está dentro do monitor e implicitamente tem controle do lock daquele monitor
 - Dilema: se o **signal** irá acordar outro processo, existem agora dois processos que poderiam executar
 - Processo que fez o **signal**? Processo que foi acordado?
- Estratégias de sinalização
 - **Signal and Continue (SC)**: não-preemptiva → Java / ambientes Unix / Pthreads
 - O processo sinalizador continua sua execução e o sinalizado passa para a fila de entrada do monitor
 - **Signal and Wait (SW)**: preemptiva
 - O processo sinalizador vai para a fila de entrada do monitor e o processo sinalizado inicia sua execução
 - **Signal and Urgent Wait (SU)**: preemptiva
 - O processo sinalizador vai para a frente da fila de entrada do monitor e o processo sinalizado inicia sua execução

Diagrama de estados

Signal and Continue



Signal and Wait



Primitivas adicionais para *condition variables* (cv)

- Operações adicionais na fila de variável de condição
 - **wait** e **signal** determinam que processos são acordados na ordem em que foram colocados em espera (FIFO)
 - Como exercer mais controle?
 - **wait(cv, rank)** : processo é posto na fila da cv, mas ordenados de acordo com o valor de *rank* (crescente)
 - **minrank(cv)** : determina rank do processo na frente da fila de cv
 - **empty(cv)** : Retorna *true* se a fila da cv está vazia, senão *false*
 - **signal_all(cv)** : usado se mais de um processo bloqueado puder proceder (ou se o sinalizador não sabe quem deve proceder)
 - Sinaliza todos os processo que estão na fila da cv.
 - Comportamento bem definido para a estratégia SC
 - O que ocorre se a estratégia SW?

Resumo de operações em condition variables

Operação	Descrição
wait(cv)	Espera no final da fila
wait(cv,rank)	Espera em ordem crescente de rank
signal(cv)	Acorda processo da frente da fila e continua
signal_all(cv)	Acorda todos os processos da fila e continua
empty(cv)	True se fila estiver vazia, false caso contrário
minrank(cv)	Valor do rank do processo na frente da fila

Produtor/consumidor com buffer limitado (outra vez...)

- Relação produtor-consumidor
 - Produtor insere mensagens em um buffer compartilhado
 - Consumidor retira mensagens
 - Buffer (*array buf*) de tamanho n contém mensagens que foram inseridas, mas ainda não foram lidas

Produtor:

```
buf[rear] = data;  
rear = (rear+1) % n;
```

Consumidor:

```
result = buf[front];  
front=(front+1) % n;
```

- Com monitor:
 - Exclusão mútua é garantida por definição:
 - Não há necessidade de usar mecanismos para exclusão mútua
 - Dois tipos de sincronização de condição:
 - Buffer não está cheio → possível produzir até completá-lo
 - Buffer não está vazio → possível consumir até esvaziá-lo

Buffer limitado: Monitor

```
monitor Bounded_Buffer {  
    typeT buf[n];  
    int    front = 0, rear = 0, count = 0;  
    cond not_full,  
        not_empty;  
  
    procedure deposit(typeT data) {  
        while(count == n) wait(not_full);  
        buf[rear] = data; rear = (rear+1) % n; count++;  
        signal(not_empty);  
    }  
    procedure fetch(typeT &result) {  
        while(count == 0) wait(not_empty);  
        result = buf[front]; front = (front+1) % n; count--;  
        signal(not_full);  
    }  
}
```

Leitores e escritores

(agora com monitores!!)

- Dois tipos de processos, leitores e escritores, e uma base de dados
 - Leitores consultam registros e escritores modificam registros
 - Processos escritores devem ter acesso exclusivo a base de dados
 - Processos leitores podem acessar concorrentemente a base de dados
 - Leitores disputam c/ escritores, escritores disputam entre si e c/ leitores
- Condições:
 - Leitores devem esperar até não haver escritores
 - Escritores devem esperar até não haver leitores ou outros escritores
- Questão: posso modelar a base de dados como um monitor?
 - **Não!** Leitores não poderiam acessar monitor concorrentemente
 - Ao invés disso, utiliza monitor apenas para arbitrar acesso à base
 - Processos informam quando querem acessar e quando terminam

Leitores e escritores: Monitor

```
monitor RW_Controller {
    int  nr = 0, nw = 0;
    cond oktoread,          #signaled when nw == 0
        oktowrite;         #signaled when nr == 0 e nw == 0
    procedure request_read( ) {
        while (nw > 0) wait(oktoread);
        nr = nr + 1;
    }
    procedure release_read( ) {
        nr = nr - 1;
        if (nr == 0) signal(oktowrite);
    }
    procedure request_write( ) {
        while (nr > 0 || nw > 0) wait(oktowrite);
        nw = nw + 1;
    }
    procedure release_write( ) {
        nw = nw - 1;
        signal(oktowrite);
        signal_all(oktoread);
    }
}
```

Aninhamento de monitores

- Procedimento de um monitor chama procedimento de outro monitor
 - Lembre-se: processos executam com exclusão mútua dentro do monitor
- Duas estratégias:
 - **closed call**: a exclusão mútua no primeiro monitor é mantida
 - **open call**: o primeiro monitor é liberado e no retorno a ele o processo deve entrar na fila de espera de entrada
- Como de costume, relação custo x benefício

Aninhamento de monitores

- ***closed call***
 - Por definição não há conflitos com variáveis permanentes
 - Serialização de acessos aos monitores reduz concorrência
 - Aumento a possibilidade de dealock (*nested monitor lockout*)
 - Utilizado na implementação de **monitores em Java**
- ***open call***
 - Necessário manter maior controle das variáveis do monitor
 - Maior complexidade

Leituras adicionais

- Andrews, G. *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - Capítulo 5
- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 6 (seção 6.7)

Exercícios

1. Escreva um monitor para simular o comportamento de uma *barreira* de valor N:
 - Monitor deve possuir um único procedimento denominado *gather*, sem argumentos
 - Valor de N é definido por uma variável permanente no próprio monitor

```
monitor Barreira {  
  
    ...  
  
    procedure gather() {  
  
        ...  
  
    }  
  
}
```

Exercícios

2. Escreva um monitor para controlar o acesso a um recurso compartilhado por até N processos concorrentemente:
- Monitor deve ter dois procedimentos *aloca* e *desaloca*, sem argumentos
 - Valor de N é definido por uma variável permanente no próprio monitor

```
monitor Alocador {  
  
    ...  
  
    procedure aloca() {  
        ...  
    }  
    procedure desaloca() {  
        ...  
    }  
}
```

INF01151 – Sistemas Operacionais II

Aula 07 - Da teoria à prática... Parte I
(Processos, threads, mutex, semáforos,
variáveis de condição e monitores)

Prof. Alberto Egon Schaeffer Filho

Introdução

- Fluxos de controle para programação concorrente
 - Processos ou threads
- Processos/threads interativos
 - São concorrentes
 - Compartilham dados (sincronização de acesso – exclusão mútua) ou coordenam suas atividades entre si (sincronização de condição)
- Questões em aberto:
 - Como criar, na prática, fluxos de controle?
 - Como sincronizar, na prática, as atividades?

Processos no mundo UNIX

- *fork()* cria um processo filho cópia do processo pai
 - Após o *fork()* tem-se dois processos:
 - Processo pai: *fork* retorna o pid do filho
 - Processo filho: o *fork* retorna zero
 - Combinado com a chamada de sistema *exec()* e similares
 - Substitui o espaço de endereçamento do processo filho
- *waitpid (pid_t pid)* faz o processo que a executa esperar pelo término do processo *pid*
- *exit(int status)* termina o processo

/* Orion Lawlor's Short UNIX Examples, olawlor@acm.org 2004/9/5

Shows how to use fork() in a UNIX program.

*/

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h> /* for pid_t */

#include <sys/wait.h> /* for wait */

void doWork(char *arg) {

// snipped code

}

int main()

{

/*Spawn a new process to run alongside us.*/

pid_t pid = fork();

if (pid == 0) { /* child process */

doWork("child");

exit(0);

}

else { /* pid!=0; parent process */

doWork("parent");

waitpid(pid,0,0); /* wait for child to exit */

}

return 0;

}

APIs para *threads*

- Principais APIs
 - **Microsoft Windows**
 - Microsoft Win32 e Microsoft Foundation Class (MFC)
 - Microsoft .Net e Common Language Runtime (CLR)
 - **Java Threads**
 - Suporte a *threads* “embutido” na própria linguagem Java e suportado pela Java Virtual Machine
 - **POSIX Threads**
 - Padrão para plataformas UNIX
 - Possui uma versão open source para windows (*pthread-win32*)
- Apresentam funcionalidades similares de formas diferentes
 - Criação, sincronização, término

Java threads

- Suporte a *threads* é integrado na linguagem
 - Permite criação e gerenciamento de threads de forma independente do sistema operacional ou de bibliotecas
 - Cada thread de execução tem sua própria *call stack*
- Todo programa java tem ao menos uma *thread*
 - *Main* thread, que contém método **main**
- Criação de threads
 - 1º método: Estendendo classe **Thread**
 - 2º método: Implementando interface **Runnable**
- Pacote **java.lang**

Criação de threads: 1º método

- Estendendo a classe **Thread**
 - Necessário sobrescrever o método **run** (the job to do)
 - Método **start** é usado para criar thread, e implicitamente chama **run**

```
class Simple extends Thread {  
    public void run() {  
        System.out.println("I'm a  
thread");  
    }  
}
```

```
public class First {  
    public static void main(String args[]) {  
        Simple s = new Simple();  
  
        s.start();  
        System.out.println("I'm the main thread");  
    }  
}
```

Criação de threads: 2º método

- Implementando interface **Runnable**

- Necessário implementar o método **run** (the job to do)
- Método **start** é usado para criar thread, e implicitamente chama **run**
- Objeto *Runnable* passado como o *target* da thread

```
class Simple implements Runnable {  
    public void run() {  
        System.out.println("I'm a  
thread");  
    }  
}
```

```
public class Second {  
    public static void main(String args[]) {  
        Runnable s = new Simple( );  
        Thread thr = new Thread(s);  
  
        thr.start();  
        System.out.println("I'm the main thread");  
    }  
}
```

Considerações gerais: Java threads

- Etapas para uso de threads em Java

① Definir uma nova classe que:

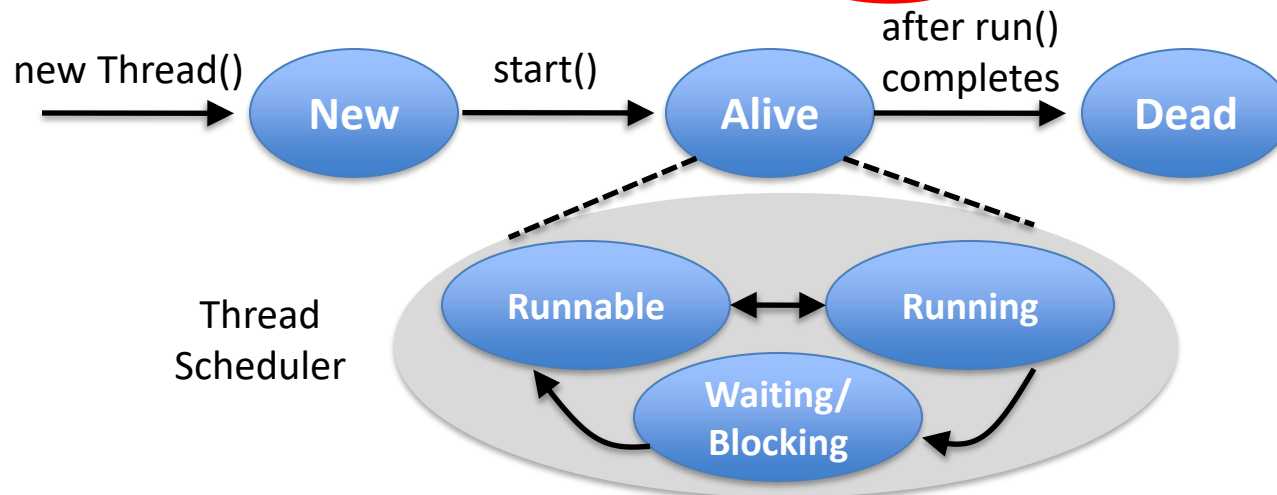
- Estenda a classe **Thread**; ou que
- Implemente a interface **Runnable**

```
Thread t = new Thread(runObj);  
t.run(); // Permitido, mas não  
        // inicia nova thread!
```

② Definir um novo método **run** (código da thread)

③ Criar uma nova instância através de **new**

④ Disparar a thread usando o método **start**



Considerações gerais: Java threads

- Métodos permitem “influenciar” o escalonamento de threads
 - `public static void sleep(long millis) throws InterruptedException`
 - `public static void yield()`
 - `public final void join() throws InterruptedException`
 - `public final void setPriority(int newPriority) // entre 1 e 10`
- Uma thread não pode dizer para outra bloquear:
 - Métodos `sleep()`, `yield()` e `join()` afetam a thread corrente que está executando!
 - Antigos `stop()`, `suspend()` e `resume()` são **deprecated**
- Comportamento do escalonador de threads
 - Depende da implementação da JVM, mas em geral...
 - Prioridade das threads em `Runnable` é menor ou igual à prioridade da thread em `Running`
 - Se prioridades forem as mesmas, qualquer coisa pode acontecer

POSIX Threads (*Pthreads*)

- Biblioteca de *threads* portátil para várias plataformas
 - Conjunto de rotinas C para programação multithread
 - Padrão usado no Linux e em sistemas UNIX em geral
 - IEEE POSIX 1003.1c standard (1995)
 - Especificação, não implementação
- Possui um grande número de funções
 - Pthreads API contém em torno de 100 subrotinas
 - Foco principal é a criação, destruição e sincronização
 - Possui funções que permitem explorar características próprias de um sistema (e.g. prioridades)
 - Atenção: comportamento varia de plataforma para plataforma

Pthreads basics

- Incluir header para biblioteca de *Pthreads*

```
#include <pthread.h>
```

- Variáveis de atributos e descritores: inicializados antes de criar a thread, mas muitos podem ser alterados mais tarde

```
pthread_attr_t attr; /* thread attributes */  
pthread_t thr;      /* thread descriptor */
```

- Atributos:
 - detached state (joinable? Default: PTHREAD_CREATE_JOINABLE. Other option: PTHREAD_CREATE_DETACHED)
 - scheduling policy (real-time? PTHREAD_INHERIT_SCHED, PTHREAD_EXPLICIT_SCHED, SCHED_OTHER)
 - scheduling parameter
 - inheritsched attribute (Default: PTHREAD_EXPLICIT_SCHED Inherit from parent: PTHREAD_INHERIT_SCHED)
 - scope (Kernel threads: PTHREAD_SCOPE_SYSTEM User threads: PTHREAD_SCOPE_PROCESS)
 - guard size
 - stack address (See unistd.h and bits/posix_opt.h _POSIX_THREAD_ATTR_STACKADDR)
 - stack size (default minimum PTHREAD_STACK_SIZE set in pthread.h)
- Inicialização dos atributos

```
pthread_attr_init(&attr);
```

- Geralmente os valores default são suficientes

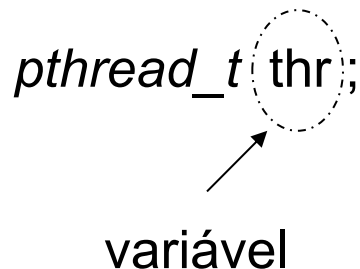
Primitiva de criação

```
int pthread_create (  
    pthread_t *thr,                //descriptor retornado  
    pthread_attr_t *attr;          //atributos (NULL = none)  
    void *(*start_routine) (void *), //ponteiro p/ função a ser executada  
    void *arg);                    //ponteiro p/ parâmetro (único)
```



Retorna 0, se criou com sucesso;
diferente de 0, caso contrário.

pthread_t thr;
variável



- Identificador
- Atributos
- Função
- Ponteiro p/ parâmetros

- Uma *thread* é uma estrutura de dados → tipo *pthread_t*

Exemplo de criação de *Pthreads*

```
#include <pthread.h>
int g;

void *do_it_1(void *arg) {
    int i, n = *(int *) arg;
    for(i = 0; i < n; i++)
        g = i;
}

void *do_it_2(void *arg) {
    int i, n = *(int *) arg;
    for(i = 0; i < n; i++)
        printf("%d\n", g);
}

int main( int argc, char *argv) {
    pthread_t th1, th2;
    int n = 10;
    pthread_create(&th1, NULL, do_it_1, &n);
    pthread_create(&th2, NULL, do_it_2, &n);
    ....
}
```

Primitivas relacionadas ao término

- Thread termina sua própria execução chamando:

```
pthread_exit(void *status);
```

- `status` é um valor de retorno (ou NULL)
- Chamada implicitamente se thread retorna da função que havia iniciado

- Thread pode ser cancelada por outra thread

```
pthread_cancel(pthread_t thr);
```

- `thr` é a thread a ser cancelada

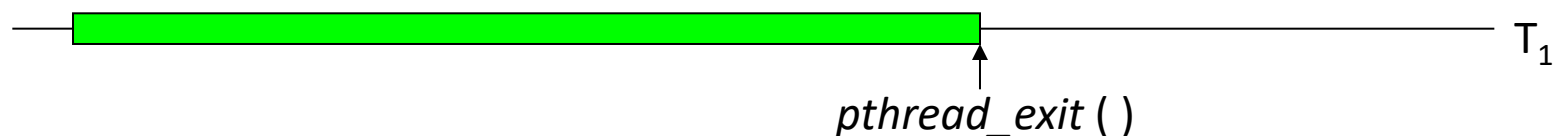
- Thread pai pode esperar por término da thread filha chamando:

```
pthread_join(pthread_t thr, void **res);
```

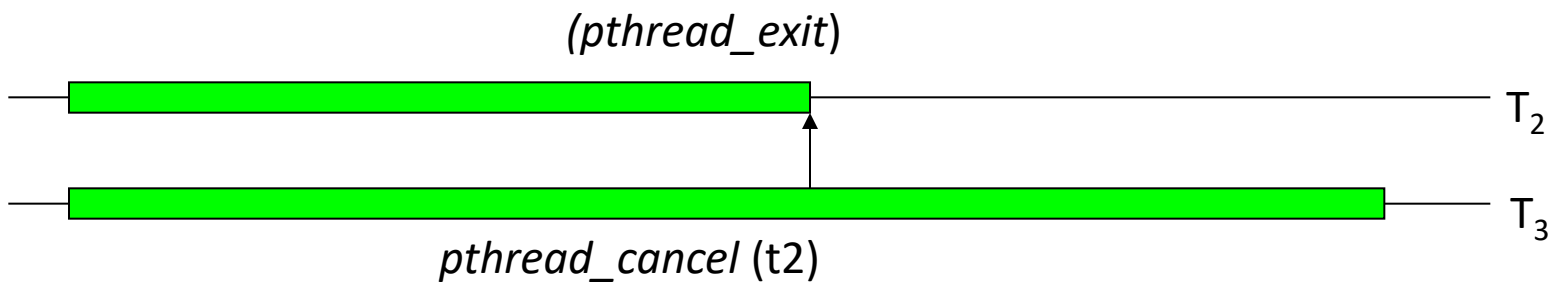
- `thr` é o decritor da thread filha
- `res` é endereço da localização do valor de retorno
 - Preenchido quando filha executa `pthread_exit`

Primitivas relacionadas ao término (destruição)

```
void pthread_exit (void *status);
```



```
int pthread_cancel (pthread_t thread);
```

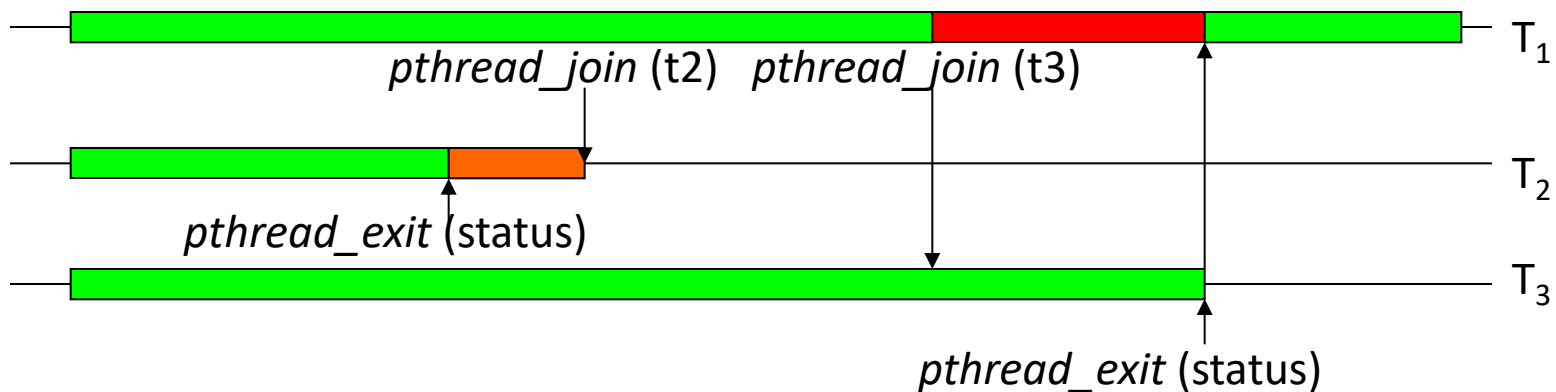


- Terminar = liberar a memória ocupada pela estrutura de dados `pthread_t`
- `exit( )` versus `pthread_exit( )`
 - Terminar um processo (`exit`) é terminar todas as threads (elas vivem “dentro”)

Primitivas relacionadas ao término (sincronização)

- Possibilidade de uma *thread* esperar pelo término de outra
 - Identificador da thread pela qual se espera
 - Possibilidade de receber um valor de retorno

```
void pthread_join (pthread_t thread, void **res);
```



- Válido apenas para *threads* com atributo `joinable` (ao invés de `detached`):
- É o **default**!

Exemplo de espera por *threads*

```
#include <pthread.h>

int g;

void *do_it_1(void *arg) {
    int i, n = *(int *) arg;
    for(i = 0; i < n; i++)
        g = i;
}

void *do_it_2(void *arg) {
    int i, n = *(int *) arg;
    for(i = 0; i < n; i++)
        printf("%d\n", g);
}
```

```
int main( int argc, char *argv) {

    pthread_t th1, th2;
    int n = 10;
    pthread_create(&th1, NULL, do_it_1, &n);
    pthread_create(&th2, NULL, do_it_2, &n);

    pthread_join(th1, NULL);
    pthread_join(th2, NULL);

}
```

Possível usar **pthread_join** para recuperar valores da thread que se espera, mas programador deve ser bom em ponteiros em C.



Outro exemplo de espera por *threads*

```
#include <pthread.h>

void *test_routine (void *arg) {
    double *value;
    value = (double*) malloc(sizeof(double));
    *value = 10.0;

    pthread_exit(value);
}

int main( int argc, char **argv) {
    pthread_t thr;
    double *res;
    int status;
    status = pthread_create(&thr, NULL, test_routine, NULL );
    if (status != 0)
        exit(1);
    status = pthread_join(thr, (void **) &res );
    if (status != 0)
        exit(1);
    printf("resultado retornado: %.2f\n", *res);
    free(res);
}
```

Leituras adicionais

- Andrews, G. *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - Capítulo 4 (seção 4.6) e capítulo 5 (seção 5.4 e 5.5)
- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 4 (seção 4.3)
- Butenhof, D. – *“Programming with POSIX Threads”*. Addison-Wesley. 1997.

INF01151 – Sistemas Operacionais II

Aula 08 - Da teoria à prática... Parte II
(Processos, threads, mutex, semáforos,
variáveis de condição e monitores)

Prof. Alberto Egon Schaeffer Filho

Mecanismos de sincronização

	UNIX	POSIX Threads	Java
Semáforos	Sim	Não ¹	Sim ²
Mutex (lock/unlock)	Não	Sim	Sim ²
Variáveis de condição	Não	Sim	Sim ^{2,3}
Monitores	Não	Não	Sim

¹ Não fazem parte da especificação *POSIX Threads* (norma POSIX 1003.1c), mas é possível empregar os semáforos Unix (norma POSIX 1003.1b)

² API específica disponível desde a versão 1.5 através do pacote *java.util.concurrent*

³ Possui uma variável de condição implícita por objeto sincronizado

Semáforos Unix

Existem dois tipos: *unnamed* e *named*

- **Unnamed**: existe no espaço de endereçamento de processos ou threads (*process-wide*)
 - Usados para sincronizar threads e processos relacionados
 - Declarado como variável global: *pshared* = 0 para threads, *pshared* ≠ 0 para processos
 - Não persistentes, existem apenas enquanto os processos executam
 - Se programador esquecer de fechar semáforo, ele é zerado quando processo termina
- **Named**: existe no sistema de arquivos, i.e., é um descritor de arquivo (*system-wide*)
 - Uso similar ao de um arquivo
 - Permite sincronização entre processos, relacionados ou não, e threads
 - Threads e processos usam da mesma forma
 - Persistentes (existem até o *shutdown* do sistema)
 - Limitado em número pelo SO

API para semáforos *named*

(sistema de arquivos)

```
#include <sys/semaphore.h>
```

```
sem_t *sem_open (const char *name, int oflag, mode_t mode, int value);
```

- *name*: caminho de um arquivo (semáforo)
 - *oflag*: modo de abertura do semáforo
 - O_CREAT para criar um semáforo se ele não existe, O_EXCL se a chamada deve falhar se já existente
 - *mode*: permissões do arquivo semáforo
 - *value*: valor inicial do semáforo
- Chamada que cria o semáforo, inicializa e configura as permissões

```
int sem_wait (sem_t *sem);          /* equivalente a primitiva P */  
int sem_trywait (sem_t *sem);       /* adquire semáforo se livre, caso  
                                     contrário retorna sem adquirir */  
int sem_post(sem_t *sem);           /* equivalente a primitiva V */  
int sem_close(sem_t *sem);          /* encerra o uso do semáforo */  
int sem_unlink(const char *name);   /* remove do sistema */
```

API para semáforos *unnamed*

```
#include <semaphore.h>
```

```
int sem_init (sem_t * sem, int pshared, unsigned int value);
```

- *sem*: variável do tipo semáforo
- *pshared*: visibilidade intra processo (= 0) ou inter processos (≠ 0)
- *value*: valor inicial do semáforo

```
int sem_wait (sem_t *sem);          /* equivalente a primitiva P */
```

```
int sem_trywait (sem_t *sem);       /* adquire semáforo se livre, caso  
                                     contrário retorna sem adquirir */
```

```
int sem_post(sem_t *sem);           /* equivalente a primitiva V */
```

```
int sem_destroy(sem_t *sem);        /* destruição do semáforo */
```

Exemplo de uso semáforos *unnamed*

```
#include <pthread.h>
#include <semaphore.h>
#include <stdlib.h>
#include <stdio.h>

#define N      2
#define TRUE  1

int      buffer[N], in=0, out=0;
sem_t    vazio, cheio, mutexP, mutexC;

void *produtor(void *arg) {

    while(TRUE) {
        sem_wait(&vazio);
        sem_wait(&mutexP);
        buffer[in] = rand() % 100;
        printf("P %d\n", buffer[in]);
        in= (in+1) % N;
        sem_post(&mutexP);
        sem_post(&cheio);
    }
}
```

```
void *consumidor(void *arg) {
    int item;
    while(TRUE) {
        sem_wait(&cheio);
        sem_wait(&mutexC);
        item = buffer[out];
        printf("Consumindo %d\n", item);
        out = (out+1) % N;
        sem_post(&mutexC);
        sem_post(&vazio);
    }
}

int main(int argc, char *argv[ ]) {
    pthread_t cons, prod;
    sem_init(&mutexP, 0, 1);
    sem_init(&mutexC, 0, 1);
    sem_init(&vazio, 0, N);
    sem_init(&cheio, 0, 0);
    pthread_create(&prod, NULL, produtor, NULL);
    pthread_create(&cons, NULL, consumidor, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
}
```


Sincronização com POSIX threads

- Podem usar semáforos Unix
- Variáveis do tipo mutex (sincronização de acesso)
- Variáveis de condição (sincronização de controle)

POSIX threads: *mutex locks*

- É um tipo de dados abstrato (**pthread_mutex_t**)
 - Necessita ser declarado e inicializado
- Duas formas de inicializar
 - Na declaração: **pthread_mutex_t mymutex = PTHREAD_MUTEX_INITIALIZER;**
 - Dinamicamente, com **pthread_mutex_init(&mutex, &attr);**
- Atributos do tipo **pthread_mutexattr_t**, ou **NULL** para *defaults*:
 - **Protocol**: especifica protocolo usado para evitar inversão de prioridades
 - **Prioceiling**: especifica o teto de prioridade do mutex
 - **Process-shared**: especifica se o mutex é compartilhado em processos

- Semântica mais simples que semáforos
- Somente a thread que fez **lock** pode fazer **unlock**

```
pthread_mutex_t mutex;  
pthread_mutex_init(&mutex, NULL);  
...  
pthread_mutex_lock(&mutex);  
{seção crítica}  
pthread_mutex_unlock(&mutex);
```

Inicialização simplificada, utilizando mutex com os atributos default

API para mutex locks (POSIX)

```
#include <pthread.h>
```

```
int pthread_mutex_init (pthread_mutex_t *mutex,  
                        const pthread_mutexattr_t* attr );
```

```
int pthread_mutex_lock ( pthread_mutex_t *mutex );  
int pthread_mutex_trylock ( pthread_mutex_t *mutex );  
int pthread_mutex_unlock ( pthread_mutex_t *mutex );
```

```
int pthread_mutex_destroy (pthread_mutexattr_t *mutex );
```

Obtém o lock se estiver livre, senão bloqueia

Tenta obter o lock. Se não conseguir, retorna com código de erro “busy”

Libera o lock. Gera código de erro se lock já estava livre ou se lock pertence a outra thread

Exemplo de uso mutex locks (POSIX)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define TRUE 1

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void *minha_funcao (void *arg) {
    int n = *(int *) arg;
    while(TRUE) {
        pthread_mutex_lock(&m);
        printf("Thread %d iniciando a SC\n ", n);
        printf("Thread %d ainda na SC\n ", n);
        printf("Thread %d saindo da SC\n ", n);
        pthread_mutex_unlock(&m);
    }
}

int main (int argc, char *argv[]) {
    pthread_t  thr1, thr2;
    int n1 = 1; int n2 = 2;

    pthread_create(&thr1, NULL, minha_funcao, (void *) &n1);
    pthread_create(&thr2, NULL, minha_funcao, (void *) &n2);

    pthread_exit(0);
}
```

Variáveis de condição: POSIX threads

- Poderoso mecanismo de sincronização condicional
 - *Mutex*/semáforo binário apenas permitem que *threads* ou processos serializem seus acessos a uma seção crítica
 - Permite que thread bloqueie se condição não for verdadeira
 - E que outra thread sinalize quando condição ocorrer
 - Uso típico é informar o estado de dados compartilhados
 - ex.: buffer está vazio, buffer está cheio, que evento desejado ocorreu etc.
 - Deve ser sempre **usada com um *mutex*!**
 - Necessário garantir acesso exclusivo a dado p/ consultar seu valor
 - Thread que deseja fazer wait deve primeiro obter o lock
- ```
pthread_mutex_lock(&mutex);
```
- E depois executar
- ```
pthread_cond_wait(&cond, &mutex);
```
- Faz com que a thread libere o lock e espere na variável de condição

Variáveis de condição: POSIX threads

- Tipo de dados abstrato (**pthread_cond_t**)
 - Necessita ser declarado e inicializado
- Duas formas de inicializar
 - Na declaração: **pthread_cond_t mycond = PTHREAD_COND_INITIALIZER;**
 - Dinamicamente: **pthread_cond_init (&mycond, &attr);**
- Mais fácil compreensão e uso que semáforos
 - Espera por um evento: **cond_wait**
 - Sinalizando a ocorrência de um evento: **cond_signal** e **cond_broadcast**
 - “Acorda” UMA (*cond_signal*) ou TODAS (*cond_broadcast*) *threads*
 - Importante: as *threads* não memorizam *signal* (diferente de semáforos)
- Sempre devem ser usadas em conjunto com um **mutex**!
 - Liberado automaticamente e atomicamente em **cond_wait**
 - Obtido automaticamente e atomicamente em **cond_signal**

Um único atributo:
process-shared

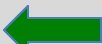
Variáveis de condição: POSIX threads

- Sequência típica (receita) de uso:

Thread Main

Declarar e inicializar **variáveis globais** que necessitam sincronização (p. ex. count, buffer, etc)
Declarar e inicializar **váriavel de condição**
Declarar e inicializar **mutex lock**
Criar threads **A** e **B**

Thread A

Executar até ponto em que condição deve ocorrer
Obter o **mutex**
Verificar valor da variável global (p.ex. count == n)
Se necessário, **bloquear** pthread_cond_wait() 
→ mutex automaticamente liberado
Acordar quando for sinalizado 
→ mutex automaticamente obtido
Liberar o **mutex**

Thread B

Executar
Obter o **mutex**
Modificar valor da variável global que Thread A
está esperando
Se valor satisfazer condição da Thread A, **sinalizar**
→ semântica *signal and continue*
Liberar o **mutex** 

Thread Main

Esperar até que threads **A** e **B** terminem (*join*)

API POSIX para variáveis de condição

```
#include <pthread.h>
```

```
int pthread_cond_init (pthread_cond_t *cond, const pthread_condattr_t *attr);  
int pthread_condattr_init(pthread_condattr_t *cond);
```

```
int pthread_cond_wait (pthread_cond_t *cond, pthread_mutex_t *mutex);  
int pthread_cond_signal (pthread_cond_t *cond);  
int pthread_cond_broadcast (pthread_cond_t *cond);
```

```
int pthread_cond_destroy (pthread_cond_t *cond);
```


Exemplo de uso variáveis de condição

```
#include <pthread.h>
#include <stdio.h>
#define TRUE 1
#define MAX_ITEMS 10

pthread_cond_t not_full, not_empty;
pthread_mutex_t mutex;

int count = 0, in = 0, out = 0;
int buffer[MAX_ITEMS];

void *produtor(void *arg) {

    while (TRUE) {
        pthread_mutex_lock(&mutex);
        while (count == MAX_ITEMS)
            pthread_cond_wait(&not_full, &mutex);
        buffer[in] = rand() % 5;
        count++;
        printf("P buffer[%d]=%d\n", in,
            buffer[in]);
        in = (in + 1) % MAX_ITEMS;
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);
    }
}
```

```
void *consumidor(void *arg) {
    int task;
    while (TRUE) {
        pthread_mutex_lock(&mutex);
        while (count == 0)
            pthread_cond_wait(&not_empty, &mutex);
        task = buffer[out];
        count--;
        printf("C buffer[%d] = %d\n", out, task);
        out = (out + 1) % MAX_ITEMS;
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);
    }
}

int main(int argc, char *argv[]) {
    pthread_t prod, cons;
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&prod, NULL, produtor, NULL);
    pthread_create(&cons, NULL, consumidor, NULL);

    pthread_exit(0);
}
```

Monitores em Java

- Fornece exclusão mútua em um objeto
 - Emprego da palavra-chave *synchronized*
 - Utilizada com um método inteiro
 - Utilizada com uma sequência de comandos

```
class Interfere {  
    private int data = 0;  
    public void update( ) {  
        data++;  
    }  
}
```



Se método **update** for chamado simultaneamente por duas threads, elas vão causar interferência nas suas execuções (seção crítica)

```
class Interfere {  
    private int data = 0;  
    public synchronized void update() {  
        data++;  
    }  
} // método sincronizado
```

```
class Interfere {  
    private int data = 0;  
    public void update() {  
        synchronized(this) {  
            data++;  
        }  
    }  
} // bloco sincronizado
```

Monitores em Java

- Como monitores são programados em Java
 - Variáveis permanentes do monitor são os dados privados da classe
 - Procedimentos são implementados como métodos **synchronized**
 - Exclusão mútua “implícita”, um lock por objeto em Java
 - Quando método sincronizado é invocado, ele espera pelo lock, executa, e libera o lock
- Primitivas para sincronização de condição
 - Precisam ser executados em porções de código **synchronized**
 - **wait**: thread que executa fica bloqueada esperando por um notify (signal)
 - Libera o monitor
 - **notify**: acorda uma única thread que está bloqueado na fila de espera
 - **notifyAll**: acorda todas as threads da fila de espera
 - Não suporta diretamente variáveis de condição, mas pode-se pensar que existe uma variável de condição (implícita) por objeto sincronizado

Monitores em Java: comentários

- Existe uma única fila por objeto sincronizado
 - A política da fila não é padronizada, mas geralmente é FIFO
- Java implementa a semântica ***signal-and-continue (SC)***
 - Como a *thread* que sinaliza continua a executar, pode acontecer que a condição sinalizada não seja mais verdadeira
 - *Thread* que foi “acordada” para entrar no monitor pode ter que voltar a esperar a condição
- Permite chamadas aninhadas, na forma ***closed call***
 - Necessário cuidar com deadlocks
- Códigos-exemplos:
 - http://wps.prenhall.com/br_deitel_sistemas_3

Leituras adicionais

- Andrews, G. *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - Capítulo 4 (seção 4.6) e capítulo 5 (seção 5.4 e 5.5)
- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 4 (seção 4.3)
- Butenhof, D. – *“Programming with POSIX Threads”*. Addison-Wesley. 1997.

Exercício para aquecimento

1. Quantos processos são criados quando o programa abaixo é executado?

```
#include <stdio.h>
#include <unistd.h>

int main() {
    /* fork a child process */
    fork();
    /* fork another child process */
    fork();
    /* and fork another */
    fork();
    return 0;
}
```

Exercício para aquecimento

1. Quantos processos são criados quando o programa abaixo é executado?

```
#include <stdio.h>
#include <unistd.h>

int main() {
    /* fork a child process */
    fork();
    /* fork another child process */
    fork();
    /* and fork another */
    fork();
    return 0;
}
```

```
1 (processo inicial)
├─ 2 [fork#1, filho de 1]
│   └─ 3 [fork#2, filho de 2]
│       └─ 4 [fork#3, filho de 3]
│           └─ 5 [fork#3, filho de 2]
├─ 6 [fork#2, filho de 1]
│   └─ 7 [fork#3, filho de 6]
└─ 8 [fork#3, filho de 1]
```

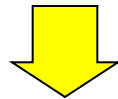
INF01151 – Sistemas Operacionais II

Aula 09 - Comunicação inter-processos

Prof. Alberto Egon Schaeffer Filho

Introdução

- Programação concorrente surgiu com a multiprogramação
 - Mecanismos básicos supõem memória compartilhada, mas isso nem sempre é possível (ou desejável)
 - Questões de segurança/privacidade
 - Máquinas com memória distribuída (e.g. redes de computadores)
- Processos UNIX têm espaço de endereçamento próprio e privativo
 - Não há compartilhamento de memória, por exemplo, `fork()` cria processo filho com espaços de endereçamento distinto (variáveis são copiadas)



Mecanismos de IPC (*Inter-Process Communication*)

Nota: as threads criadas por um mesmo processo compartilham o espaço de endereçamento daquele processo

Processos interativos

- **Tipos de processos**

- **Processos independentes**: não podem afetar ou serem afetados pela execução de outro processo (não compartilham dados)
- **Processos interativos**: podem afetar ou serem afetados pela execução de outro processos (compartilham dados)

Benefícios da interação entre processos

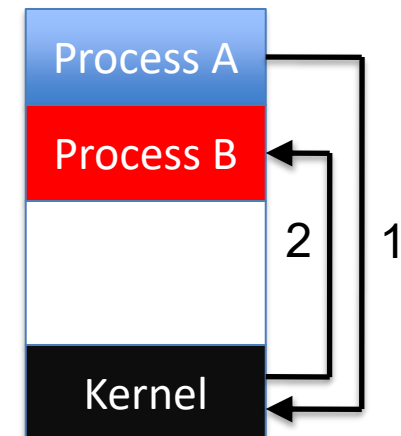
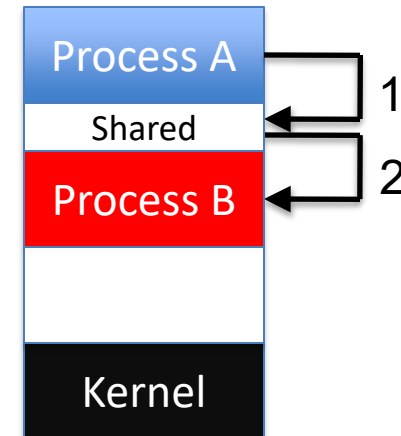
- Compartilhamento de informações
- Speed-up da computação
- Modularidade
- Conveniência

Perigos da interação entre processos

- Dados corrompidos, deadlocks, aumento da complexidade
- Necessidade que processos sincronizem seu processamento

Comunicação inter-processos

- Processos competitivos/colaborativos precisam de mecanismos IPC
 - Troca de dados e compartilhamento de informações
- Dois modelos fundamentais para IPC
 - **Memória compartilhada** (shared-memory systems)
 - Região de memória compartilhada entre processos é estabelecida
 - Processos compartilham informações através da leitura e escrita de dados na região
 - Read/Write
 - **Troca de mensagens** (message-passing systems)
 - Comunicação ocorre através de mensagens enviadas de um processo para outro
 - Send/Receive



Comunicação inter-processos

- Troca de mensagens vs. memória compartilhada
- Ambos são bastante comuns em SOs atuais, em geral...
 - Troca de mensagens
 - Requer chamadas de sistema, **intervenção do kernel**
 - Melhor para compartilhamento de **pequenas quantidades** de dados
 - Solução utilizada em **sistemas distribuídos**
 - Memória compartilhada
 - Chamadas de sistema necessárias apenas para estabelecer região
 - Após, qualquer acesso é realizado **sem assistência do kernel**
 - Comunicação mais **rápida e conveniente**

Principais mecanismos de IPC

- Comunicação inter-processos
 - Simplesmente informar outro processo que algum evento ocorreu; ou
 - Envolver transferência de dados de um processo para outro
 - Mecanismos de IPC
 - Pipes
 - Sinais
 - Filas de mensagens
 - Shared-memory
 - Semáforos (**estudados na aula 05**)
 - Sockets (**serão vistos na aula 11**)
 - RPC/RMI (**serão vistos na aula 14**)
 - Web sockets (**serão vistos na aula 15**)
 - Web services (**serão vistos na aula 16**)
- Todos permitem que processos troquem informações.
- Questão: como escolher?**
- Alguns são mais adequados para *aplicações particulares*
- Processos relacionados?
 - Comunicam por rede?
 - Unidirecional/bidirecional?
 - Mensagens curtas?
 - Acesso rápido?
 - Linguagem de programação?
 - Portabilidade?
 - ...

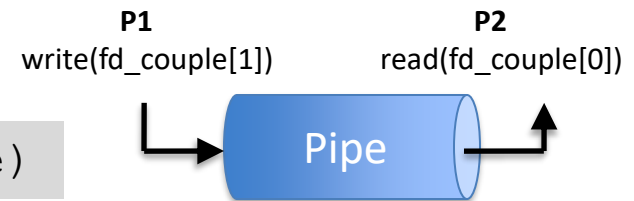
Pipes

```
#include <unistd.h>
```

- Provê um fluxo de dados **unidirecional** entre dois processos
 - Um processo escreve no pipe, outro processo lê do pipe
 - Implementado com um par de descritores de arquivo
 - O que é escrito no primeiro descritor pode ser lido no segundo (ordem FIFO)

- Criado por chamada de sistema

```
int pipe(int *fd_couple)
```



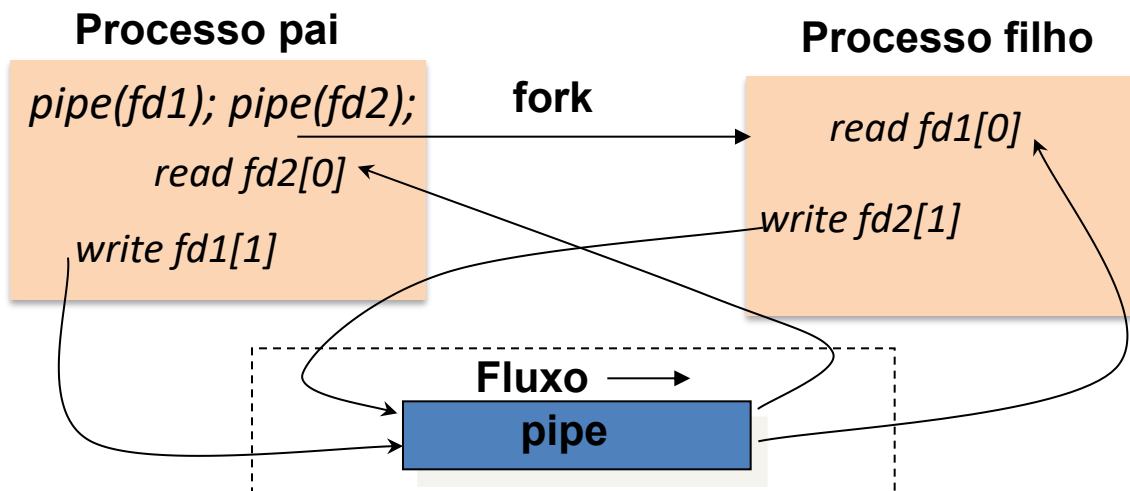
- Cria pipe e armazena descritores de arquivos `fd_couple[0]` (R) e `fd_couple[1]` (W)
 - Retorna -1 se ocorrer erro, 0 caso contrário
- Pipe é acessado por um dos descritores de arquivos
- Lido e escrito através de chamadas **read** e **write** convencionais
- Se comunicação **bidirecional** for necessária?
 - É preciso **usar 2 pipes**, cada pipe usado para enviar dados em uma direção

Detalhes sobre pipes simples

- Devem ser compartilhados pelos processos que desejam se comunicar (**apenas processos relacionados**)
- Situação genérica:
 - Processo cria dois pipes, e faz um `fork()` para criar processo filho
 - Filho herda descritores de arquivos abertos
 - Pai e filho utilizam os pipes

Alternativa: Named Pipes (FIFO)

- Comunicação pode ser bidirecional
- Relação pai/filho não é necessária

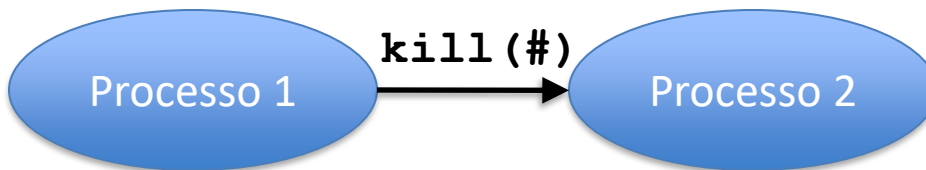


IMPORTANTE

- ① Cada pipe é uni-direcional apenas
- ② Atomicidade garantida para msg menor que PIPE_BUF bytes
- ③ Pipes meio abertos chamados de *widowed* (ou *broken*)
 - Escrita causa erro
 - Leitura retorna 0 bytes

Sinais (signals)

- Um dos primeiros mecanismos de IPC disponíveis no UNIX
 - Tipo de mensagem bastante **primitivo**
- Sinais podem interromper o fluxo de controle normal de um processo, mas **não tem capacidade de envio de dados**
 - Servem para indicar a ocorrência de um evento
 - Única informação gerada é o **número/tipo do sinal**
- Originalmente, sinais eram de um só tipo (como um *beep*) Como?
 - Quando recebido por processo, processo sempre reagia do mesmo modo
 - Historicamente, reação era terminar o processo → função para envio de sinais: `kill()`
 - Sinais foram tipificados, permitindo reagir de diferentes maneiras



Tratamento de sinais

<i>Tipo</i>	<i>Handler</i>
1	func_x()
2	func_y()

Geração de sinais

- Tipos de sinais suportados dependem da arquitetura do processador
 - Comportamento definido no padrão POSIX
- Tipos de sinais enviados a um processo
 - **Exceções**: núcleo notifica o processo falha de segmentação (SIGSEGV)
 - **Outros processos**: um processo encerra outro (SIGKILL)
 - **Notificações**: ocorrência de eventos como dispositivo pronto
 - **Alarmes**: programação de geração de alarmes (SIGALRM)
- UNIX e derivados de SVR4 e 4.4 BSD possuem 31 sinais
- API UNIX

```
#include <signal.h>
```

 - **signal()**: associa uma função de tratamento a tipo de sinal
 - **kill()**: envia um sinal

Tratamento de sinais

- Processo pode tratar o sinal de **três modos**
 - ① **Ignorar o sinal**
 - Exceto SIGSTOP e SIGKILL
 - ② **Capturar o sinal**
 - Implica na invocação de uma rotina específica para tratamento do sinal
 - ③ **Executar ação *default* definida pelo núcleo para aquele sinal**
 - Por default, núcleo define uma de 5 ações a ser executada
- **Ações *default***
 - **Abortar:** termina o processo imediatamente
 - **Memory dump:** termina o processo, mas antes gera *core dump*
 - **Ignorar:** simplesmente ignorar o sinal
 - **Suspender o processo:** também usada para depuração
 - **Continuar:** processo que estava previamente suspenso
 - Reverte ação prévia, passando estado de suspenso para pronto

Atendimento de sinais

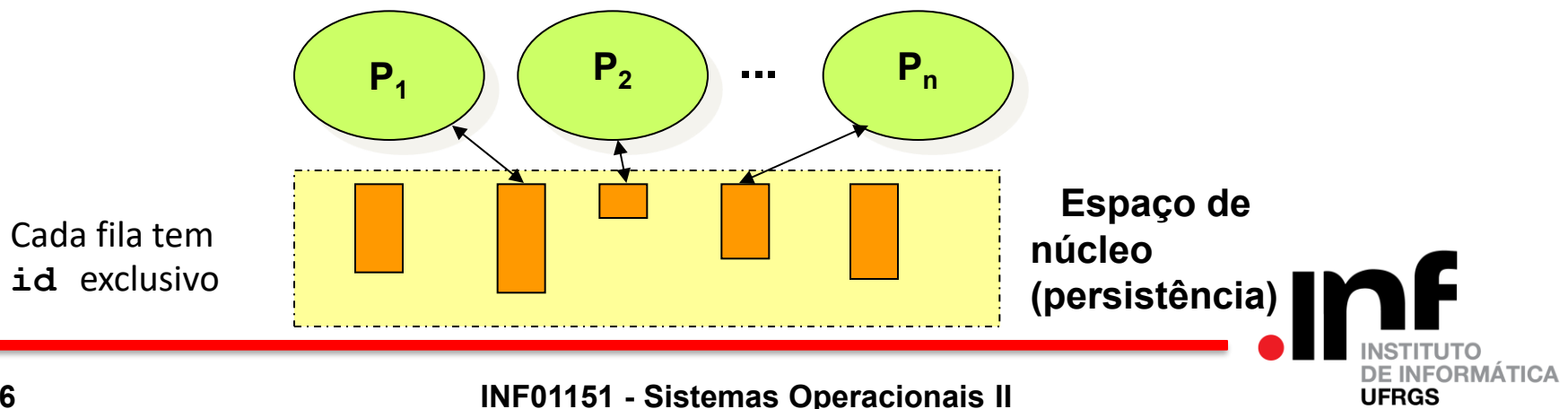
- Quem pode o que?
 - Processo A pode enviar sinal a processo B somente **se pertencem ao mesmo usuário**
 - Processos executando como superuser podem sinalizar **qualquer processo**
 - Sinais enviados **pelo próprio SO** podem não estar sujeitos a **tratamento customizado**
 - SIGSTOP e SIGKILL sempre terminaram o processo
- Quando gerados, os sinais ficam no estado “pendente” até serem encaminhados para o processo destino
 - Normalmente, o encaminhamento é rápido...

Atendimento de sinais - Importante

- Para notificar sinais, o sistema operacional tem uma **máscara de bits** para cada processo
 - Quando processo receber o sinal n , n -ésimo bit na máscara é marcado
 - Kernel chama função **issig()** para verificar sinais pendentes
 - Antes de processo bloquear e imediatamente após acordar
 - Antes de voltar para modo usuário (chamada de sistema ou interrupção)
 - Quando a função de tratamento terminar, o bit é repostado a 0
- Porém...
 - Se **issig()** não for executada antes que outros sinais do mesmo tipo cheguem, apenas o último será percebido pelo processo
 - Os outros sinais daquele tipo serão perdidos

Filas de mensagens

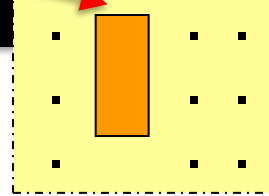
- Mecanismo que permite processos trocarem informações entre si
 - De forma completamente **assíncrona**
 - **Caixa postal** onde processos podem **depositar** e **retirar** mensagens
- Mensagens são compostas por
 - **Tipo de mensagem**
 - Não são definidas pelo núcleo; processos especificam seus próprios tipos de mensagens
 - **Área de dados de tamanho variável**
- Organizadas em filas de mensagens até que processo possa receber
 - Podem ser compartilhadas por processos que não são pai e filho



Filas de mensagens

- Descritores de **fila de mensagens**

- Fila de mensagens pendentes
- Fila de processos a espera de mensagens (receptores)
- Fila de processos a espera para enviar (emissores)
- Dados sobre tamanho e conteúdo da fila de mensagens



- Procedimento de envio e entrega

- Quando processo adiciona mensagem na fila, núcleo verifica lista de receptores esperando mensagens daquele tipo
 - Uma mensagem é entregue para **todos** processos **receptores** que a esperam
 - Se não houver processo esperando por mensagem daquele tipo
 - >> Havendo espaço: mensagem fica na **fila de mensagens** pendentes, e mensagens ficam armazenadas na fila até serem lidas
 - >> Não havendo espaço: processo se auto-insere na fila de **processos emissores** a espera para enviar
 - Persistência no kernel
- Qualquer processo pode* ler e escrever em qualquer fila
 - *Depende de autorização e permissão

API System V $\neq$ API POSIX

`send (msg, priority)`

Prioridade é usada para entrega de mensagens

SYSTEM V: usuário requisita explicitamente msg com prioridade específica (prioridade é usada como “endereço” da caixa postal)

POSIX: mensagem mais antiga com prioridade mais alta é entregue

Shared-memory

- Permite processos compartilhar uma área comum de memória
 - Comunicam-se através de leitura e escrita na área compartilhada
 - Processos devem garantir que não irão escrever no mesmo local simultaneamente
 - Como???
- Características
 - Mecanismo de simples compreensão para compartilhar dados
 - Não há transferência de dados
 - Acesso se faz através de operações de atribuição e ponteiros
 - Uma vez estabelecida a região compartilhada, acesso é feito em espaço de usuário sem intervenção do núcleo
 - Melhor desempenho por evitar trocas de modo/contexto

Shared-memory

- UNIX suporta duas interfaces padronizadas
 - **Shared-memory System V**
 - Região reside no espaço de endereçamento do processo criando o **segmento de memória compartilhada**
 - Outros processos que desejem se comunicar usando esse segmento **precisam se “anexar”** (*attach*) ao espaço de endereçamento
 - **Shared-memory POSIX**
 - Região é armazenada como um **arquivo temporário** montado em `/dev/shm`
 - Equivale a acessar arquivos (`open`, `read/write`, `close`)
 - Duas primitivas
 - `shm_open ( )`: cria um ponteiro para uma região de memória compartilhada
 - `shm_unlink ( )`: destrói ponteiro para região compartilhada

Shared-memory (System V)

- Criação de segmento de memória compartilhada

- Chamada shmget()

// SHared Memory GET

```
segment_id = shmget(IPC_PRIVATE, size, S_IRUSR | S_IWUSR);
```

Inteiro que
identifica o
segmento de
memória

- 1º parâmetro: key para o segmento; se for IPC_PRIVATE, novo segmento é criado
- 2º parâmetro: tamanho em bytes do segmento
- 3º parâmetro: modo de uso, i.e., read, write ou ambos

- Para que processo consiga acessar segmento, deve anexá-lo

- Chamada shmat()

// SHared Memory ATtach

```
shared_memory = (char *) shmat(segment_id, NULL, 0);
```

Ponteiro p/
início da
memória onde
a região
compartilhada
foi anexada

- 1º parâmetro: inteiro que identifica segmento de memória
- 2º parâmetro: ponteiro indicando onde a memória compartilhada será anexada
 - NULL = escolha do SO
- 3º parâmetro: flag indicando se região será read-only ou write-only
 - 0 = ambos

Shared-memory (System V)

- Após memória compartilhada ter sido anexada ao espaço de endereçamento de um processo
 - Acesso é feito como um acesso de leitura/escrita convencional
- Segmento compartilhado é identificado por `segment_id`
 - Deve ser conhecido pelos processos que o compartilham
 - Mantido pelo núcleo até que o segmento seja destruído e que não existam processos usando
- Primitivas adicionais:
 - `shmdt()` : desanexa segmento compartilhada
 - `shmctl()` : remove segmento de memória compartilhada do sistema

```
#include <stdio.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main (int argc, char *argv[]) {

    in segment_id;
    char *shared_memory;
    const int size = 4096;

    /* allocate shared memory segment */
    segment_id = shmget(IPC_PRIVATE, size, S_IRUSR | S_IWUSR);

    /* attach the shared memory segment */
    shared_memory = (char *) shmat(segment_id, NULL, 0);

    /* write a message to the shared memory segment */
    sprintf(shared_memory, "Hi there!");

    /* now print out the string from shared memory */
    printf("%s\n", shared_memory);

    /* detach the shared memory segment */
    shmdt(shared_memory);

    /* remove the shared memory segment */
    shmctl(segment_id, IPC_RMID, NULL);

    return 0;
}
```

Leituras adicionais

- Deitel, Deitel e Choffnes, Sistemas Operacionais, 3a edição, Pearson Prentice Hall
 - Capítulo 20 (seção 20.10)
- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 3 (seções 3.4 e 3.5)
- Interprocess communication / UNIX IPC programming
 - <http://mij.oltrelinux.com/devel/unixprg/>

INF01151 – Sistemas Operacionais II

Aula 10 - Comunicação inter-processos por troca de mensagens

Prof. Alberto Egon Schaeffer Filho

Introdução

- Em ambientes de **memória distribuída**, comunicação entre processos é feita por **troca de mensagem**
 - Processos **não compartilham** o mesmo espaço de endereçamento
 - Troca de mensagem
 - **TAMBÉM** pode ser empregada em ambientes de memória compartilhada
 - Capacidades de comunicação e/ou sincronização **dependem das primitivas** fornecidas pela biblioteca
 - `send(message)`
 - `receive(message)`
- Cada uma pode ser implementada com diferentes semânticas!!
- Associado a bibliotecas de comunicação
- Comunicação pode ser
 - Ponto a ponto
 - Grupo (multicast)

Características desejáveis

① Simplicidade e clareza de uso

- Semântica intuitiva

② Uniformidade

- Processos locais e remotos usam a mesma interface de comunicação

③ Eficiência

- Baixo custo de processamento
- Uso otimizado de recursos do sistema

④ Confiabilidade

- Garantir a entrega das mensagens em presença de falhas
- Eliminar (tratar) replicação de mensagens
- Entrega ordenada



Características desejáveis (continuação)

⑤ Correção (associado a comunicação em grupo)

- Atomicidade: todos ou nenhum recebem
- Entrega ordenada: mensagens entregues de acordo com algum critério

⑥ Flexibilidade

- Nem todas as aplicações tem as mesmas necessidades

⑦ Segurança

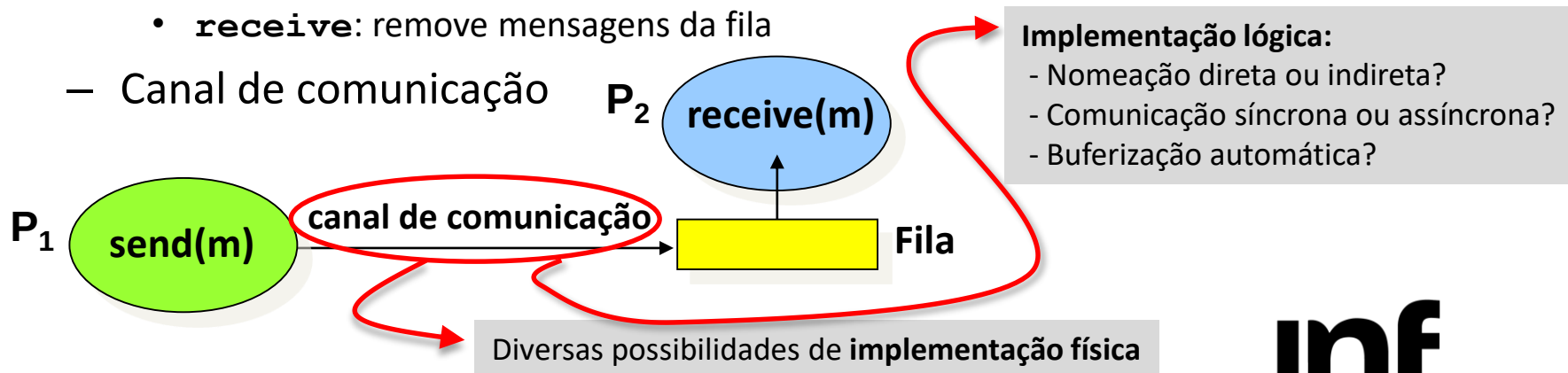
- Autenticação dos pares
- Cifragem das mensagens

⑧ Portabilidade

- Aplicações e da própria biblioteca (heterogeneidade)

Modelo de comunicação

- Duas primitivas básicas: **send** e **receive**
- Modelo
 - Origem (emissor, remetente) e Destino (receptor, destinatário)
 - Mensagem
 - Tamanho **fixo**: implementação do sistema simples, uso é mais difícil
 - Tamanho **variável**: implementação do sistema é mais difícil, uso se torna simples
 - Fila associada ao canal de comunicação
 - **send**: faz com que mensagens sejam adicionadas na fila
 - **receive**: remove mensagens da fila
 - Canal de comunicação



Semânticas das primitivas *send* e *receive*

- Tudo depende da semântica da primitiva...
 - **send**
 - Deve ser indicado um destinatário específico ou a mensagem pode ser entregue a qualquer processo?
 - Remetente fica bloqueado até a mensagem ser aceita pelo destinatário?
 - Há buferização de mensagens?
 - **receive**
 - Deve ser indicado um remetente específico ou se a mensagem pode ser proveniente de qualquer processo?
 - Receptor fica bloqueado se não houver mensagem a ser recebida?
 - Há buferização de mensagens?
- Remete a questões de nomeação e sincronismo ...

Nomeação direta ou indireta

- Processos precisam referenciar uns aos outros
 - Indicar se a comunicação é **direta/explícita** ou **indireta/implícita**
 - Nomear ou não o processo com o qual se deseja interagir
- Nomeação **explícita (direta)**
 - Determinação de processo destino/remetente específicos
 - `send(P, message)`: Envia mensagem para processo P
 - `receive(Q, message)`: Recebe mensagem de processo Q
 - Propriedades
 - Canal estabelecido automaticamente entre cada par de processos, precisam apenas saber as identidades de cada processo
 - Exatamente um canal associado a exatamente dois processos
 - Simetria de endereçamento: cada processo precisa saber o nome do outro
 - Variante assimétrica
 - `send(P, message)`: Envia mensagem para processo P
 - `receive("any", message)`: Recebe msg de qualquer processo

Limitação?

Nomeação direta ou indireta

- Nomeação **implícita (indireta)**

Ver também: JavaSpaces, Rinda, PyLinda, etc

- Mensagens são enviadas e recebidas através de **caixas postais**
 - Cada caixa postal possui um **id único**
 - Mensagens podem ser **colocadas** e **retiradas**
- Exemplo típico: fila de mensagens (`MessageQueue` no UNIX)
- Processos só podem se comunicar se compartilham uma caixa postal
 - `send(A, message)`: Envia mensagem para caixa postal A
 - `receive(A, message)`: Recebe mensagem da caixa postal A
- Propriedades
 - Canal só é estabelecido entre um par de processos se ambos compartilham a caixa
 - Canal pode estar associado a mais de dois processos
 - Entre cada par de processos, podem haver diferentes canais. Como?
- P1, P2 e P3 compartilham caixa postal A: **Quem recebe? P2 ou P3?**

Diferentes caixas postais!

```
P1: send(A, message);
```

```
P2: receive(A, message);
```

```
P3: receive(A, message);
```

Comunicação síncrona ou assíncrona

- Comunicação entre processos através das primitivas **send/receive**. Como implementar: bloqueantes ou não?
 - Envio **bloqueante**: processo que faz **send** fica bloqueado até mensagem ser recebida pelo outro processo ou pela mailbox
 - Envio **não-bloqueante**: processo que faz **send** continua a execução após chamar a primitiva
 - Recebimento **bloqueante**: processo que faz **receive** bloqueia até que alguma mensagem esteja disponível
 - Recebimento **não-bloqueante**: processo que faz **receive** obtém uma mensagem válida ou NULL
- Diferentes combinações são possíveis ...
 - Com **send** e **receive bloqueantes**  comunicação síncrona
 - Situação chamada de “rendezvous”
 - Com **send** ou **receive não-bloqueantes**
 - Há apenas mecanismo de comunicação, não de sincronização

Buferização

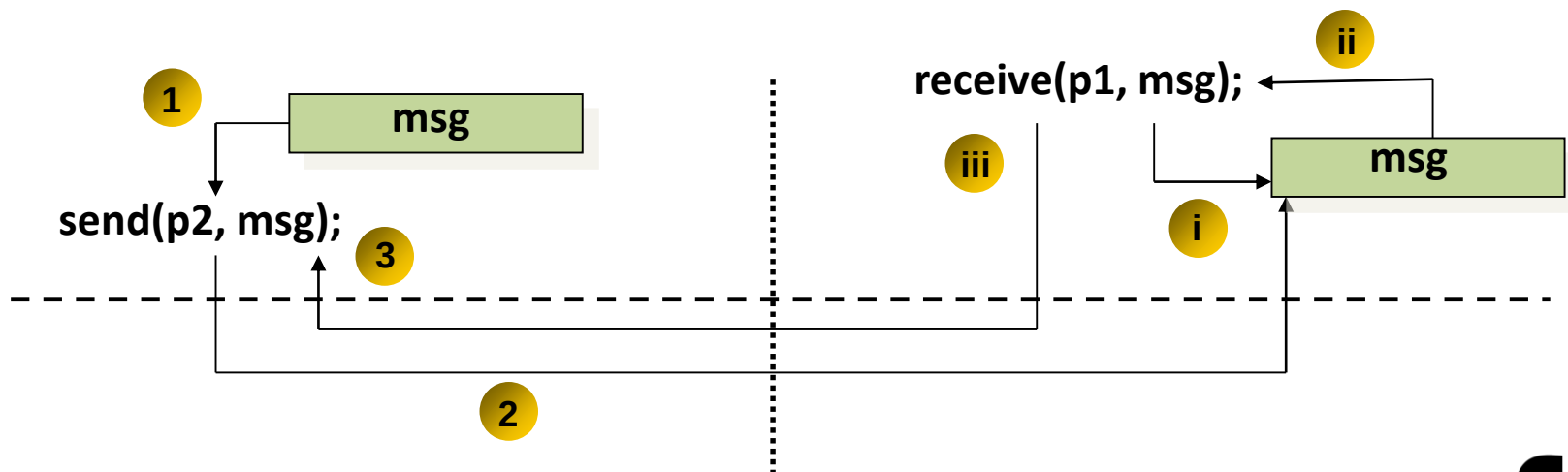
- Em canal de comunicação, direto ou indireto, mensagens são mantidas em fila temporária associada ao canal.
- Três alternativas:
 - **Zero capacity**
 - Fila tem tamanho zero, e canal não pode manter nenhuma mensagem pendente
 - **send** deve ser **bloqueante** até que outro processo possa receber a mensagem
 - Maior eficiência: não é feito cópia local dos dados
 - **Bounded capacity**
 - Fila tem tamanho fixo n , e canal pode manter até n mensagens pendentes
 - Se a **fila não estiver cheia**, quando nova mensagem é enviada **send** é **não-bloqueante**
 - Se a **fila estiver cheia**, quando nova mensagem é enviada **send** deve **bloquear**
 - **Unbounded capacity**
 - **send** sempre é **não-bloqueante**
 - Na prática, não existe; n pode ser muito superior aos casos usuais

Sem buferização

Buferização automática

Implementação comunicação síncrona

- Primitiva **send** e **receive** forem **bloqueantes**
 - Dispensa o uso de fila e buffers
- Procedimento:
 - **send**: processo fica bloqueado até que a recepção seja efetivada
 - **receive**: processo fica bloqueado até que o envio seja feito e a mensagem seja recebida



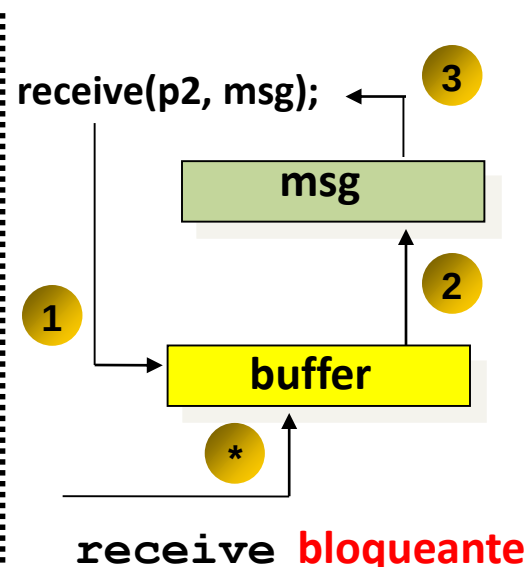
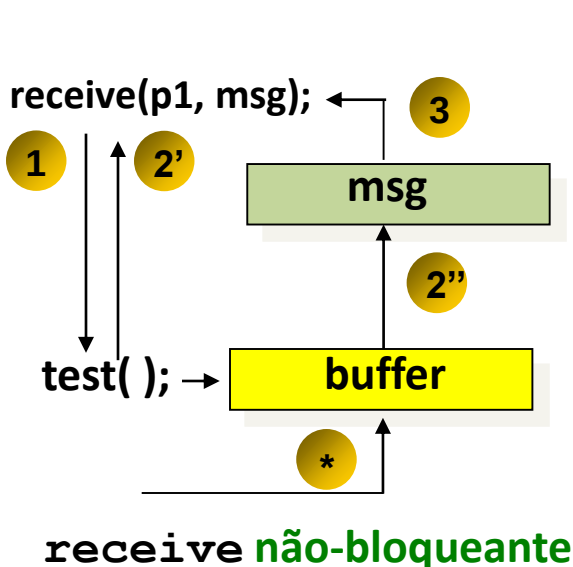
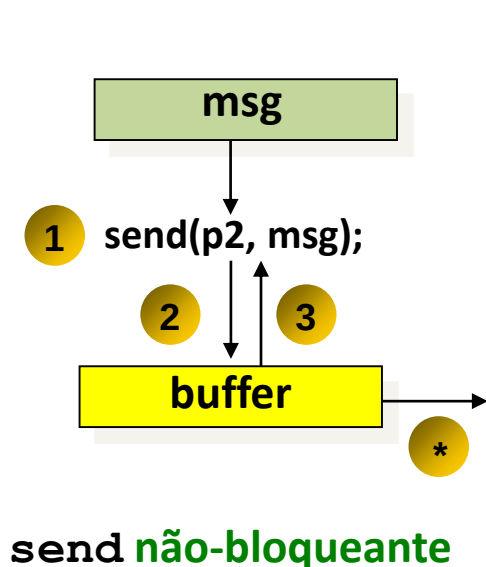
Implementação comunicação assíncrona

- Envolve a utilização de buffers

- **send** é **não-bloqueante**
- **receive** pode ser
 - **Não-bloqueante**
 - **Bloqueante**

E na prática?

Receive é bloqueante: Uma thread pode ficar bloqueada em um receive, enquanto outras threads executam



* Momento de tempo não preciso!

Comparação: comunicação síncrona vs. assíncrona

- Comunicação:
 - **Síncrona:** processos usam versão bloqueante das primitivas
 - **Assíncrona:** processos usam versão não-bloqueante de pelo menos uma das primitivas
- Desvantagens da comunicação síncrona?
 - Limitação da concorrência
 - Possibilidade de *deadlocks*
 - Como??
- Vantagens da comunicação síncrona?
 - Programação simples

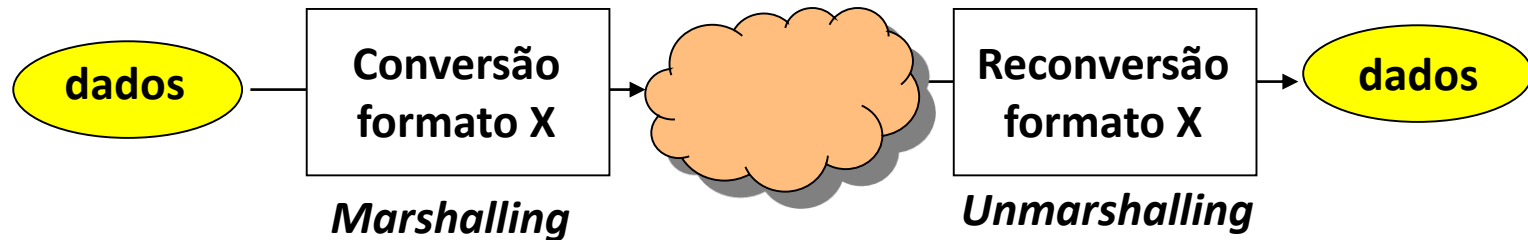
Vantagens e desvantagens da assíncrona são “*dual*” da síncrona

Representação externa de dados

- Informação (dados) é representada em uma estrutura de dados
 - Na comunicação informação é igual a uma seqüência de bytes
 - Necessário converter estrutura de dados $\leftrightarrow$ seqüência de bytes
- Problema se processos estão executando em sistemas remotos
 - Diferentes sistemas = diferentes representações de dados
 - Código ASCII usa 1 byte por caracter vs. Unicode usa 2 bytes por caracter
 - Big endian (byte mais significativo 1º) vs. little endian (byte menos significativo 1º)
 - Formatos de números em ponto flutuante, etc
- Como resolver?
 - ① Representação em um formato independente de máquina
 - Formato de dados acordado (*external data representation*)
 - ② Enviar no formato do emissor e incluir informação sobre o formato empregado

Representação externa de dados

- Marshalling e unmarshalling
 - Conversão de dados em um formato próprio para transmissão
 - Envio: conversão formato local → *external data representation*
 - Recebimento: conversão *external data representation* → formato local
 - Auxilia a tratar a heterogeneidade de representação de dados entre os processos



- Ideal é que seja feito sem envolvimento explícito da aplicação
 - Responsabilidade da biblioteca ou *middleware*
 - Exemplos:
 - **CORBA CDR**: representação de tipos tipos estruturados e primitivos (binário)
 - **Serialização Java**: utilizado para converter objetos em bytestream (binário)
 - **XML/JSON**: utilizado para representar dados trocados por web services

Modelo de falhas

- Processos e/ou canal de comunicação podem falhar
 - Especialmente se **comunicação for distribuída**
 - Transmissões podem falhar, ou serem executadas mais de uma vez
 - Erros de transmissão na rede
- Modelo de falhas
 - Define **como falhas podem se manifestar** de forma a proporcionar um entendimento sobre seus efeitos e conseqüências
- Tipos de falhas:
 - **Omissão**: processo e/ou comunicação não executam sua função
 - Ex: processo termina inesperadamente, mensagem não chega ao destino
 - **Arbitrária (bizantina)**: processo e/ou comunicação executam com erros
 - Ex: mensagem chega no destino com erro, duplicada e/ou fora de ordem
 - **Temporização**: quando limites de tempo não são respeitados
 - Ex: transmissão da mensagem ultrapassa limite estabelecido

Modelo de falhas (continuação)

- Mascaramento de falhas
 - Sistemas geralmente construídos através de coleção de outros componentes
 - Possível construir um sistema confiável a partir de componentes propensos a falhas
 - Quando um sistema **esconde que houve uma falha** ou **transforma falha em um tipo menos grave** (mais “aceitável”)
 - Ex: checksum usado para mascarar mensagem corrompida (bizantina → omissão)
 - Em alguns sistemas, é possível esconder falhas de omissão através de retransmissão
 - Outras técnicas: retransmissão de mensagens, replicação dados/serviços, restaurar processo com info gravada em disco, etc etc
- Confiabilidade na comunicação
 - Definido em termos de:
 - **Validade**: mensagem é em algum momento entregue (garantia)
 - **Integridade**: o que é recebido é exatamente o que foi enviado
 - Tratar duplicação, mensagens espúrias, modificadas

Leituras adicionais

- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 3 (seções 3.4 e 3.5)
- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.– *“Distributed Systems: Concepts and Design”* (5th edition), Addison Wesley, 2012
 - Capítulos 2 (seção 2.4), 4 (seções 4.1, 4.2 e 4.3)

Exercícios

1. Implementar o problema do produtor consumidor, utilizando 1 único produtor, 1 único consumidor. Os processos devem produzir/consumir 1 item por vez: produtor não deve produzir duas vezes em sequência, consumidor não deve consumir duas vezes em sequência. Utilize **send/receive**. Faça o esboço da solução.
 - Preciso de buferização?
 - Comunicação direta ou indireta?
 - Send/receive bloqueante ou não-bloqueante?
 - Preciso de primitivas de sincronização como mutex/semáforos?

INF01151 – Sistemas Operacionais II

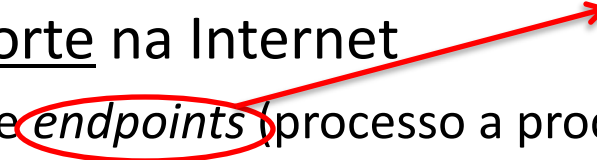
Aula 11 - Estudo de caso: sockets API

Prof. Alberto Egon Schaeffer Filho

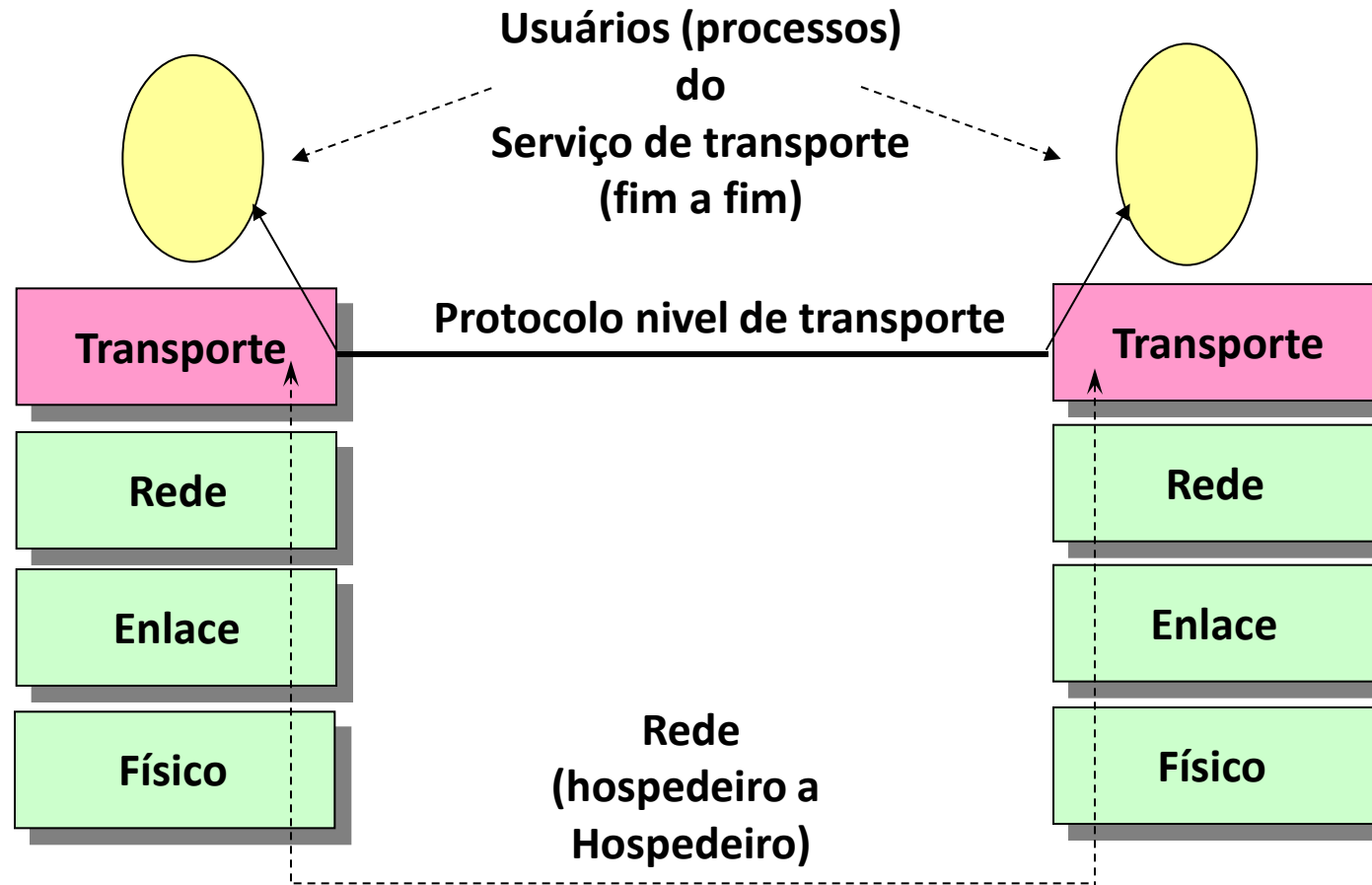
Introdução

- Mecanismo de comunicação **bidirecional** entre processos por troca de mensagens
 - Entre **aplicações** (processos não-relacionados) e **diferentes máquinas**
 - Baseado nos protocolos de transporte da Internet (UDP e TCP)
- Interface de programação (API) para ambientes de redes
 - Independente de rede específica, mas **IP é o uso mais popular** de sockets
 - Conjunto de funções implementadas por uma biblioteca
 - Características desejáveis:
 - Genérica/uniforme
 - Independente de sistema operacional
 - Suporte a comunicações orientadas a mensagens e a conexão (flexibilidade)
 - Representação externa de dados (inteiros, strings, *byte order* etc.)
- Aplicações em rede seguindo modelo cliente/servidor

Mas antes: uma “revisão rápida” sobre redes de computadores...

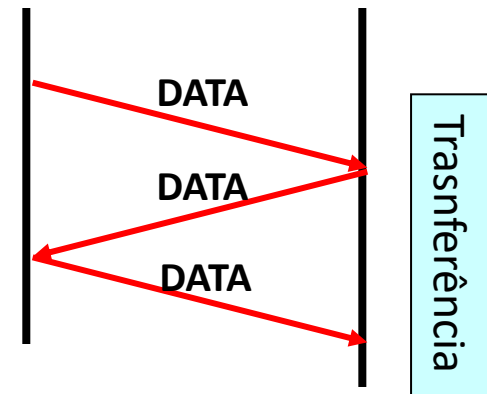
- Camada de rede na Internet
 - Endereço IP
 - Identificador de máquina
 - Embute informações de roteamento
 - Camada de transporte na Internet
 - Comunicação entre endpoints (processo a processo)
 - Processos são identificados por portas
 - Serviços:
 - Orientado a conexão (Transmission Control Protocol – TCP)
 - Não orientado a conexão (User Datagram Protocol – UDP)
- Socket provê abstração para endpoint
- 

Contexto dos protocolos das camadas de rede e de transporte



User Datagram Protocol (UDP)

- Não orientado a conexão
- Noção de mensagem (*datagram*)
 - Dados são delimitados na T-PDU
- Modelo de falhas:
 - **Validade:** falhas por omissão, isso é, não entrega
 - **Integridade:** pode haver perdas, erros de ordenamento e duplicação

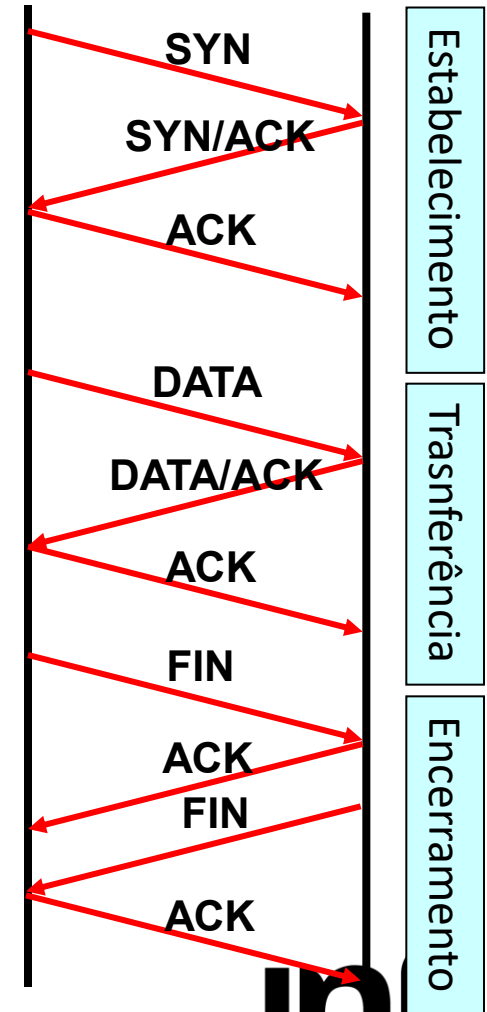


PDU = *Protocol Data Unit* (equivale a um “pacote” com cabeçalho e dados)

T = camada de transporte

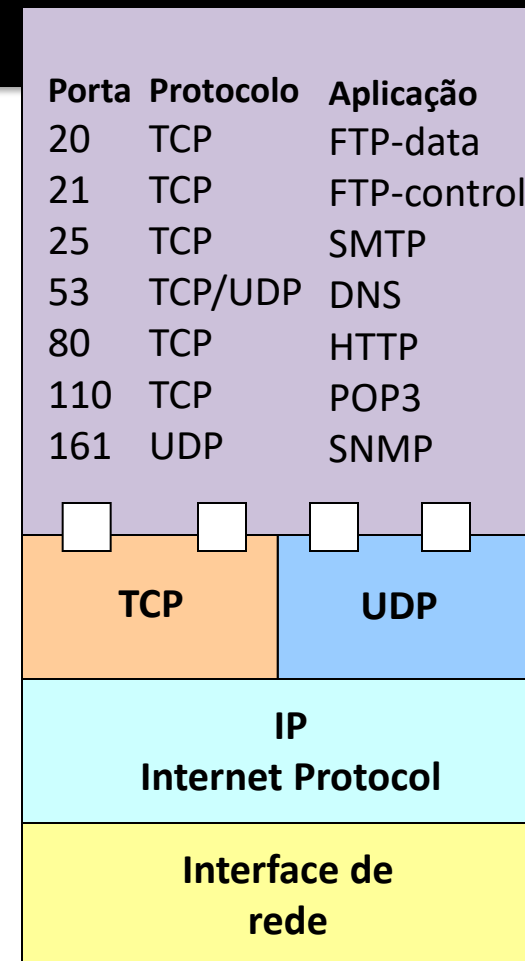
Transmission Control Protocol (TCP)

- Orientado a conexão
- Noção de fluxo de dados (*byte stream*)
 - Dados não são delimitados pela T-PDU
- Modelo falhas:
 - **Validade:** garante a entrega das mensagens
 - **Integridade:** garante ordenamento e não duplicação de mensagens
- Conexão fornece garantia de entrega, não duplicação e ordem



O conceito de porta

- Número de 16 bits usado como identificador
- Três tipos de portas
 - Bem conhecidas (*well-know ports*): 1 a 1023
 - *Registered ports*: 1024 a 49151
 - *Dynamics and/or private ports*: 49152 a 65535
- Uma aplicação é **completamente identificada** por:
 - **End. IP + Porta + Protocolo** (TCP ou UDP)
 - Uma porta não pode ser compartilhada por vários processos (exceção: *multicast IP*)
 - Portas TCP são independentes de Portas UDP
 - Porta 100 (TCP) ≠ Porta 100 (UDP), mas se convencionou “alocar” as duas simultaneamente para uma mesma aplicação



Modelo cliente-servidor

- Paradigma muito usado na implementação de aplicações em rede
 - Pode ser usado com serviços orientados a conexão ou não
- Modelo assimétrico (diferentes papéis)
 - Cliente envia requisições para o servidor
 - Servidor envia respostas para o cliente
- Necessário identificar
 - Máquina: endereço IP
 - Protocolo: TCP ou UDP
 - Processo: Porta

• Cliente

- Usuário de um serviço provido por um processo (servidor)
- Em serviço orientado a conexão, cliente solicita abertura da conexão
- Solicita execução do serviço no servidor e espera uma resposta

Implementação lógica: **tipo de nomeação?**

• Servidor

- Fornece um serviço (aplicação) a outros processos (clientes)
- Espera passivamente a solicitação de clientes

Arquitetura TCP/IP e programação

- Arquitetura TCP/IP não define uma interface de programação
- Funções necessárias:
 - Especificar ponto de comunicação local e remoto
 - Iniciar uma comunicação, se TCP:
 - Esperar por um pedido de abertura de conexão
 - Encerrar uma conexão de forma negociada (graciosa) ou não
 - Enviar e receber dados
 - Tratamento de erro
- Existem várias formas de se programar usando TCP/IP
 - Sockets (padrão POSIX), TLI, *winsock*, MacTCP etc

Sockets

- Abstração de um ponto de comunicação (*endpoint*)
- Características básicas:
 - Permite comunicação bidirecional
 - Prove suporte a vários tipos de protocolos
 - Confiabilidade da comunicação depende
 - Orientado a conexão (TCP)
 - Não orientado a conexão (UDP)
- API *socket* é genérica contendo
 - Funções para criar, nomear, conectar e destruir *sockets*
 - Esperar pedido de conexão
 - Funções para envio e recepção
 - Configuração, gerenciamento e controle de *sockets*
 - Tratamento de erros

Implementação de sockets no UNIX

- Mecanismos de comunicação bidirecional
- Dois domínios básicos:
 - Protocolos de rede da Internet (IPv4 = AF_INET)
 - Canal de comunicação é definido pela associação (5-tupla)
 - [IP_destino, Porta_destino, IP_fonte, Porta_fonte, Protocolo]
 - Envolve um par de sockets (*endpoint* local e *endpoint* remoto)
 - Protocolo de comunicação local UNIX (AF_UNIX)
 - Permite comunicação com interface sockets em um mesmo host
 - Outro mecanismo de IPC local
 - Vantagem é o desempenho (se comparado com AF_INET)
 - Canal de comunicação é um “nome” no sistema de arquivos
 - Semelhantes a **named pipes**

Interface de sockets

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
```

- Socket é um descritor de arquivo
 - Paradigma abrir-ler-escrever-fechar
- As primitivas básicas são:
 - `socket( )`
 - `bind( )`
 - `listen( )`
 - `accept( )`
 - `connect( )`
 - `write( )`
 - `sendto( )`
 - `read( )`
 - `recvfrom( )`
 - `close( )`

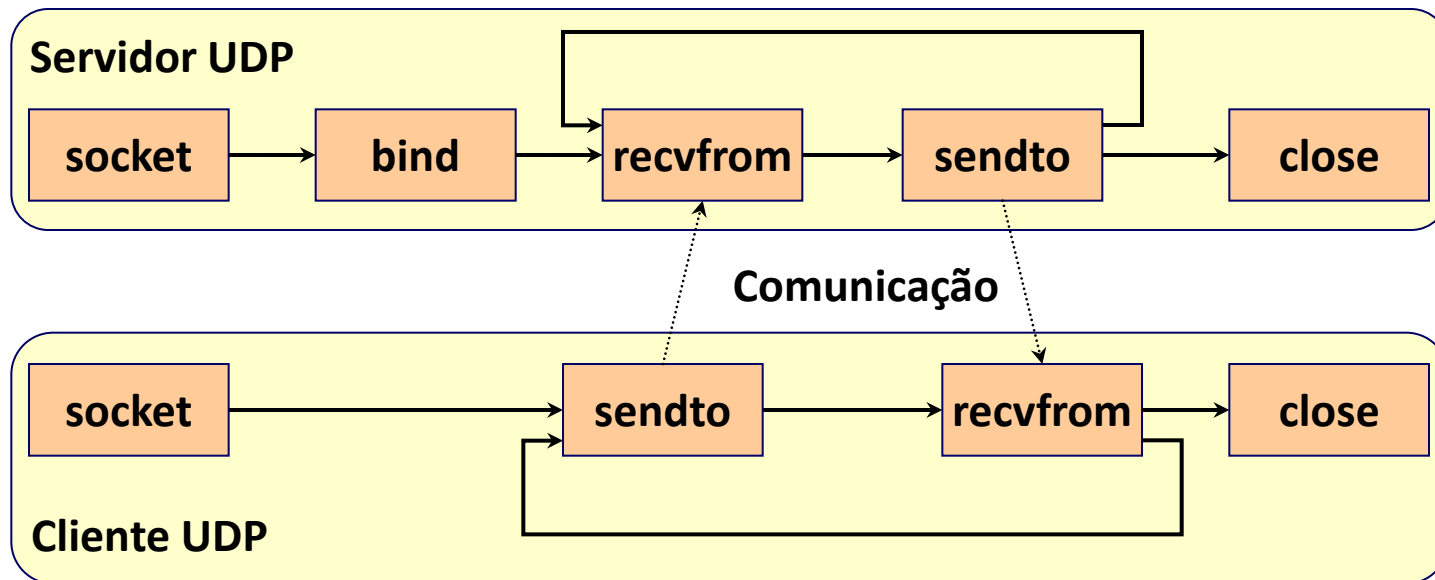
	TCP	UDP
Cliente	<code>socket()</code> <code>connect()</code> <code>read()</code> <code>write()</code> <code>close()</code>	<code>socket()</code> <code>recvfrom()</code> <code>sendto()</code> <code>close()</code>
Servidor	<code>socket()</code> <code>bind()</code> <code>listen()</code> <code>accept()</code> <code>read()</code> <code>write()</code> <code>close()</code>	<code>socket()</code> <code>bind()</code> <code>recvfrom()</code> <code>sendto()</code> <code>close()</code>

Comunicação UDP (por datagrama)

- Baseado em unidade de transmissão (*datagram*)
 - Tamanho limitado
 - Transmitido sem confirmações ou tentativas de reenvio
 - Mensagens podem não chegar
- Aspectos a serem considerados:
 - Necessidade de especificar o tamanho da área de recepção
 - Se menor que o necessário → mensagem truncada
 - Bloqueio: `send` é não bloqueante e o `receive` é bloqueante
 - Time-out: limitar o tempo de espera do `receive` configurável via *sockoption*
 - Recepção anônima
- Correspondência entre itens de dados
 - Processos devem “concordar” quanto ao conteúdo dos dados
 - Se um envia n bytes como sendo um inteiro, outro deve ler como tal

Como tratar a **representação externa de dados**?

Clientes e servidores UDP

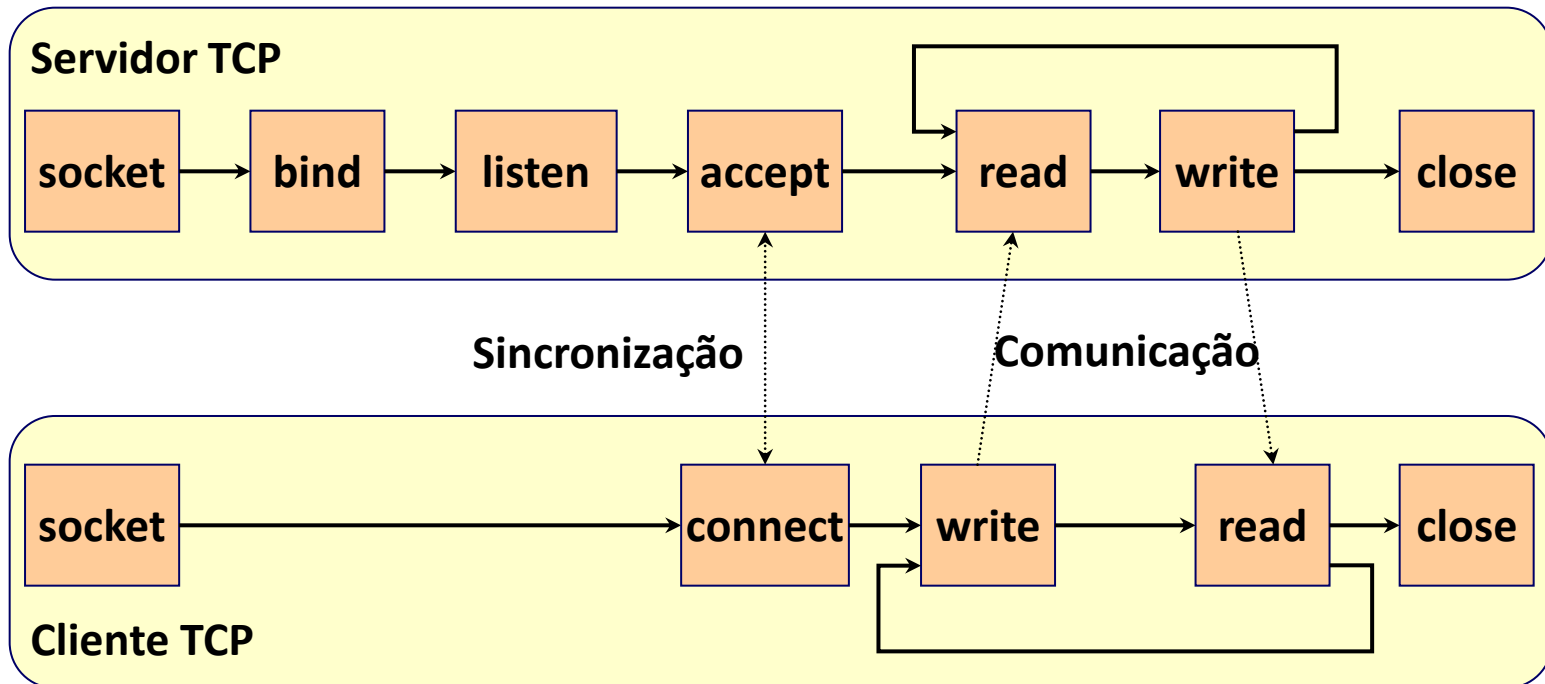


Comunicação TCP (por fluxo)

- Baseado em abstração de fluxo (*stream*)
 - Não define limites de tamanho para as mensagens (tamanho variável)
 - Características de um fluxo TCP
 - Garantia de entrega, entrega ordenada e a não-duplicação
- Aspectos a serem considerados:
 - Bloqueio
 - Processo destino é bloqueado se tenta ler dados não disponíveis
 - Processo remetente é bloqueado pelo controle de fluxo do TCP, se não houver espaço disponível no destinatário para recepção
 - Threads
 - Recomendável para simplificar a programação (bloqueante)
- Correspondência entre itens de dados
 - Processos devem “concordar” quanto ao conteúdo dos dados
 - Se um envia n bytes como sendo um inteiro, o outro deve ler como tal

Como tratar a **representação externa de dados**?

Cientes e servidores TCP



socket() – Obtendo o descritor

- Executado por qualquer processo que queira utilizar um socket

```
int socket(int domain, int type, int protocol)
```

- **domain** especifica família de endereçamento
 - AF_INET (IPv4)
 - AF_INET6 (IPv6)
 - AF_UNIX (canal local)
 - AF_NS (Xerox)
- **type** (para AF_INET)
 - Não orientado a conexão (UDP): SOCK_DGRAM
 - Orientado a conexão (TCP): SOCK_STREAM
- **protocol**: protocolo a ser usado para aquele tipo de serviço
 - Caso exista mais de um protocolo oferecendo o mesmo tipo de serviço
 - Valor 0 indica *default*

bind() – Associando endereço/porta

- Executado pelo **servidor** para atribuir uma identidade ao socket
 - Sockets devem funcionar com diversos tipos de comunicação
 - Necessário ser parametrizado com estrutura de endereço

```
int bind(int socket, struct sockaddr *address, socklen_t address_len)
```

- **socket** especifica o descritor do socket
 - Obtido através da chamada a função `socket()`
- **address**
 - Struct `sockaddr` é um container genérico
 - Diversas variações dependendo da família
 - para `AF_INET` `sockaddr_in`; para `AF_UNIX` `sockaddr_un`
 - Contém elementos pertinentes aquele tipo de comunicação
 - Estrutura deve ser preenchida antes de se chamar o `bind()`
- **address_len**
 - Necessário especificar tamanho da estrutura

Endereço de um socket AF_UNIX

- Definido em uma estrutura especial (`struct sockaddr_un`)
 - Passado para as chamadas da biblioteca *socket*
 - Família AF_UNIX
 - Pathname: nome do arquivo
- Endereço de um socket no domínio UNIX é string de caracteres
 - Essencialmente é uma entrada no sistema de arquivos

```
struct sockaddr_un {  
    sa_family_t    sun_family;    /* AF_UNIX */  
    char           sun_path[ ];    /* Pathname */  
};
```

Endereço de um socket AF_INET

- Definido em uma estrutura especial (`struct sockaddr_in`)
- Necessário posicionar:
 - Endereços IP (32 bits para o IPv4)
 - Endereço porta (16 bits)

```
struct sockaddr_in {  
    uint_t          sin_len;  
    sa_family_t     sin_family;          /* AF_INET */  
    in_port_t       sin_port;           /* Port number */  
    struct in_addr   sin_addr;          /* Internet address */  
    char            sin_zero[8];  
};
```



```
struct in_addr {  
    in_addr_t      s_addr;  
};
```

listen() – Disponível para receber pedidos

- Executado pelo **servidor** para informar que socket pode receber conexões
 - Apenas para serviços **orientados a conexão** (SOCK_STREAM)
- Necessário armazenar pedidos de aberturas de conexão até que eles possam ser atendidos
 - Implica em ter uma fila (e um tamanho) desses pedidos
 - Em caso de overflow, os pedidos de conexão são perdidos

```
int listen(int socket, int backlog)
```

- **socket**
 - Especifica o descritor do socket
- **backlog**
 - Define o número máximo de conexões que podem ficar pendentes

accept() – Aceitando conexões

- Executado pelo **servidor** para receber conexão
 - Apenas para serviços **orientados a conexões** (SOCK_STREAM)
 - Recupera o primeiro pedido da fila de requisições (`listen`)
 - Cria novo socket para tratar uma determinada requisição
 - Socket **original** usado **apenas** para **aceitar conexões**, não para troca de dados
 - Default é bloquear se a fila estiver vazia
 - Mas é possível modificar com opção `O_NONBLOCK` passada via chamada `fcntl()`

```
int accept(int socket, struct sockaddr *address, size_t
*address_len)
--socket
```

- Socket disponível para aceitar conexões (`listen`)
- **address**
 - Estrutura que é preenchida (automaticamente) com o endereço do cliente
- **address_len**
 - Tamanho da estrutura de endereçamento

connect() – Requisitando conexões

- Executado pelo **cliente** para solicitar abertura de conexão
 - Apenas para serviços **orientados a conexões** (SOCK_STREAM)
 - Default é **bloquear** até a conexão ser aceita ou expirar timeout
 - Mas é possível modificar comportamento via `fcntl()` com opção `O_NONBLOCK`

```
int connect(int socket, struct sockaddr *address, size_t address_len)
```

- **socket** especifica o descritor do socket
 - Obtido através da chamada a função `socket()`
- **address**
 - Endereço e porta do (socket do) servidor remoto
 - Struct `sockaddr` é um container genérico
 - Diversas variações dependendo da família
 - para `AF_INET` `sockaddr_in`; para `AF_UNIX` `sockaddr_un`
- **address_len**
 - Necessário especificar tamanho da estrutura

close() – Encerrando um socket

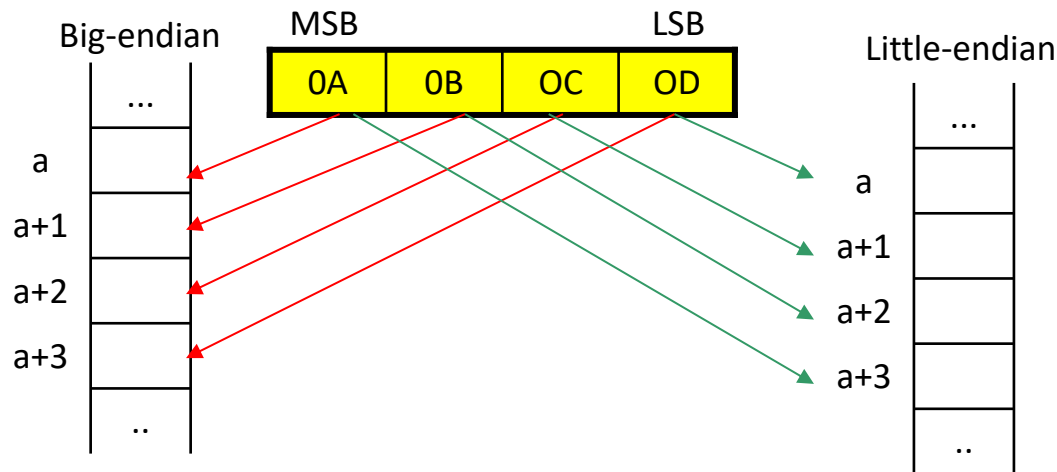
- Terminar uma conexão entre um cliente e um servidor
- Chamada `close()`
 - Similar ao que se faz com um arquivo
 - Deve-se chamar o encerramento em ambos os lados (cliente e servidor) para liberar descritores
- O encerramento pode não ser imediato se ainda houver dados em trânsito

Exemplos de código:

```
client_udp.c  
client_tcp.c
```

```
server_udp.c  
server_tcp.c
```

Uma armadilha: *network byte order*



- Protocolos Internet empregam *big-endian (network order)*
 - Necessário fazer programas que funcionem independente da representação interna da máquina (*host order*)
 - Tipicamente as informações de porta e de endereço IP
 - Funções específicas para auxiliar nesse tratamento

Uma armadilha: *network byte order*

- Os valores a serem armazenados em `sockadd_in` devem ser fornecidos em **network byte order**
 - `sin_port` e `sin_addr`
- Funções específicas para tratar esse aspecto

`uint16_t`
`uint16_t`

`htons (uint16_t) ;`
`ntohs (uint16_t) ;`

`uint32_t`
`uint32_t`

`htonl (uint32_t) ;`
`ntohl (uint32_t) ;`



h: host byte order
n: network byte order
s: short (16 bits)
l: long (32 bits)

Funções de conversão

- Conversão de endereços IP (ASCII) em formato de rede (binário)
- Funções para IPv4
 - `inet_aton`: end. IPv4 (ASCII) para binário
 - `inet_ntoa`: binário para end. IPv4 (ASCII)
 - `inet_addr` (obsoleta): similar a `inet_aton`
- Funções para IPv4 e IPv6
 - `inet_pton`: presentation to network
 - `inet_ntop`: network to presentation

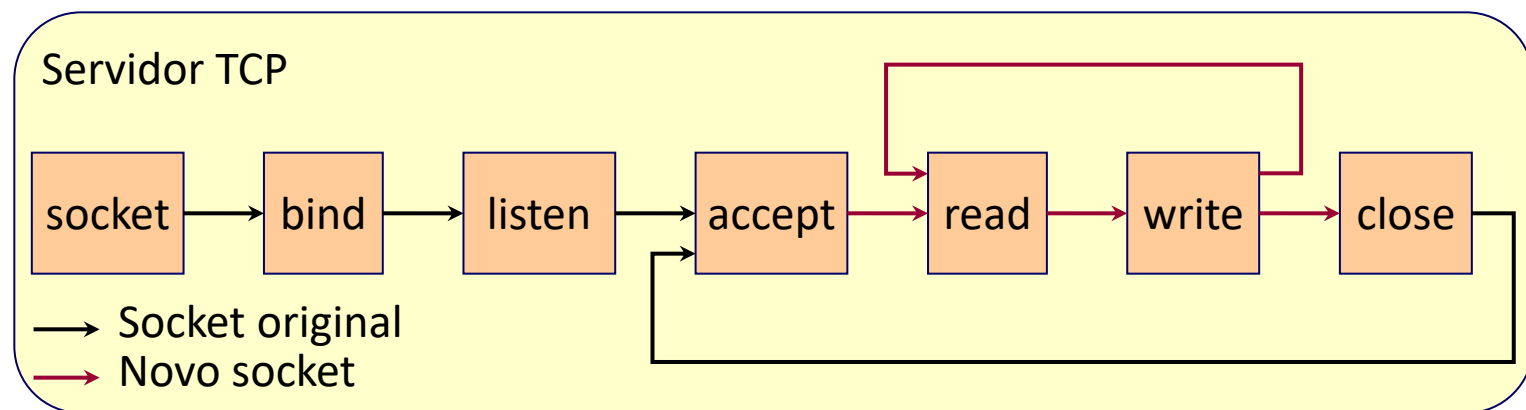
Necessário indicar a família (`AF_INET` ou `AF_INET6`)

Servidor iterativo vs. Servidor concorrente

- Em função de como o servidor executa o tratamento de requisições
 - Servidor iterativo (**single-threaded**)
 - Trata requisições de um único cliente a cada instante
 - Implementado como um único processo
 - Servidor concorrente (**multi-threaded**)
 - Trata simultaneamente requisições de vários clientes
 - Implementado com vários processos ou threads independentes
 - Cada thread trata individualmente requisições de um determinado cliente

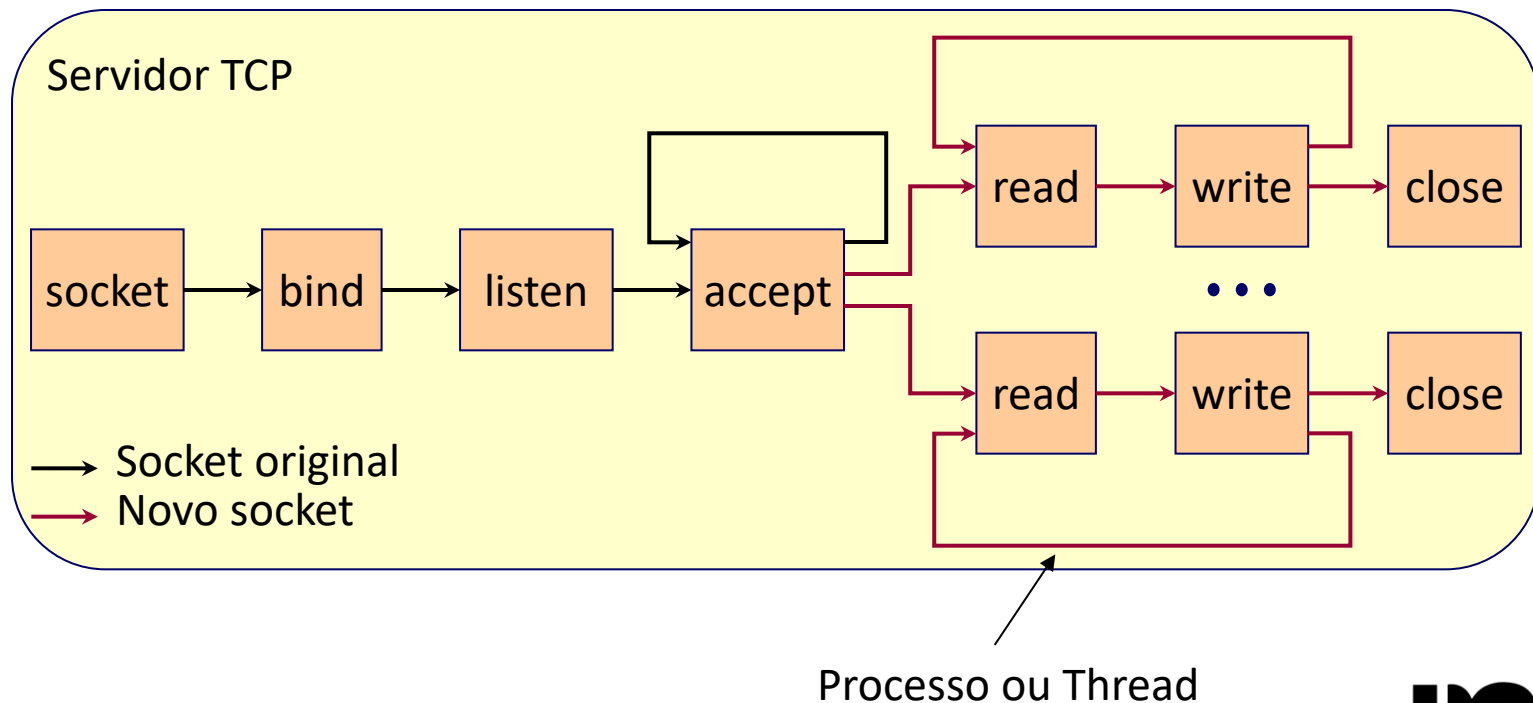
Servidor iterativo

- Adequado para serviços com reduzida taxa de requisições e serviços que possuem baixa carga de processamento



Servidor concorrente

- Adequado para serviços com elevada taxa de requisições e serviços que possuem alta carga de processamento



Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 4 (seção 4.2)
- Silberschatz, A. and Galvin, P. “*Operating Systems Concepts*”.
Wiley, 8th edition, 2009.
 - Capítulo 3 (seção 3.6)
- Beej's Guide to Network Programming - Using Internet Sockets
 - <http://beej.us/guide/bgnet/>
- Socket Programming
 - <https://www.ibm.com/docs/en/i/7.4.0?topic=communications-socket-programming>

INF01151 – Sistemas Operacionais II

Aula 12 - Comunicação confiável entre processos

Prof. Alberto Egon Schaeffer Filho

Introdução

- Problema
 - Comunicação entre processos pode falhar devido a problemas de interconexão (enlace) ou falhas no cliente ou no servidor
 - Falhas de omissão, mensagens duplicadas, mensagens entregues fora de ordem

- Métodos para tratar falhas

- Confirmações (ACK)
 - Envia uma confirmação a origem da mensagem
- Timeout
 - Tempo máximo de espera para receber confirmação, pode indicar ao cliente falha
 - **Requisição** do cliente **nunca chegou** ao servidor?
 - **Resposta** do servidor **nunca voltou** ao cliente? Operação requisitada executou ou não?

Operação idempotente?

Operação que pode ser executada repetidamente com o mesmo efeito de ter sido executada apenas uma vez

- Retransmissão

- Sempre que ocorrer um timeout reenvia a mensagem

- Descarte de mensagens duplicadas

- Se retransmissão for utilizada, servidor pode receber **várias cópias**

Servidor já enviou resposta?

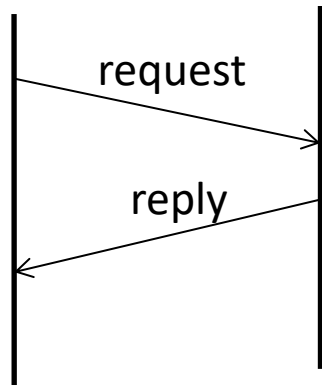
Se sim, executa operação novamente? Ou utiliza resultados da execução original?



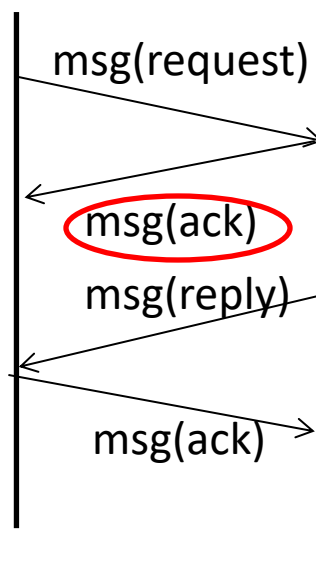
Protocolos request-reply

- Introdução ao modelo de invocação remota
 - Como processos ou objetos se comunicam em sistemas distribuídos
- Protocolos de request-reply
 - Troca de mensagens de forma bidirecional em modelos cliente/servidor
 - Fornecem suporte direto para implementação de RPC/RMI
 - Camada acima de comunicação entre processos, permitir transparência
 - Padrão de comunicação utilizado no topo de primitivas para troca de mensagens
- Normalmente, síncrono e confiável
 - Como implementar: TCP ou UDP?
 - Protocolo pode ser construído sobre UDP para evitar overhead do TCP
 - Acks, estabelecimento de conexão, controle de fluxo
 - Se UDP for usado, garantias de entrega devem ser providas pelo protocolo de request-reply

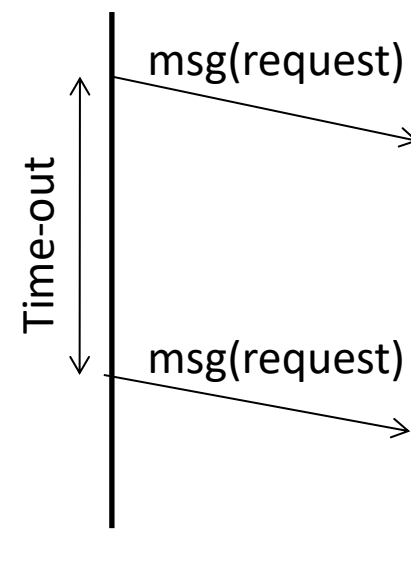
Fornecendo confiabilidade



Abstração de mais alto nível



Implementação com confirmações (explícitas)



Mensagens não confirmadas são retransmitidas (request ou reply)

Problema:

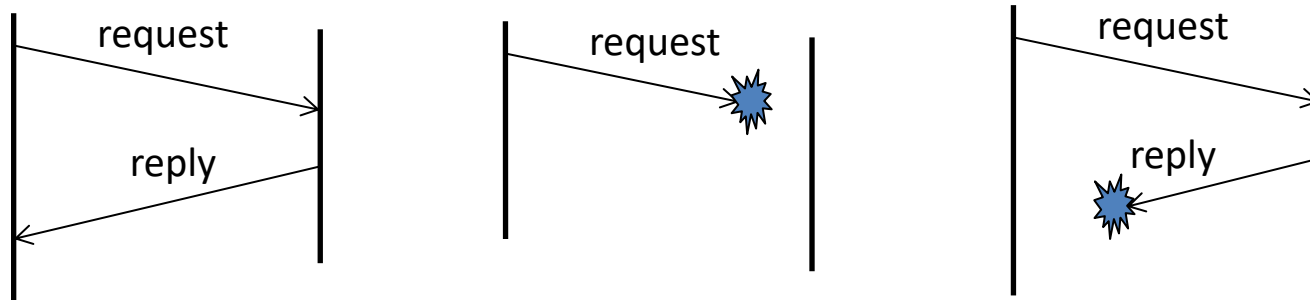
O que fazer se ack não chegar? A mensagem de *request* foi recebida? Foi a mensagem de *request* ou a mensagem de confirmação (*ack*) que deu problema?

Questão de semântica

- A semântica da comunicação depende de uma combinação de confirmações e retransmissões
 - Podem ser implementadas usando protocolos de request-reply
 - Opções de mecanismos para suportar diferentes garantias de entrega
 - Retransmissão de requisições?
 - Filtragem de mensagens duplicadas?
 - Retransmissão de resultados passados?
- Três semânticas possíveis baseadas em combinações de opções:
 - ***at most once*** (no máximo uma vez) → Modelo CORBA
 - ***at least once*** (pelo menos uma vez) → Modelo RPC
 - ***exactly once*** (exatamente uma vez) → Modelo Java RMI

Semântica no máximo uma vez (at most once)

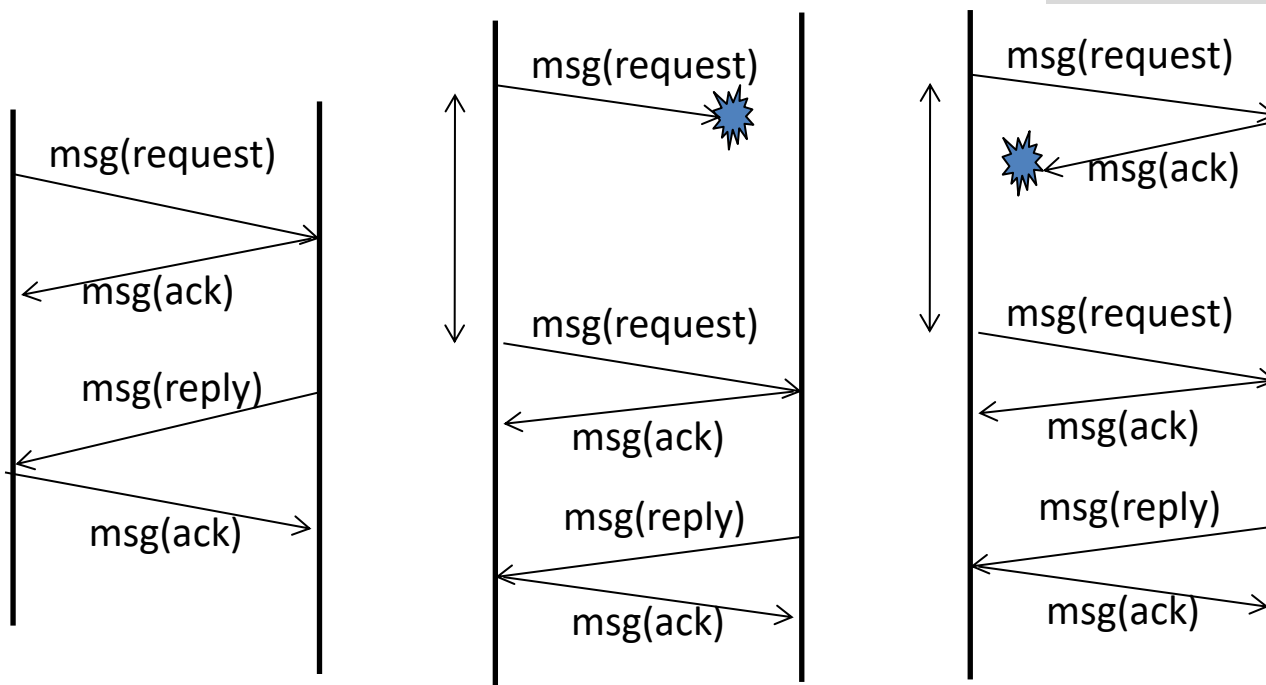
- O processo destino recebe nenhuma ou uma mensagem
 - Emissor não sabe se procedimento remoto executou ou não
- Não há nenhum tipo de confirmação ou retransmissão de mensagem
 - Método simples
 - Eficiência na comunicação já que não há *send* de confirmação ou retransmissão



Semântica pelo menos uma vez (at least once)

- Obtido com o uso de confirmações (ack) e retransmissões
 - Pode gerar mensagens duplicadas
- É garantido que o processo destino recebeu uma mensagem
 - Mas ele pode ter recebido mais de uma

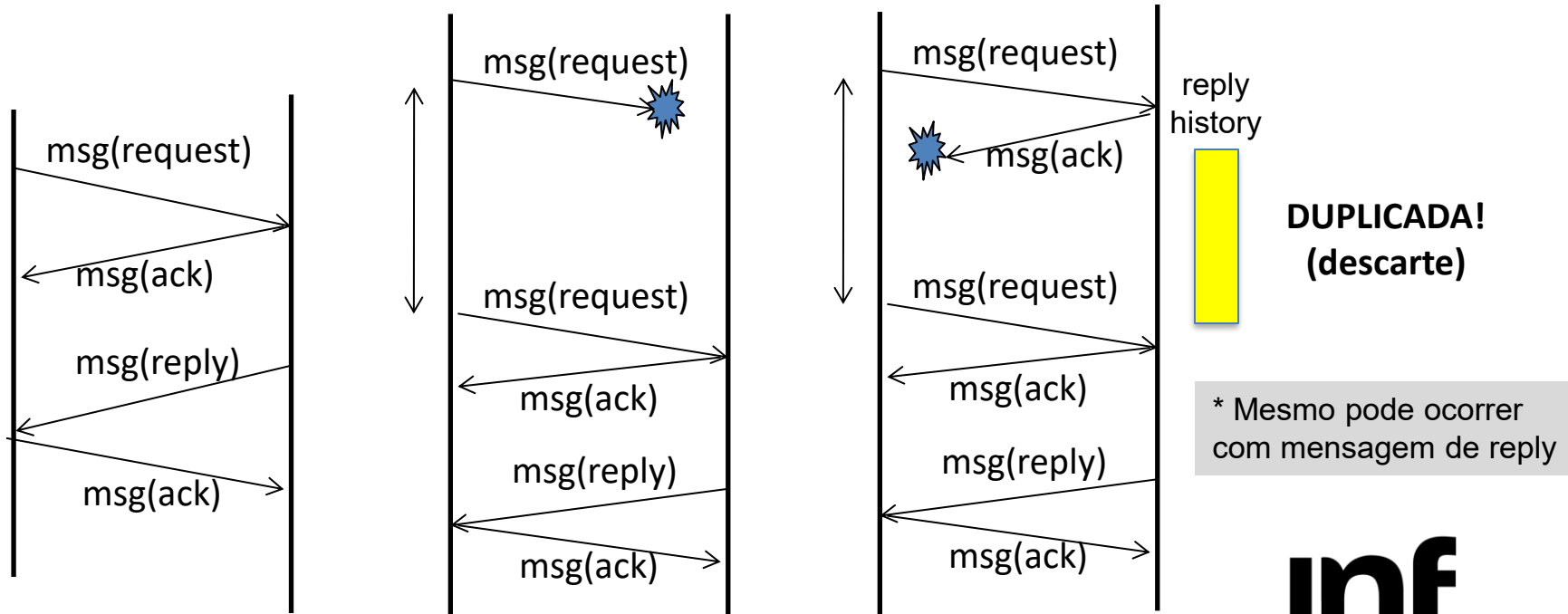
→ Quando isso não é um problema?



* Mesmo pode ocorrer com mensagem de reply

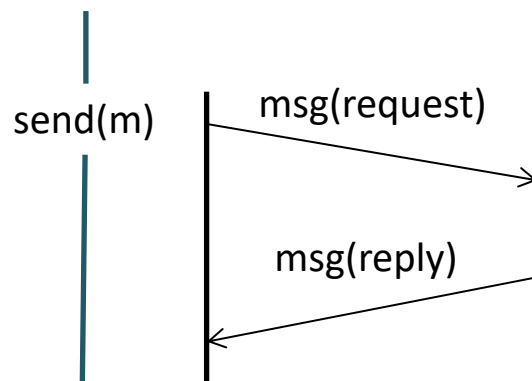
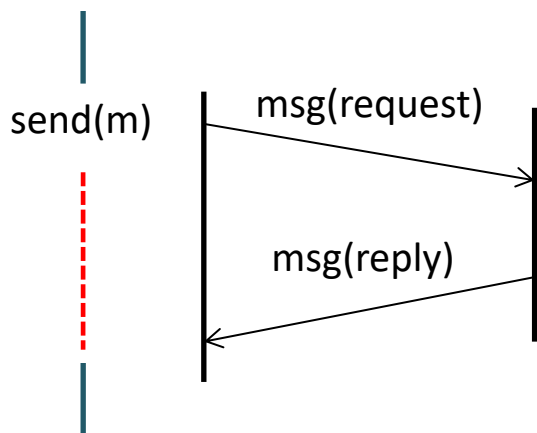
Semântica exatamente uma vez (exactly once)

- Obtida com o uso de confirmações (ack) e retransmissões
 - Mas protocolo deve reconhecer mensagens duplicadas e descartá-las
- Processo destino processa cada mensagem exatamente uma vez
 - Controle de ID, # sequência de mensagens, buferização de replies



Protocolo bloqueante e não bloqueante

- Protocolos síncronos (bloqueante) ou assíncronos (não bloqueante)
 - Diferença é se o processo emissor da mensagem de request é bloqueado até que ele receba um *reply* desta mensagem (não o ACK!!)
 - Envolvem questões de buferização do sistema
 - **Síncrono**: retorna somente após a recepção do *reply*
 - **Assíncrono**: retorna logo que o *request* for copiada para buffer do canal de comunicação



Estilos de protocolos [Spector, 1982]

- Produzem diferentes comportamentos na presença de falhas de comunicação
 - Request (R) protocol
 - Não há valor de retorno, cliente não precisa de confirmação, não bloqueia, implementado sobre UDP
 - Request-Reply (RR) protocol
 - Sem necessidades de acks pois resposta do servidor serve como confirmação
 - Request subsequente do cliente pode ser tratada como confirmação do reply
 - Pode **mascarar falhas** do UDP através de retransmissão, filtragem de duplicadas, histórico
 - Request-Reply-Ack (RRA) protocol
 - ID da requisição que vem no ack permite que o servidor descarte entradas do histórico

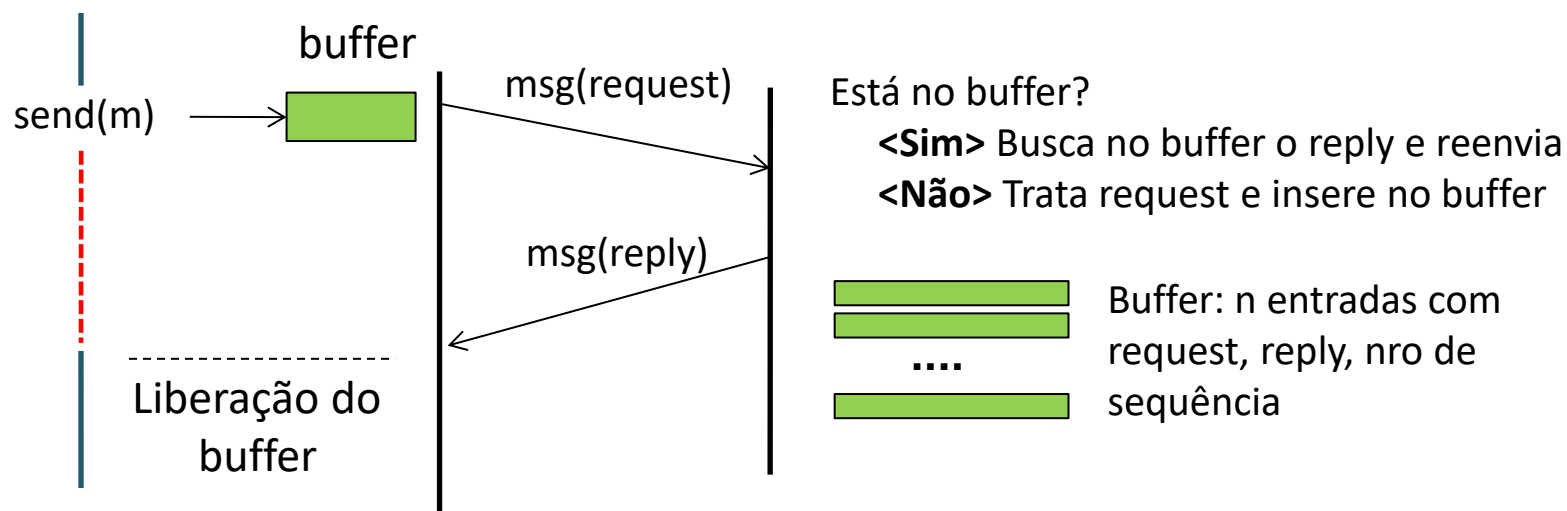
Protocolo	Cliente	Servidor	Cliente
R	Request		
RR	Request	Reply	
RRA	Request	Reply	Ack Reply

Protocolo Request-Reply (RR)

- Funcionamento:
 - Recepção do *reply* implica que destino recebeu o *request*
 - Sem confirmação explícita para o request
- Detalhes:
 - Implementa uma semântica ***at least once*** porque as mensagens de *request* podem ser duplicadas
 - Retransmissão de mensagens
 - Possível fazer uma semântica ***exactly once*** se descartar as duplicadas
 - Numerar sequencialmente os *requests*
 - Número de sequência é usado para identificar as duplicatas

Protocolo Request-Reply (RR)

- Versão bloqueante



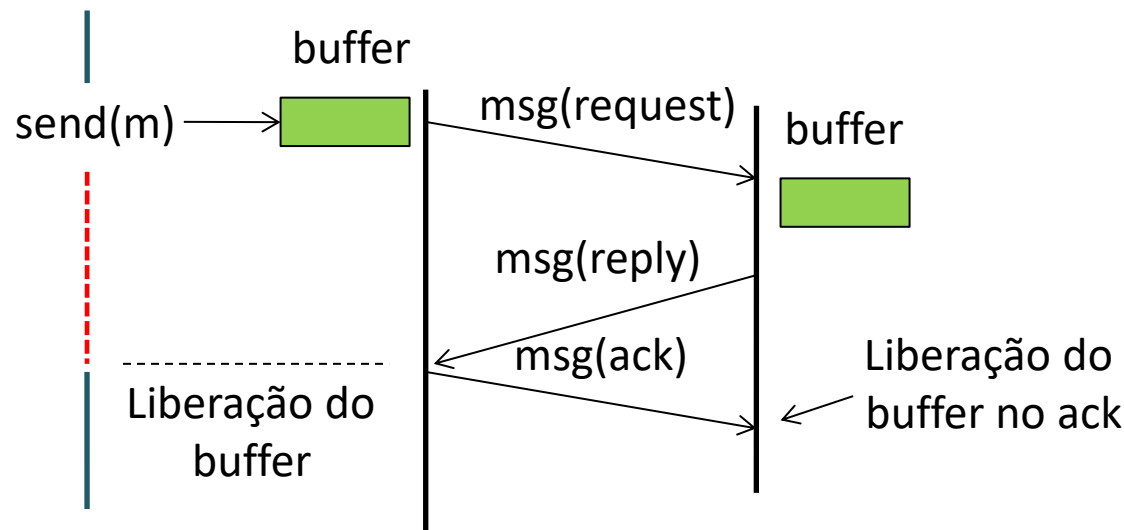
Problema?

Quando liberar as informações do buffer?
Noção de sistema *stateless* e *statefull*...

Protocolo Request-Reply-Ack (RRA)

- Funcionamento:
 - Recepção do *reply* implica que destino recebeu o *request*
 - Sem confirmação explícita para o request
 - A mensagem de reply **possui um ack explícito**
- Detalhes:
 - Implementa uma semântica **at least once** porque as mensagens de *request* podem ser duplicadas
 - Retransmissão de mensagens
 - Possível fazer uma semântica **exactly once** se descartar as duplicadas
 - Numerar sequencialmente os *requests*
 - Número de sequência é usado para identificar as duplicatas
- Quase igual ao RR, porém...

Protocolo Request-Reply-Ack (RRA)



- Implementação
 - Buffers são mantidos para o caso de necessitar retransmissão das mensagens
 - Se necessário ***exactly once***, necessário manter também os números de sequência

Características da implementação de request-reply sobre...

UDP

- Mensagens com tamanho limitado
 - Pode necessitar o uso de múltiplos datagramas para realizar um request
- Mensagens entregues de forma não confiável
 - Protocolo de request-reply precisa implementar as garantias
 - Implementação é mais difícil
- Menos overhead, se algumas funcionalidades TCP não são necessárias
 - Mensagens de tamanho fixo?
 - Todas as operações são idempotentes?
 - Perda de mensagens é tolerada?

TCP

- Mensagens sem limite de tamanho
 - Evita protocolos multipacket
- Mensagens entregues de forma confiável
 - Protocolo de request-reply não precisa lidar com retransmissões, filtragem, históricos
 - Implementação de request-reply é simples
- Protocolo TCP implica em overhead de comunicação

Slide extra: diferentes interpretações

- Semântica de invocação remota depende do modelo de falhas sendo considerado
 - Ocasional perda de mensagens, duplicações, mensagens fora de ordem

Como visto em aula*

- No máximo uma vez (*at-most-once*)
- Pelo menos uma vez (*at-least-once*)
- Exatamente uma vez (*exactly-once*)

* Não considerando falha permanente da rede ou do servidor

Coulouris⁺

- Talvez (*maybe*)
- Pelo menos uma vez (*at-least-once*)
- No máximo uma vez (*at-most-once*)

⁺ Considerando falha permanente da rede ou do servidor

- Claramente, se servidor falhar, a semântica *exactly-once* não será satisfeita...
 - **Coulouris**: considera a semântica *exactly-once* apenas para procedimentos locais

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 5 (seções 5.1, 5.2, 5.3)

INF01151 – Sistemas Operacionais II

Aula 14 - Remote Procedure Call - RPC

Prof. Alberto Egon Schaeffer Filho

Introdução

- Necessidade de desenvolver aplicações distribuídas de forma simples (intuitiva, transparente)
 - Solução: estender modelos de programação “convencionais”
 - Utilização de primitivas de mais alto-nível
- Três modelos:
 - **Procedural**: chamada remota de procedimentos (RPC)
 - **Orientado a objetos**: invocação remota de métodos (RMI)
 - **Baseado em eventos**: atendimento de eventos remotos
- Provido através de um *middleware* de comunicação

} Síncrono
} Assíncrono

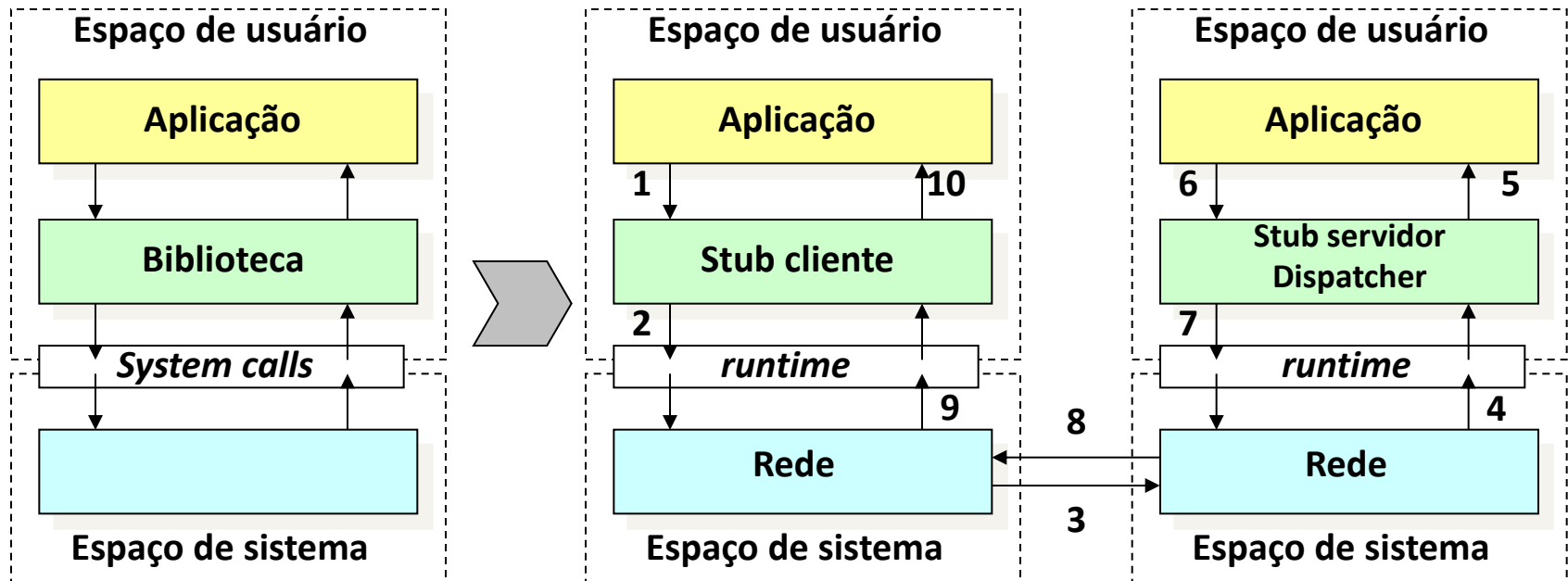
“Software que provê um modelo de programação oferecendo abstração de processos, comunicação e serviços através de primitivas próprias”

Aspectos importantes (ideais) de um *middleware*

- Transparência de acesso e de localização (uniformidade)
 - Mesmo comportamento (sintaxe? e/ou semântica?) independente dos procedimentos serem locais ou remotos
- Protocolo de comunicação (flexibilidade)
 - Comportamento independente do protocolo de transporte (garantias)
- Independência de plataforma (hardware e sistema operacional)
 - Uso de representação externa de dados (*marshalling* e *unmarshalling*)
 - Primitivas para criação de fluxos de execução, sincronização, etc, independentes do sistema operacional
- Uso de linguagens de programação
 - Permitir ou não o emprego de várias linguagens de programação no desenvolvimento da aplicação distribuída

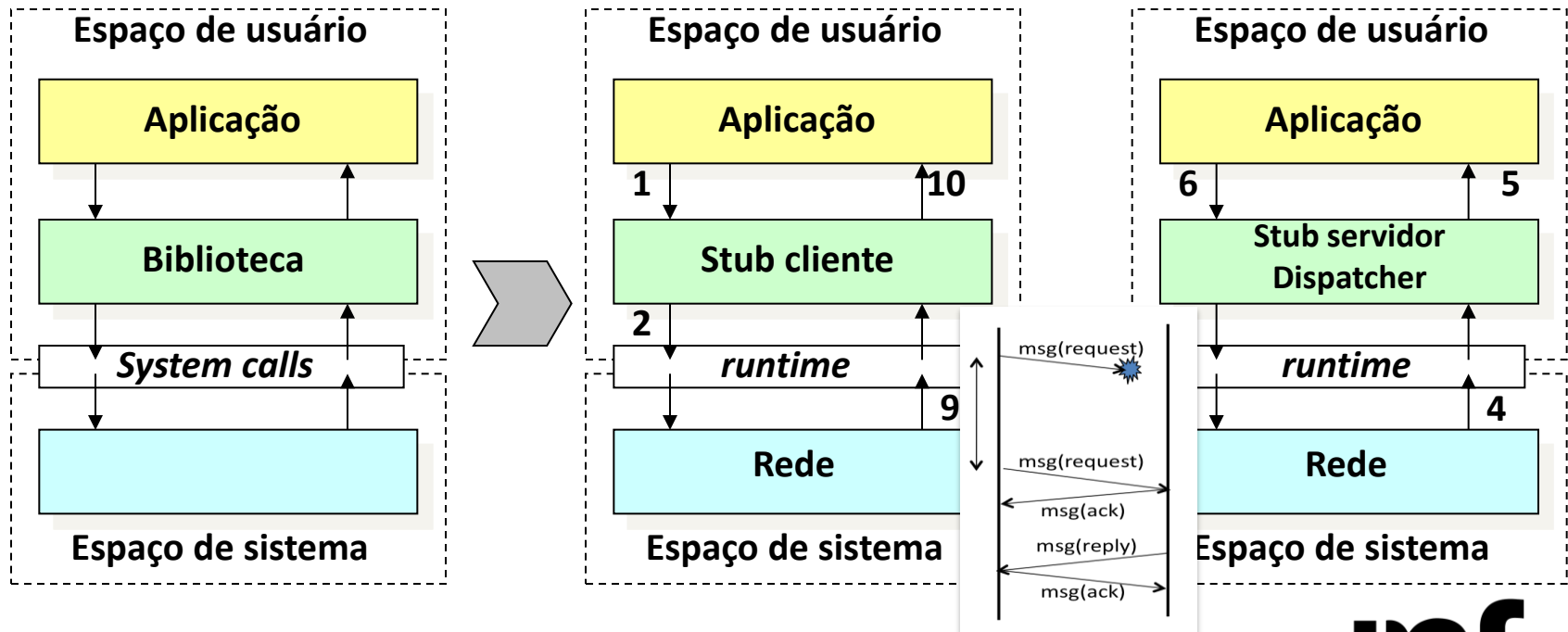
Remote Procedure Call (RPC)

- Generalização do conceito de subrotinas locais
 - Permite um programa chamar um procedimento que não se encontra no espaço de endereçamento do processo



Remote Procedure Call (RPC)

- Generalização do conceito de subrotinas locais
 - Permite um programa chamar um procedimento que não se encontra no espaço de endereçamento do processo



Implementação do mecanismo de RPC

- Oferece transparência sintática através de quatro elementos
 - *Runtime*,
 - *Stub* cliente,
 - *Stub* servidor,
 - *Dispatcher*
- *Middleware (runtime)*
 - Responsável pela troca de mensagens entre cliente e servidor
 - Implementa um protocolo da família request-reply
 - Necessário definir semântica de invocação, independente da camada de transporte
 - Necessário prover mecanismos para confirmação de recebimentos, retransmissões, time-out, controle de duplicações, históricos, etc
- *Stubs*: implementações “ocas” dos procedimentos
 - Responsáveis por permitir comunicação remota apenas
 - Não implementam “lógica de negócio/serviço”

Stub cliente / Stub servidor

- Programa cliente
 - Stub executa o *marshalling* dos argumentos
 - Envio de uma mensagem (requisição) ao processo servidor
 - Bloqueia até receber mensagem (resposta)
 - Stub efetua *unmarshalling* e repassa à aplicação
- Programa servidor
 - ***Dispatcher*** seleciona qual procedimento stub executar, conforme ID da requisição
 - Stub executa o *unmarshalling* dos argumentos
 - Chama o procedimento associado (chamada local)
 - Stub executa o *marshalling* do resultado
 - Envia mensagem (resposta) para o processo cliente

Geração de stubs:

- **Manualmente:** programador constrói os *stubs* e parâmetros
- **Automaticamente:** baseado em uma *Interface Definition Language* (IDL)
 - Compilador IDL gera código a ser integrado ao do programador

Programando com interfaces

- Maioria das linguagens permite organizar programas como conjuntos de módulos que podem se comunicar
 - Aplicação distribuída = conjunto de módulos que se comunicam entre si
 - Para controlar interações, definir uma **interface** para cada módulo
 - Apresenta apenas serviços que podem ser acessados, esconde-se o resto
 - Implementação dos serviços podem ser alteradas desde que interface permaneça
 - Serviços providos pelo servidor podem ser otimizados sem afetar clientes do módulo
- **Interface** = descrição de procedimentos que podem ser acessados por outros módulos
 - Nome e parâmetros de entrada e saída (inclui tipos)
 - Passagem de parâmetros por **referência** e/ou por **valor**??
 - Passagem de parâmetros é problemática em modelo distribuído
 - E.g.: não faz sentido passar ponteiros
 - Endereços não podem ser passados como argumento ou retornados como resultados de procedimentos

Semântica de passagem de parâmetros

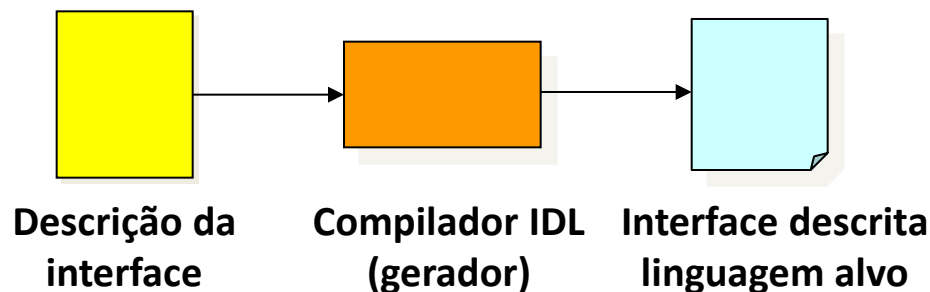
- Passagem **por valor**
 - Parâmetros são copiados na mensagem e enviados ao servidor
 - Não representa maiores problemas
- Passagem **por referência**
 - Sem sentido, em princípio, pois não há espaço de endereçamento comum
 - Soluções possíveis:
 - Empregar uma passagem de parâmetros do tipo *call-by-copy-restore*
 - Resolver o ponteiro a cada referência no servidor (mensagens adicionais)
 - Usar mecanismos de DSM (*Distributed Shared Memory*)

Solução na prática: descrever parâmetros na forma entrada/saída

- **Entrada:** enviados ao módulo remoto (mensagem de requisição)
- **Saída:** recebido pelo módulo solicitante (mensagem de resposta)

Interface Definition Language (IDL)

- Interface de serviços
 - Especificação de procedimentos + argumentos (entrada/saída)
 - Modelo RPC
 - Especificação de métodos + argumentos (entrada/saída, referência)
 - Modelo RMI
- Notação para definir interfaces
 - Integrado diretamente em uma linguagem
 - Java RMI
 - Mapeado (compilado) para uma linguagem específica
 - XDR para C, C++



Transparência: Localização de serviços

- *Stub* cliente necessita descobrir o servidor antes de realizar RPC
 - Para quem ele deve enviar a mensagem??
- **Servidor de nomes** [Birrel e Nelson, 1984]
 - Também chamado de agente de binding
 - Interface de nomes para serviços remotos
 - **Hostname**: servidor que implementa o serviço
 - Permite a existência de vários servidores
 - Se qualquer servidor pode prover o serviço, então define apenas o tipo (serviço)
 - **Tipo**: especifica a interface (serviço a ser usado)
 - Tipicamente a interface também oferece um **número de versão**
 - Distinção entre versões novas e antigas de um mesmo serviço
 - Manter as antigas durante fase de migração, sistemas legados, compatibilidade, etc

Perguntas:

- ① Como cliente especifica o agente de binding?
- ② Como o agente de binding localiza o serviço?
- ③ Quando a localização é realizada?
- ④ Possível trocar o binding durante execução (on the fly)?
- ⑤ Pode haver múltiplos bindings?

Agente de binding

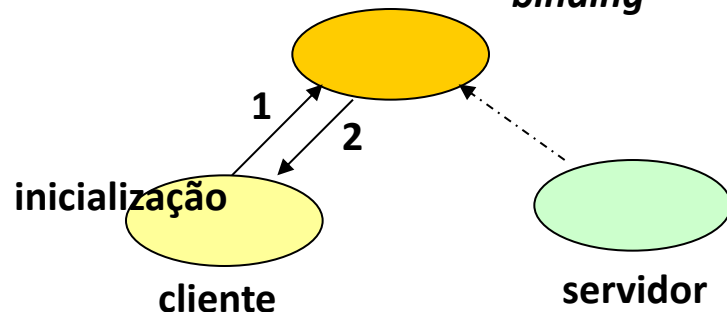
- Agente de amarração (*binding*) ou servidor de nomes
 - Permite que o cliente obtenha um **handler** para acessar o serviço
 - **Servidores se registram** no agente de binding na sua inicialização
 - **Cientes consultam** o agente de binding para saber quem provê o serviço
 - **Interface do agente de binding**
 - **register**: para servidor cadastrar um serviço no agente
 - **deregister**: para servidor descadastrar um serviço no agente
 - **lookup**: para o cliente localizar um determinado serviço
 - Como localizar um agente de binding?
 - Endereço fixo conhecido e informado a todos clientes e servidores
 - Empregar *broadcasting* para localizar o agente de binding

Quando a localização é realizada?

- Tempo de **compilação**
 - Localização dos serviços e dos servidores é *hard-coded*
- Tempo de **inicialização** do cliente
 - Cliente consulta o agente de binding e armazena resposta em uma cache
 - Chamadas subsequentes não envolvem consultas ao agente de binding
- Tempo de **execução** da chamada
 - Cliente consulta agente de binding na chamada do serviço
 - Agente de binding localiza servidor (possível fazer a chamada em nome do cliente)

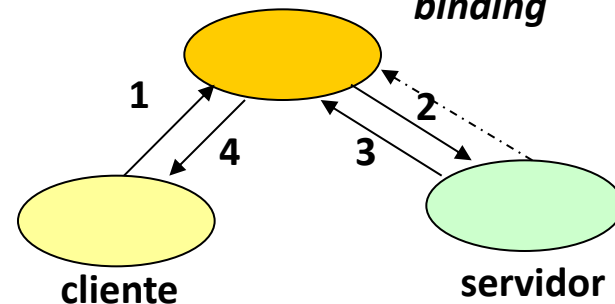
Tempo de inicialização

Agente de
binding



Tempo de execução

Agente de
binding



Estudo de caso: *binding* em sistemas Unix

- O agente de *binding* é o daemon **portmapper**

- É um serviço bem-conhecido (porta 111 – Sun RPC)

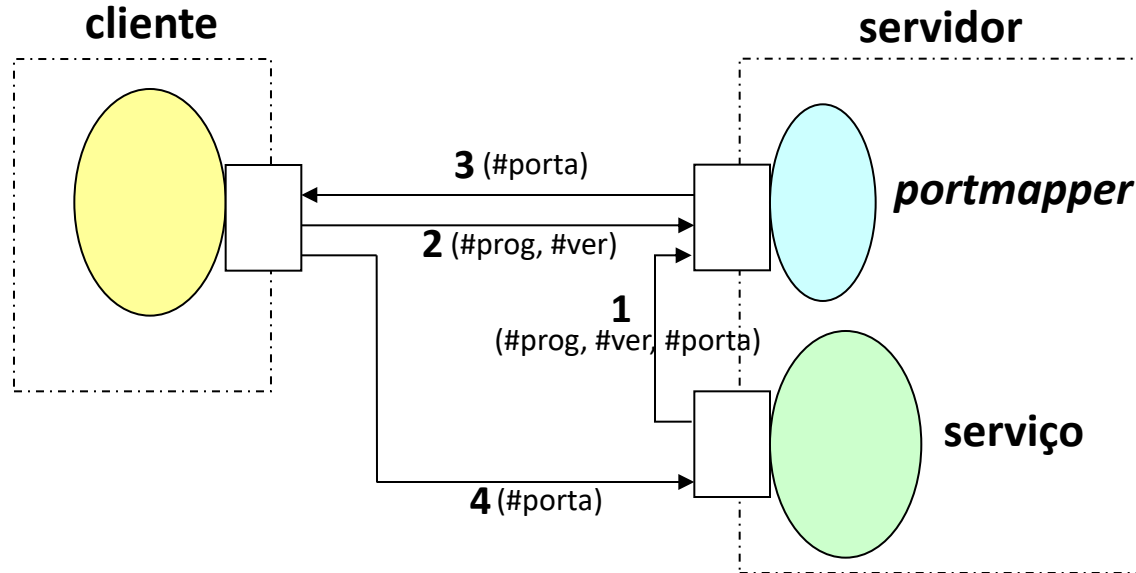
```
%>rpcinfo -p localhost
```

- Como determinar a máquina servidora?

- *Hard-coded* ou variável de ambiente

- *Broadcast* a todos os *portmappers*

program	vers	proto	port	
100000	2	tcp	111	portmapper
100000	2	udp	111	portmapper
50405925	1	udp	57164	
50405925	1	tcp	53652	



Remote Procedure Call *versus* Local Procedure Call

- Aspectos de aplicações distribuídas que **dificultam a questão da transparência**
 - Falhas parciais
 - Concorrência
 - Latência
 - Acesso a memória

Mais questões sobre transparência

- Em RPC fácil oferecer transparência **sintática**, porém não **semântica**
 - Acesso a **variáveis globais** (como fazer?)
 - Problema com passagem de **parâmetros por referência**
 - Vulnerabilidade a **falhas**
 - Mensagens (*request/reply*) podem ser perdidas
 - Servidor pode pendurar (antes ou depois de ter executado a requisição??)
 - Cliente pode pendurar (antes ou depois de ter aceito o resultado??)
 - A **concorrência é real** (processo local e processo remoto)
 - **Latência da rede** na execução
 - Tempo de execução é sensível a atrasos na rede
- Conclusão: **transparência semântica é difícil** de ser obtida...
 - Pergunta: desejável “esconder” que se trata de comunicação remota?
 - Necessita reagir adequadamente a latência, condições de falhas...
 - Explicitar diferença entre chamada local e remota nas interfaces

Execução de serviços remotos na presença de falhas

- Sistemas distribuídos são mais vulneráveis a falhas
 - Uma vez que envolvem a rede, outro computador e outro processo
- Pontos de falhas por colapso: Cliente? Servidor? Rede?
 - Problema: saber se falha ocorreu antes/depois de processar requisição
 - Impossível distinguir entre falha da rede ou falha do servidor

Falhas no servidor/rede

- Definir uma semântica de invocação
 - No máximo uma vez (*at-most-once*)
 - Pelo menos uma vez (*at-least-once*)
 - Exatamente uma vez (*exactly-once*)
- Implementar operações idempotentes
 - Idempotentes: podem ser realizadas n vezes fornecendo o mesmo resultado como tivessem sido executadas uma só vez
 - e.g.: ler um bloco de dados de um arquivo, inserir um elemento em uma posição fixa de um vetor, etc
 - Não idempotentes: o resultado final é modificado pela re-execução
 - e.g.: creditar ou debitar um valor em uma conta bancária, inserir ou remover um elemento no final de uma lista, etc
 - Operações idempotentes simplificam a execução de serviços remotos em caso de falhas

Falhas no cliente

- Cliente “pendura” após ter enviado a requisição
 - Reiniciar o cliente (processo e/ou máquina), porém consequência é se ter uma **computação orfã**
- Problemas com computações orfãs:
 - Desperdício de cpu no servidor, *locks* realizados no cliente e não liberados, confusão com a resposta que chega (quem fez?), problemas com números de seqüência, etc
- Soluções possíveis:
 - Extermínio
 - Reincarnação
 - Expiração

Falhas no cliente (continuação)

- Soluções possíveis
 - **Extermínio:**
 - Baseado em logs
 - Cliente loga tudo que executa e em caso de reboot anula todos resquícios de computações orfãs
 - Problema: tamanho do log em disco, e computações disparadas por orfãos
 - **Reincarnação:**
 - Cada reboot gera número (número de época) que é usado para identificar request e reply
 - Anula tudo que pertence a uma época anterior a atual
 - **Expiração:**
 - Cliente deve confirmar a recepção do resultado (commit)
 - Se servidor não receber commit, desfazer operação (transações)

Estudo de caso: Sun RPC

- Projetado para cliente/servidor no Sun NFS (Network File System)
 - Semântica adotada é ***at-least-once*** (ações no NFS são idempotentes)
 - Se usado em outro contexto, cabe ao usuário fazer tratamento adequado
 - Sun RPC é parte de **Sun OS** e outros sistemas operacionais **UNIX**
 - Implementações tem escolha de implementar RPC sobre **UDP** ou **TCP**
- Define uma linguagem (**XDR**) e compilador (**rpcgen**) para IDLs
- Características gerais da XDR:
 - Originalmente projetada p/ especificar representação externa de dados
 - Especifica definições de procedimentos, parâmetro in/ parâmetro out
 - Uso de structs para permitir mais campos
 - Não fornece servidor de nome global
 - Usa **portmapper**: servidor de nome local (porta x interface, versão)
 - Cliente deve conhecer a máquina que executa o serviço (endereço IP ou nome DNS simbólico)

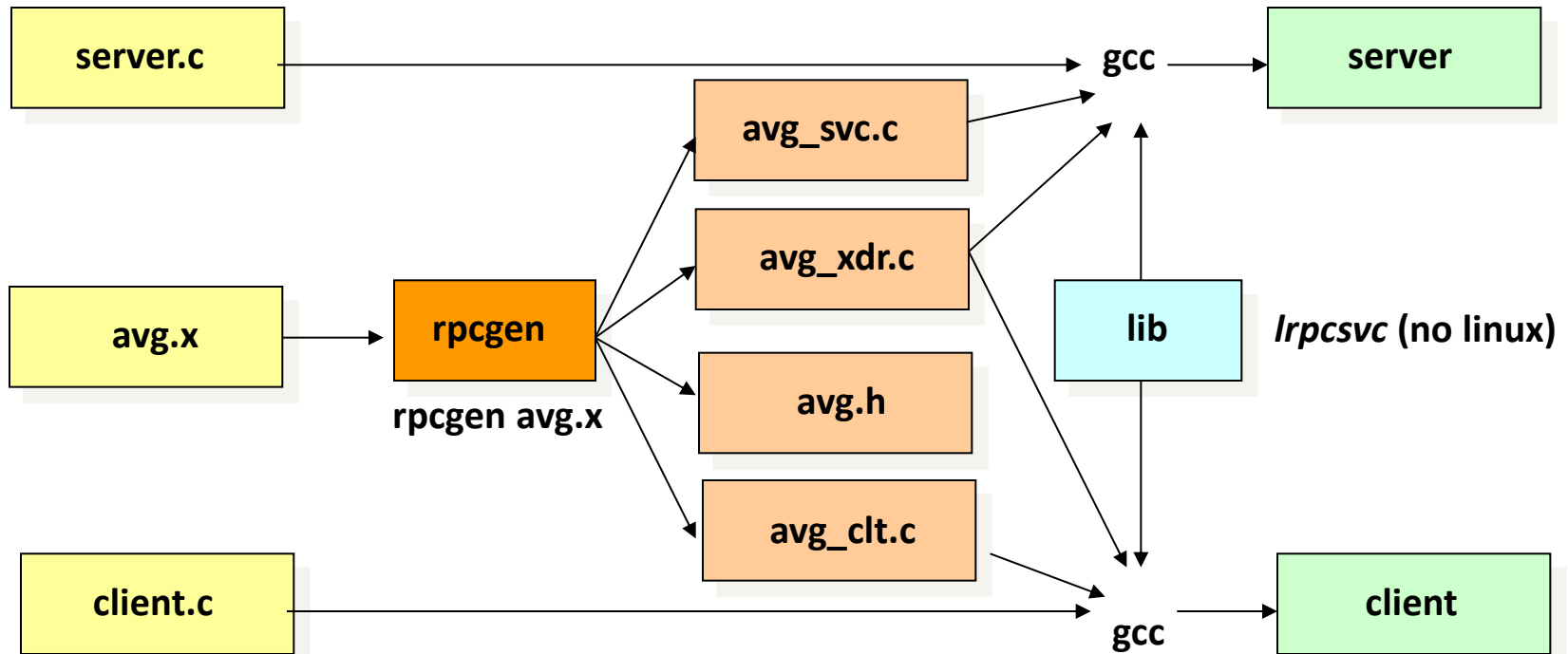
Program number
Version number

Desenvolvimento em Sun RPC

- Como usar um procedimento remoto? Passos:
 - ① Escrever uma definição de interface usando XDR
 - Notação primitiva, sintaxe baseada em C
 - Contém a especificação do procedimento remoto e parâmetro
 - ② Compilar com **rpcgen** gerando:
 - Stub cliente
 - Stub servidor
 - Arquivos .h
 - Duas funções de manipulação de parâmetros a serem usadas pelo cliente e pelo servidor (fazem o *marshalling* e o *unmarshalling*)
 - ③ Programa cliente é escrito, compilado e ligado junto com o stub cliente
 - usa o arquivo .h
 - ④ Programa servidor é escrito, compilado e ligado junto com o stub servidor
 - usa o arquivo .h

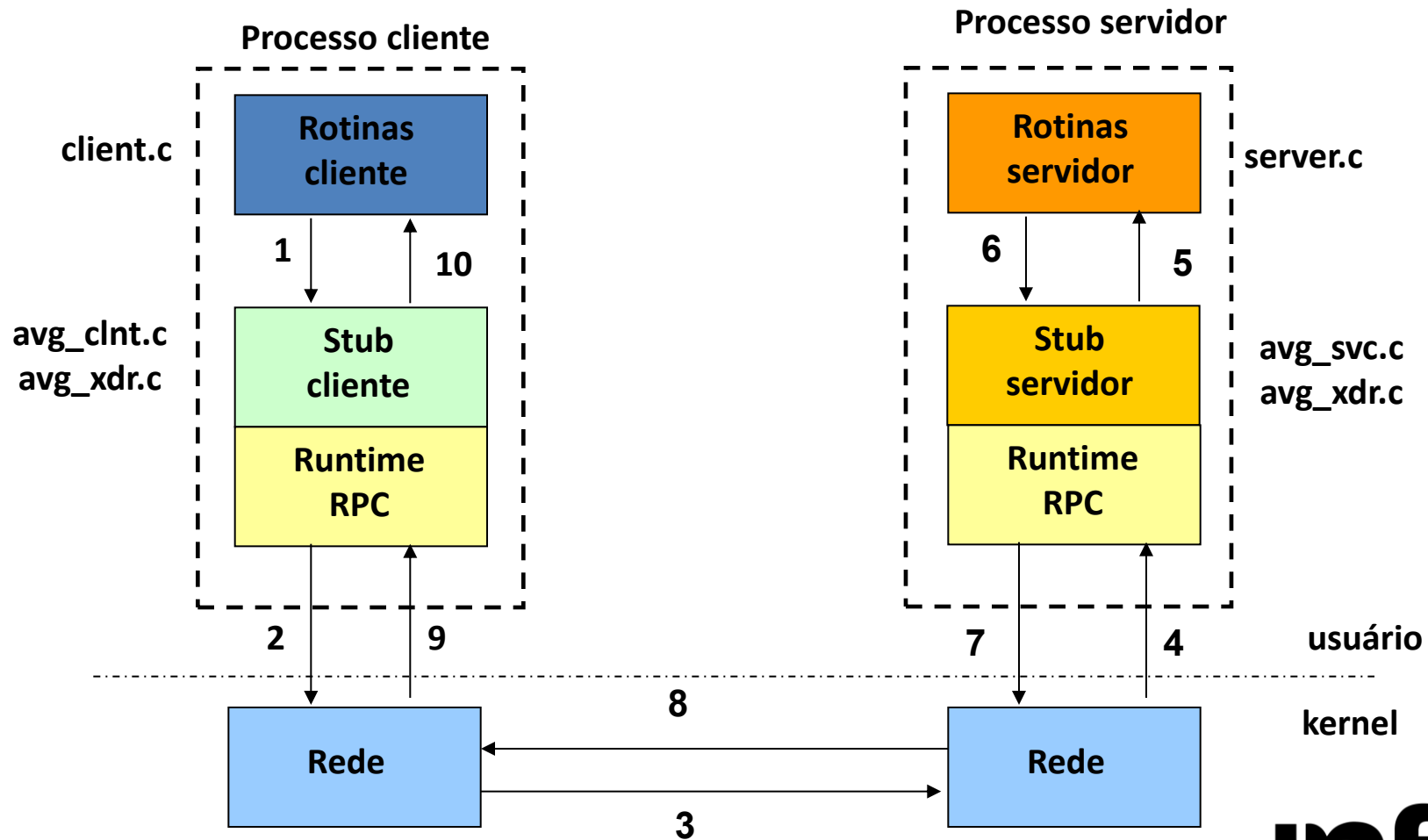
Exemplo: `rpcgen` (*Sun* RPC IDL)

```
gcc -o server server.c avg_svc.c avg_xdr.c -lrpcsvc
```



```
gcc -o client client.c avg_clt.c avg_xdr.c -lrpcsvc
```

Passos envolvidos na execução



Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulos 5 (seção 5.3)
- Remote Procedure Calls – Linux Journal (1997)
 - <http://www.linuxjournal.com/article/2204>

INF01151 – Sistemas Operacionais II

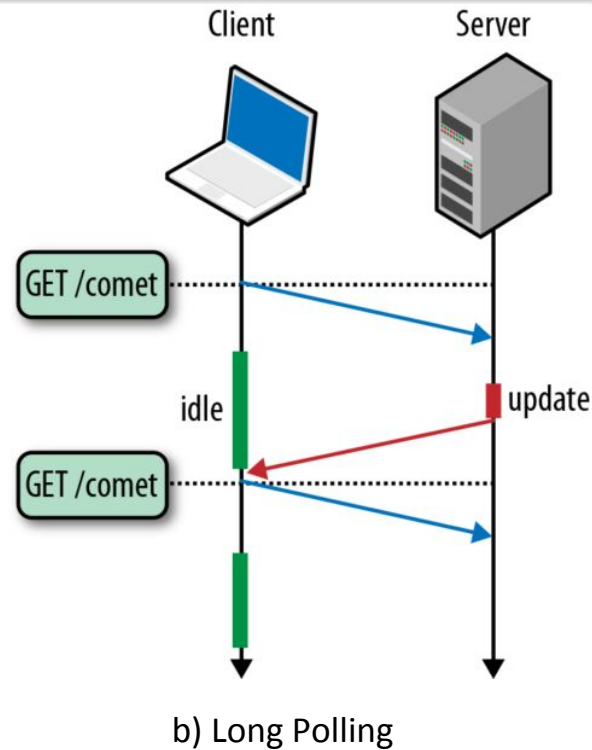
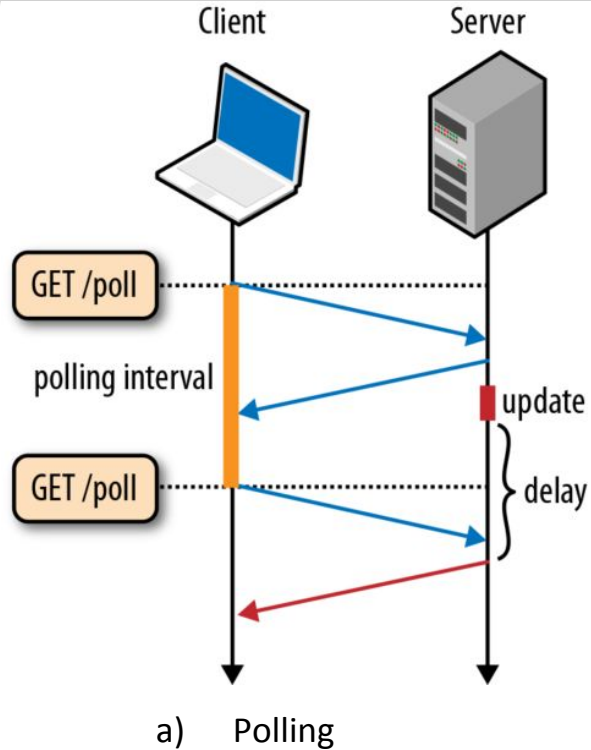
Aula 15 - WebSockets

Prof. Alberto Egon Schaeffer Filho

Introdução

- **Problema:** Criar aplicações WEB (*client-server*) que necessitavam de comunicação bidirecional requeria que o servidor ficasse sendo consultado de forma contínua em busca de atualizações. (*polling*)

Introdução



Introdução

Essa abordagem acarreta alguns problemas:

- **Servidor com inúmeras conexões TCP para cada cliente:** tanto para enviar informações e para receber as requisições
- **Consumo de Largura de banda:** requisições sendo respondidas mesmo quando nenhum dado novo está disponível
- **Aplicações-cliente:** Mapeamento das várias conexões feitas às conexões de resposta.

Introdução

- **A solução:** Utilizar uma única conexão TCP para o tráfego de forma bidirecional, sem utilizar *polling* HTTP. Assim, têm-se o protocolo **WebSocket**, bem como a sua API (WSAPI).

WebSockets: a definição

“O protocolo WebSocket permite a comunicação em duas vias entre um cliente, executando código não-confiável em um ambiente controlado, a um host remoto que aceitou estabelecer comunicação a esse código.” [RFC 6442]

WebSockets: a definição

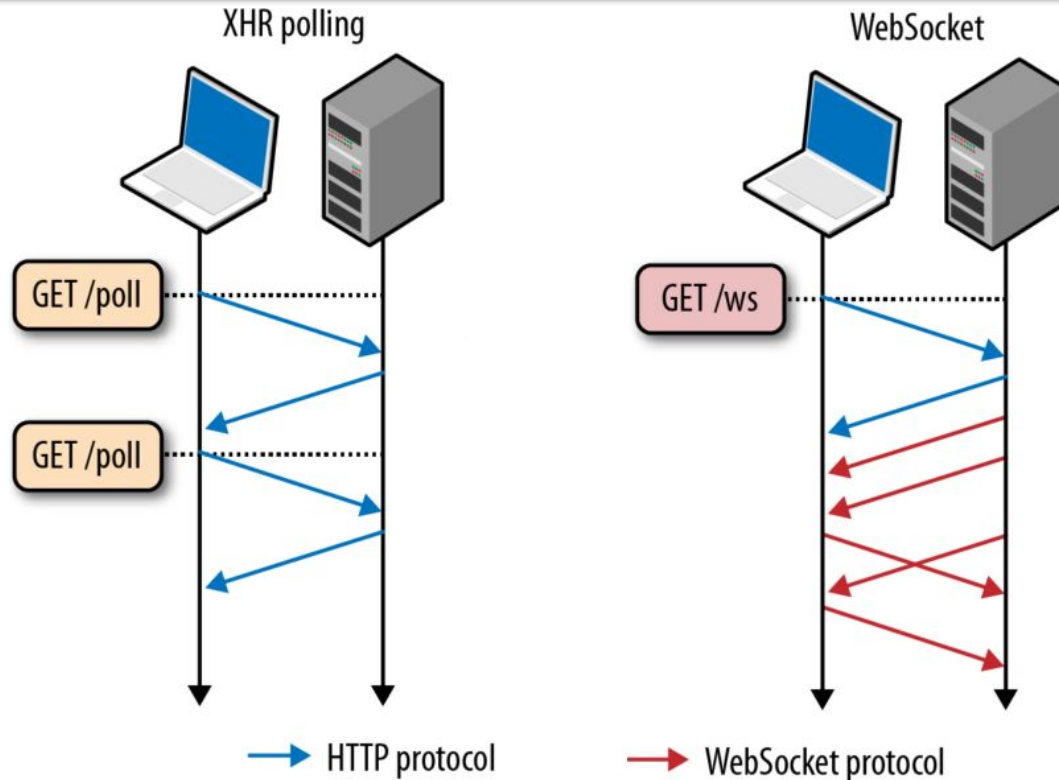
O protocolo WebSocket é um protocolo independente baseado em TCP, que utiliza HTTP para processar o handshake entre as entidades.

Utiliza a porta 80 para conexões regulares e 443 para conexões com TLS (Transport Layer Security), por padrão.

Objetivos de WebSockets

Resolver os empecilhos encontrados nas soluções HTTP existentes que implementavam comunicação bidirecional: Prover uma maneira para as aplicações estabelecerem comunicação em duas vias com servidores, sem depender da abertura de múltiplas conexões HTTP.
(ex: *XMLHttpRequest*, *long polling*, ...)

Objetivos de WebSockets

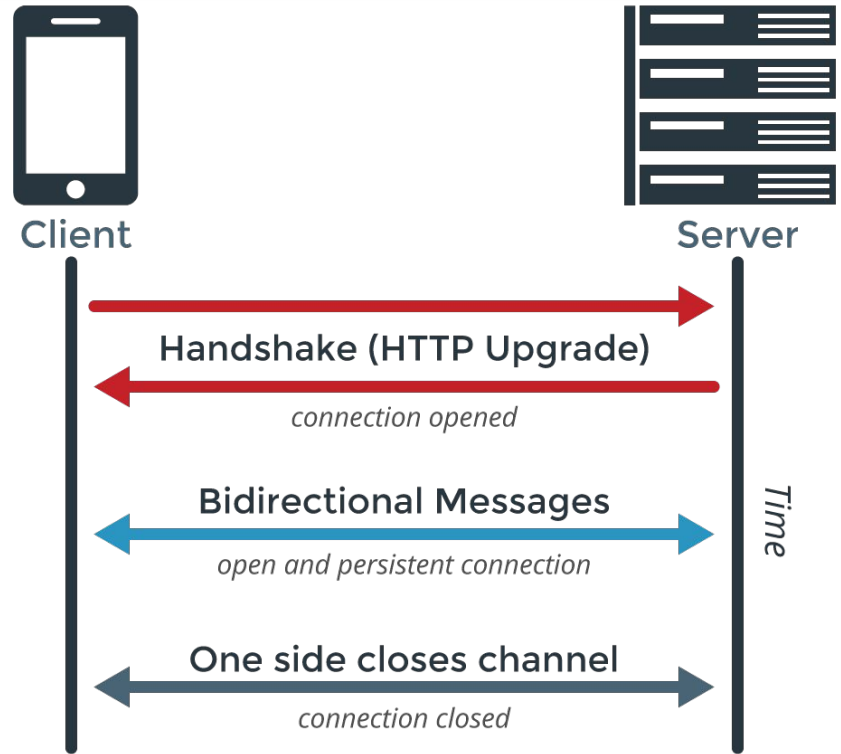


WebSockets: estrutura

O protocolo possui duas partes essenciais: o *handshake* (para estabelecimento da conexão entre ambas as partes) e o envio de dados.

Após o envio do *handshake* por ambas as partes (e se foi bem sucedido), ambas as entidades estão aptas a enviar dados através do canal estabelecido.

WebSockets: estrutura



WebSockets: estrutura

Opening Handshake

- O *handshake* cliente é uma requisição de Upgrade HTTP
- A ordem dos campos no *header* não importa

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Origin: http://example.com
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
```

WebSockets: estrutura

Opening Handshake (client)

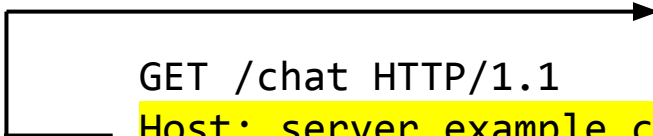
“Request URI”: identificar o endpoint da conexão WebSocket. Para permitir vários domínios servidos por um único endereço IP e múltiplos endpoints servidos por um único servidor.



```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Origin: http://example.com
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
```

WebSockets: estrutura

Opening Handshake (client)



```
GET /chat HTTP/1.1
```

```
Host: server.example.com
```

```
Upgrade: websocket
```

```
Connection: Upgrade
```

```
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
```

```
Origin: http://example.com
```

```
Sec-WebSocket-Protocol: chat, superchat
```

```
Sec-WebSocket-Version: 13
```

Cliente inclui o *hostname* na requisição para que ambos (cliente e servidor) possam concordar qual host estará em uso.

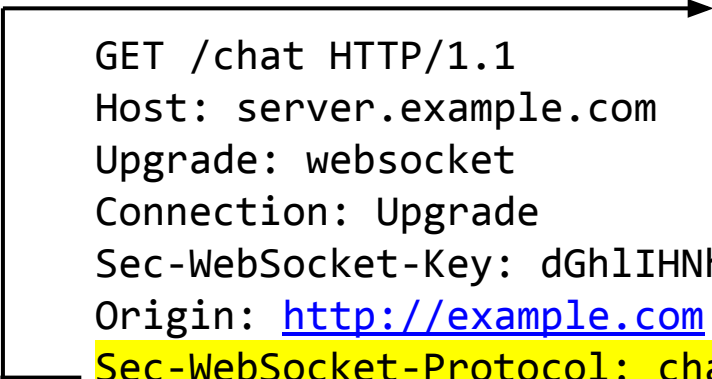
WebSockets: estrutura

Opening Handshake (client)



WebSockets: estrutura

Opening Handshake (client)

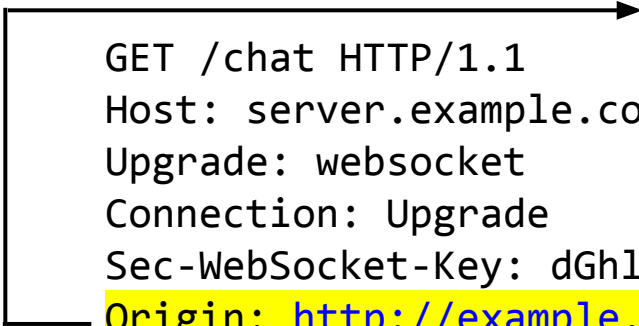


```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Origin: http://example.com
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
```

Subprotocol Selector: campo adicional para indicar quais subprotocolos (a nível de aplicação) são aceitos pelo cliente. O servidor seleciona qual usará e o mencionará no seu handshake para indicar que o selecionou.

WebSockets: estrutura

Opening Handshake (client)

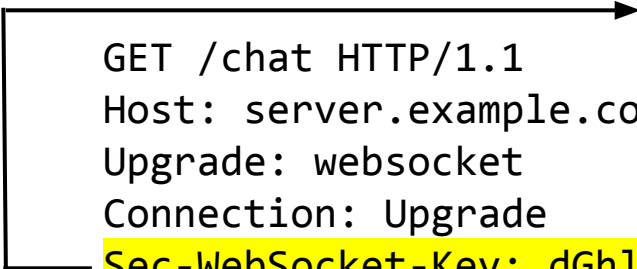


```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Origin: http://example.com
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
```

Informa o servidor sobre a origem da requisição.
Se o servidor não deseja estabelecer conexão com
essa origem, responde com um código de erro
HTTP para rejeitá-la.

WebSockets: estrutura

Opening Handshake (client)



```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Origin: http://example.com
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
```

Para provar que o handshake foi recebido, o servidor pega o valor desse campo, concatena-o com seu identificador (GUID), aplica hash SHA-1, codifica com Base64 e o retorna em seu próprio *handshake*.

WebSockets: estrutura

Opening Handshake (server)



HTTP status line: retorna o código HTTP 101 para sinalizar ao cliente qual protocolo foi aceito. Qualquer código diferente de 101 indica que o handshake WebSocket não foi concluído e que a semântica do HTTP ainda é aplicável.

HTTP/1.1 101 Switching Protocols

Upgrade: websocket

Connection: Upgrade

Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=

WebSockets: estrutura

Opening Handshake (server)

Indica o aceite da parte do servidor em manter conexão. Contém a concatenação da hash do cliente, presente em **Sec-WebSocket-Key**. Esse campo será verificado pelo cliente e se não for o valor esperado, a conexão não será estabelecida.

HTTP/1.1 101 Switching Protocols

Upgrade: websocket

Connection: Upgrade

Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=

WebSockets: estrutura

Opening Handshake: requisitos para o cliente

- Prover campos válidos para a requisição
- Se um cliente já possui conexão para um host identificado por <host><port> ele deve esperar ela ser estabelecida ou falhar para realizar outra como mesmo esquema. Não deve existir mais de uma conexão no estado CONNECTING.

WebSockets: estrutura

Opening Handshake: requisitos para o cliente

Não há limites para o número de conexões WebSocket estabelecidas que um cliente possa ter com um mesmo servidor, porém servidores podem recusar estabelecer conexão com hosts com número excessivo de conexões já estabelecidas, quando sobrecarregados.

WebSockets: estrutura

Opening Handshake: requisitos para o servidor

- Processar parte do *opening handshake* do cliente para extrair informações necessárias.
- Verificar violações de formato nos dados recebidos
- Retornar códigos de erro caso encontre violações;
- Se aceitar a conexão do cliente, retornar a ele uma resposta HTTP válida.

WebSockets: estrutura

WebSockets URIs

- Define dois esquemas para URIs: ws e wss

ws-URI = “ws:” “//” host[“:” port] path [“?” query]

wss-URI = “wss:” “//” host[“:” port] path [“?” query]

ex: “ws://example.com/home”

WebSockets: *Framing*

- Dados são enviados em sequências de quadros (*frames*)
- O cliente deve mascarar os *frames* enviados
- O servidor não deve mascarar os *frames* enviados ao cliente.

WebSockets: *frames*

- *Control frames*
 - Ping
 - Pong
 - *Close frame*
- *Data frames*
 - *Text*
 - *Binary*

WebSockets: Envio de dados

Para o envio de dados sobre uma conexão WebSocket:

- Assegurar que a conexão está aberta, ou abortar o envio.
- Encapsular os dados em um quadro WebSocket. Se o tamanho for muito grande, encapsular em uma série de *frames*
- Cliente deve mascarar os dados.

WebSockets: Receber dados

- “Escutar” na conexão estabelecida
- Checar o tipo dos dados recebidos (*data frames*)
- Processar *control frames*
- Observância dos FIN bits
- Remover o mascaramento dos dados feito pelo cliente

WebSockets: *closing handshake*

- Deve-se utilizar um método que feche a conexão de forma segura.
- Realizar o envio de um *close frame* identificando a razão da finalização da conexão.
- Após o envio e recebimento do *close frame*, o endpoint deve fechar a conexão WebSocket.

WebSockets e HTTP

- WebSocket utiliza HTTP/1.1 para o processamento do Handshake.
- HTTP/2 não menciona WebSockets em sua definição
 - Porém possui melhorias para HTTP/1.1
 - Algumas de suas *features* podem ser confundidas como uma substituição do protocolo WebSocket.

WebSocket VS. HTTP/2.0

	HTTP/2	WebSocket
Headers	Compressed (HPACK)	None
Binary	Yes	Binary or Textual
Multiplexing	Yes	Yes
Prioritization	Yes	No
Compression	Yes	Yes
Direction	Client/Server + Server Push	Bidirectional
Full-duplex	Yes	Yes



Permite múltiplos fluxos de dados sobre uma única conexão TCP

WebSocket VS. HTTP/2.0

	HTTP/2	WebSocket
Headers	Compressed (HPACK)	None
Binary	Yes	Binary or Textual
Multiplexing	Yes	Yes
Prioritization	Yes	No
Compression	Yes	Yes
Direction	Client/Server + Server Push	Bidirectional
Full-duplex	Yes	Yes

Server Push: Servidor encaminha à cache do cliente dados disponíveis, sem necessidade de uma requisição. Porém, cabe à aplicação a monitoração dos eventos originados pelo servidor.

Leituras adicionais

- [RFC 6455] - The WebSocket Protocol.
<<https://tools.ietf.org/html/rfc6455>>
- [RFC 6202] - Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP.
<<https://tools.ietf.org/html/rfc6202>>

INF01151 – Sistemas Operacionais II

Aula 16 - Web services (SHORT VERSION)

Prof. Alberto Egon Schaeffer Filho



Web services

- Definição W3C
 - “Um **web service** é um sistema **identificado por uma URI** cujas interfaces são definidas e **descritas usando XML**. Sua definição **pode ser descoberta** por algum outro sistema ou software. Esses sistemas devem **interagir** com um serviço web de forma presente em sua definição utilizando **mensagens baseadas em XML** transmitidas **através de protocolos da Internet**.”
- Porém, com a popularização da arquitetura REST, pode-se definir web service como “qualquer sistema desenvolvido para suportar operações entre computadores através de uma rede



Introdução

- Necessidade de comunicação cliente/servidor entre sistemas heterogêneos, com necessidade de interoperabilidade
 - Ex: DCOM, Java RMI, Sun RPC, etc
 - Problema: “conversam naturalmente” apenas consigo mesmo, isto é, apenas com software homogêneo
- Solução: explorar formato e/ou arquitetura padrão para troca de informações
 - XML-RPC (XML sobre HTTP)
 - SOAP (XML + protocolo de comunicação)
 - REST (métodos HTTP + XML/JSON)

evolução



Web service

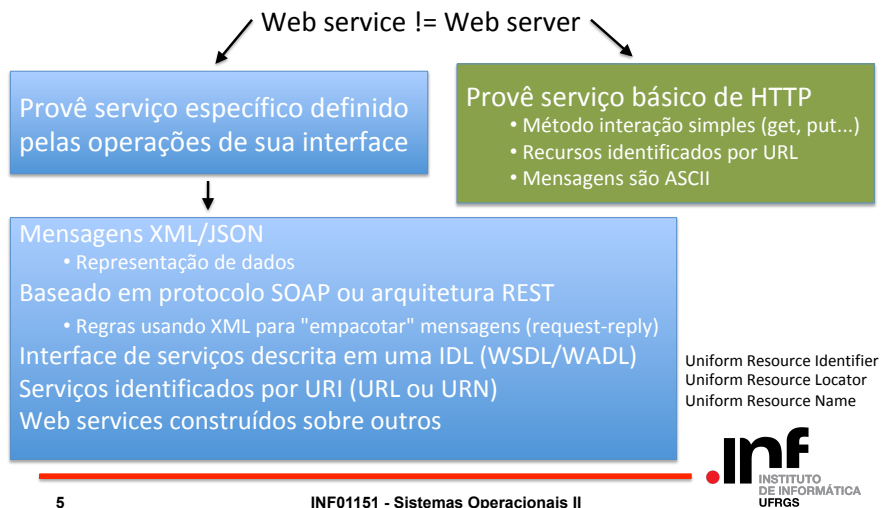


Objetivos dos web services

- Substituir os protocolos atuais (RPC, DCOM e RMI) por uma abordagem aberta:
 - Interoperabilidade baseada na web
 - Reusável
 - Independente de plataforma/linguagem
- Permitir a interação de componentes heterogêneos, remotos e independentes através de um protocolo (SOAP) ou arquitetura (REST) padrão
- Voltado para organizações
 - Integração B2B (Business to Business)
 - EAI (Enterprise Application Integration)
 - P2P (Peer-to-Peer)



Web services vs. Web server



Web services vs. Web server

- HTTP é um protocolo request-reply
 - Web server
 - São acessados por um browser
 - Provê um serviço HTTP básico (métodos padrões HTTP)
 - Acessados por URL
 - Web service
 - Sem interação humana
 - Provê serviços baseados nesta interface
 - Acessados por URI
- URI (Uniform Resource Identifier)
 - URL (Uniform Resource Location)
 - Fornece a localização de um recurso
 - URN (Uniform Resource Name)
 - Independe de localização
 - Serviço de lookup para transformar URN em uma URL

Web services

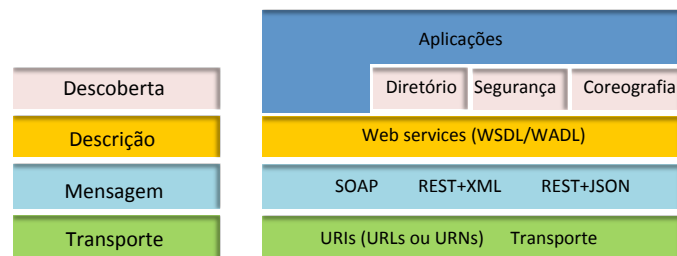
- Característica chave:
 - Processamento de mensagens XML empacotadas em SOAP
 - Alternativa: Arquitetura REST (*Representational State Transfer*)
 - Clientes usam URL e HTTP (get, put, delete e post) para manipular recursos
 - Ênfase em manipulação de dados ao invés de interfaces
- Implantação de serviços web
 - Gerenciado por um servidor web com páginas web; ou
 - Aplicação totalmente separada de um servidor web
 - Utiliza seu próprio *service description* para lidar com características específicas das mensagens recebidas
 - Por exemplo, web services providos pela amazon.com (SOAP ou REST)
 - Permitir que clientes obtenham características sobre um produto
 - Adicionar produto ao carrinho
 - Verificar status de uma transação

Composição de web services

- Combinar operações de uma interface com interfaces de outros serviços para oferecer novas funcionalidades (agente de viagens)
 - Reservar vôos, hotéis, aluguel de carros usando diferentes web services



Web services: infra-estrutura e componentes



Detalhes:

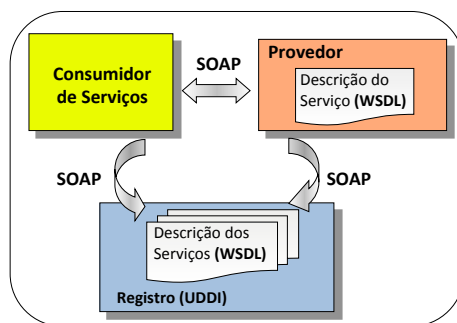
- Representação externa de dados e marshalling feito em XML ou JSON (readability/debugging)
- Transporte é um protocolo de aplicação da Internet (**HTTP**, SMTP, FTP, etc)
- SOAP: regras para usar XML no empacotamento de mensagens (em request-reply protocols)
 - Encapsula mensagens e transmite elas sobre HTTP ou outro protocolo
- Web services construídos sobre outros, podem prover funcionalidades mais gerais
 - Descoberta envolve serviços de diretórios e de nome (UDDI, ebXML, ...)

E mais concretamente?

- Uma nova tecnologia para objetos distribuídos
 - Invocação remota de WS: SOAP (≈ RMI, RPC)
 - Descrição de WS: WSDL (≈ IDL)
 - Registro e descoberta de WS: UDDI (≈ rmiregistry, portmapper)
- Baseado em padrões XML
 - W3C: XML, SOAP, WSDL
 - Industriais: UDDI, ebXML
 - Proprietários: Disco, WSDD, WSFL, ASMX,...
- Implementações atuais:
 - Microsoft .NET
 - Sun JavaONE: J2EE+WebServices (JAXP, JAX-RPC, JAXM...)
 - Apache XSOAP
 - Oracle, Bea, Iona, Enhydra...

Arquitetura de web services

- Baseado em arquiteturas orientadas a serviço (*service oriented architecture* – SOA)
- Três componentes:
 - Consumidor
 - Registro
 - Provedor
- Tecnologias
 - SOAP
 - WSDL
 - UDDI



XML – eXtensible Markup Language

- Prover mecanismo para identificação de dados visando a troca de informações
 - Formatação e filtragem de dados
 - Manipulação de informações
- XML está presente nos principais níveis da pilha tecnológica
 - Formatação de mensagens SOAP
 - Descrição de serviços (WSDL)
 - Publicação de serviços (UDDI)
- XML *schema* é um conjunto de regras para definição de elementos
 - Modelo abstrato de dados

JSON – JavaScript Object Notation

- Alternativa a XML para troca de mensagens em web services
 - Formato originalmente derivado da linguagem JavaScript
 - Requer menos recursos computacionais e de rede comparado a XML
 - Não necessita de validação/*schema*
 - Facilmente incorporado a linguagens populares (JS, Java, Python..)
 - Legibilidade boa (debugging)
 - Suporta hierarquia e tipos de dados compostos (i.e. Listas)
- XML ainda é necessário para descrição de serviços REST (WADL)

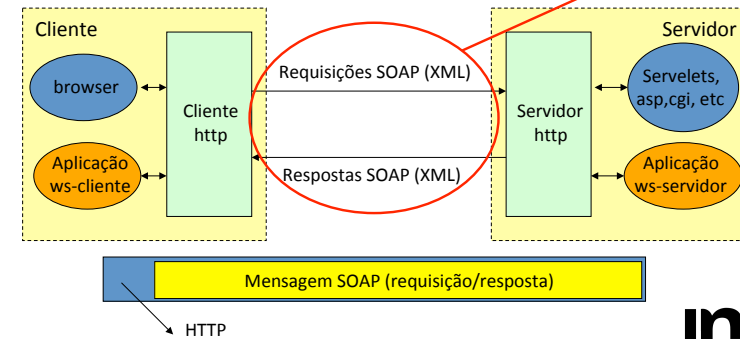
13

INF01151 - Sistemas Operacionais II

Simple Object Access Protocol (SOAP)

- Projetado para suportar interação entre web services
 - Empacotar mensagens em XML
 - [Tipicamente] usa HTTP para comunicar mensagens

Especifica protocolo
para troca de
mensagens em XML



14

INF01151 - Sistemas Operacionais II

Especificação SOAP

- O que define o padrão SOAP?
 - Como XML é usado para representar conteúdo de mensagens
 - Como mensagens podem ser combinadas para fazer request-reply
 - Regras de processamento do XML pelo receptor da mensagem
 - Como HTTP (e outros) são usados para comunicar mensagens SOAP
 - Mensagem SOAP definida em um envelope
 - header: contexto para um serviço
 - E.g., transaction ID, username, public key
 - body: XML para um web service em particular
 - Documento a ser comunicado diretamente no corpo (assíncrono)
 - Mensagem de request ou reply (síncrono)
- ✓ Mensagens SOAP NÃO contém referência ao destinatário

15

INF01151 - Sistemas Operacionais II

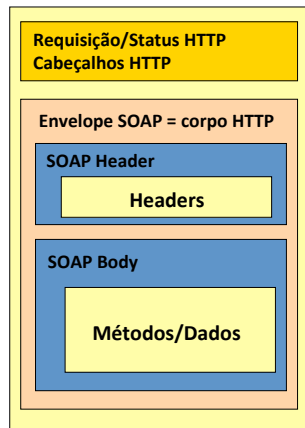
Transporte de mensagens SOAP

- Mensagem SOAP não contém referência sobre seu destino
 - Necessidade de “alguém” fornecer o destino
 - Protocolo de transporte
- Protocolo de transporte
 - Indicar quem deve tratar a URI (URN ou URL)
 - HTTP é tipicamente empregado
- SOAP request
 - Método HTTP GET:
 - Para obter informações (sem argumento, sem ações)
 - Método HTTP POST:
 - Para realizar ações no servidor ou enviar argumentos

16

INF01151 - Sistemas Operacionais II

Mensagem SOAP sobre HTTP



- Uma comunicação por mensagem
- Estrutura de mensagem
 - Envelope
 - Moldura base
 - Fornece o *namespace*
 - Cabeçalho (Header)
 - Opcional
 - Suporte a transações, sessões, segurança
 - Corpo (Body)
 - Elementos da mensagem
 - Métodos, argumentos, valores de retorno
 - Elementos de erro (*fault*)

17

INF01151 - Sistemas Operacionais II

Mensagem SOAP sobre HTTP

```
POST /examples/stringer
Host: www.cdk4.net
Content-Type: application/soap+xml; charset=utf-8
Content-Length: 299
SOAPAction: "http://www.cdk4.net/examples/stringer#GetStockPrice"
```

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
  </soap:Header>
  <soap:Body>
    <m:GetStockPrice xmlns:m="http://www.example.org/stock">
      <m:StockName>IBM</m:StockName>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>
```

18

INF01151 - Sistemas Operacionais II

REpresentational State Transfer

- Arquitetura REST visando web services
 - Principal foco é em como prover acesso a recursos de maneira interoperável, fácil e *lightweight*
 - Faz uso dos métodos HTTP para realizar operações (GET, POST, DELETE)
 - Questões de segurança cabem ao protocolo de transporte (HTTPS)
 - Diferentemente de SOAP, permite representação de dados em formatos distintos (texto puro, XML, JSON, YAML)
 - Não necessita guardar estado entre operações (cada requisição é autocontida)
- URI utilizada para expor lógica de serviços
 - Pode conter informações como namespace, nome do serviço e opcionalmente parâmetros
 - Informações adicionais passadas via HTTP POST

19

INF01151 - Sistemas Operacionais II

Web services x objetos distribuídos

- Comparação web services vs. modelo de objetos distribuídos
 - Superficialmente, interação cliente/servidor é muito parecida
 - RMI: cliente utiliza referência remota para invocar método no objeto remoto
 - Web services: cliente usa URI para invocar operação no recurso correspondente
- Referências remotas x URI
 - Web service não pode criar novas instâncias e passá-las como parâmetros
 - Web service consiste em um único objeto remoto
 - Garbage Collection não se aplica
 - Referência remota não se aplica
- Organizações usam firewalls para proteger suas redes locais
 - Protocolos usados por Java RMI e CORBA normalmente não poderiam passar pelo firewall
 - Por outro lado, normalmente firewalls permitem tráfego HTTP e SMTP

20

INF01151 - Sistemas Operacionais II

SOAP API

- Como proteger o programador de detalhes?
 - Marshalling, unmarshalling
 - Sintaxe para invocação remota
- SOAP API: disponível em várias linguagens (Java, Python, C++)
- SOAP API para Java: JAX-RPC
 - Interface do web service deve estender Remote (como em RMI)
 - Mas, objeto que implementa interface remota **NÃO** pode ser passado como parâmetro
 - Compiladores usados para gerar stubs e descrição do WS (em WSDL)
 - wscompile & wsdeploy
 - Arquitetura similar aquela utilizada no middleware RMI
 - Porém, não existe módulo de referência

Nada disso é provido por infraestrutura de web services

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.–
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 9